

Server Manage 운영 위키

DECS 서버 관리 코드의 기능, 경계와 설계 의도

문서 기준: 2026-07-17 저장소 상태

소스: /home/jy/server_manage/wiki

목차

1. 전체 구조
2. container-images
3. kerberos-nfs
4. monitoring
5. remote-operations
6. server-state
7. user-lifecycle

System

원본: md/system/index.md

개요

System 영역은 **DECS GPU 서버와 사용자 container를 일관된 상태로 운영하기 위한 시스템 관리 코드**를 담당한다. 새로운 서버를 기존 서버와 같은 설정으로 구축하고, 사용자가 사용할 container와 image를 관리하며, 서버 전원부터 monitoring과 공유 storage 접근까지 운영에 필요한 전체 흐름을 유지하는 것이 목적이다.

주요 담당 범위는 다음과 같다.

- 모든 GPU 서버의 Docker, NVIDIA, Kubernetes와 network 공통 설정을 점검하고 신규 서버에도 같은 기준을 적용한다.
- 사용자 container의 생성·수정·삭제와 공통 container image를 관리한다.
- 서버 원격 부팅과 부팅 순서를 관리하여 사용자 관리 DB와 GPU 서버가 올바른 순서로 준비되도록 한다.
- 서버, GPU, container와 사용자 상태를 수집하여 장애와 이상 상태를 관측한다.
- AD/Kerberos 인증을 사용해 사용자가 NAS/NFS 공유 storage에 안전하게 접근하도록 한다.

즉, Backend나 Frontend 애플리케이션 기능이 아니라 **GPU 서버에서 사용자 작업 환경이 실제로 생성되고, 실행되고, 관측되고, storage에 연결되는 운영 기반**을 관리하는 영역이다.

목차

구성요소	상세 문서	PDF
전체 통합 매뉴얼	-	PDF 열기
전체 구조	현재 페이지	PDF 열기
container-images/	문서 열기	PDF 열기
user-lifecycle/	문서 열기	PDF 열기
server-state/	문서 열기	PDF 열기
remote-operations/	문서 열기	PDF 열기
monitoring/	문서 열기	PDF 열기
kerberos-nfs/	문서 열기	PDF 열기

container-images

원본: md/system/container-images/index.md, md/system/container-images/design.md, md/system/container-images/operations.md

역할: DECS GPU 컨테이너 이미지의 빌드 입력, 시작 시 런타임 구성, 검증을 소유한다.

이 문서는 container-images 문서의 시작점이다. **설계**는 이미지 variant manifest, Dockerfile, entrypoint 구성과 **user-lifecycle** 과의 계약을 설명하고, **운영**은 빌드·배포에 사용하는 명령, 테스트 전략과 변경 절차를 설명한다.

문서 구성

문서	핵심 내용
현재 페이지	책임 범위와 문서 안내
설계	디렉터리 지도, manifest 기반 variant, Dockerfile/entrypoint 구성, user-lifecycle 계약
운영	빌드/배포 흐름, 테스트 전략, 변경 가이드, 운영 안전 수칙

처음 보는 사람은 **설계 문서**에서 이미지가 무엇을 책임지고 무엇을 책임지지 않는지 먼저 확인하고, 실제로 빌드하거나 새 variant를 추가해야 할 때 **운영 문서**로 이동한다.

container-images 설계

개요 · 운영

역할: DECS GPU 컨테이너 이미지의 빌드 입력, 시작 시 런타임 구성, 검증을 소유한다.

1. 책임 범위

container-images 는 CUDA/TensorFlow 조합별로 이미지를 만들되, 그 안의 구성(계정, SSH, Jupyter 등)은 모든 이 이미지에서 동일하게 유지한다. 이미지가 책임지는 것은 OS·CUDA·Python/Jupyter 패키지와 컨테이너 안의 계정, SSH, VNC(선택), Kerberos ccache 사용 환경이다.

이미지는 UID/GID를 결정하지 않는다. 사용자 identity, 포트, 홈 디렉터리와 Kerberos keytab은 **user-lifecycle** 와 호스트가 관리한다. 이 경계를 지켜야 이미지가 사용자 DB나 특정 서버에 종속되지 않는다.

2. 디렉터리 지도

경로	핵심 기능
Dockerfile	모든 variant가 공유하는 단일 이미지 정의
image-variants.json	base image, CUDA/TensorFlow, 최소 driver, alias의 권위 있는 목록
entrypoint.sh	실행 시 사용자·그룹·SSH·Jupyter·VNC·Kerberos 환경 구성
scripts/variant_matrix.py	manifest를 Docker/GitHub Actions build matrix로 변환
scripts/build_variants.py	로컬 build/push 명령 생성과 실행
scripts/test_image_variants.py	이미지 메타데이터, TensorFlow와 선택적 GPU smoke test
scripts/test_uid_create_container.py	user-lifecycle CLI와의 dry-run 계약 검증
tests/	root_squash, Kerberos, sudo와 build 설정 회귀 테스트
.github/workflows/docker-publish.yml	PR merge 또는 수동 실행 시 matrix build/push

3. 핵심 기능

3.1 manifest 기반 이미지 variant

image-variants.json 한 곳에 다음 항목을 둔다.

- variant ID와 base NVIDIA CUDA image
- CUDA, TensorFlow, Python, Ubuntu 버전
- TensorFlow 설치 패키지와 conda 패키지 제약
- 필요한 최소 NVIDIA driver
- stable 또는 experimental 지원 상태
- 날짜가 없는 사용자용 alias

현재 안정 계열은 CUDA 11.8, 12.2, 12.3, 12.5이며 CUDA 12.8/H200 계열은 실장비 검증 전까지 experimental이다. 빌드 도구와 GitHub Actions가 같은 manifest를 읽으므로 로컬과 CI의 variant가 달라지지 않는다.

3.2 단일 Dockerfile

Dockerfile은 BASE_IMAGE, CUDA_VERSION, TENSORFLOW_VERSION, MIN_NVIDIA_DRIVER, CONDA_PACKAGES 등을 build argument로 받는다. 모든 variant는 공통으로 SSH, Kerberos client, 한국어 글꼴/입력기, Chrome, Xfce, TigerVNC/noVNC, Miniforge, Jupyter를 포함한다.

variant별 Dockerfile을 복제하지 않은 이유는 보안 패치나 공통 패키지 변경을 한 번만 적용하고, 차이는 manifest의 데이터로 검토하기 위해서다. 버전별 예외가 필요하면 먼저 manifest 값으로 표현하고, 정말 다른 설치 흐름일 때만 Dockerfile에 조건을 추가한다.

3.3 실행 시 identity 구성

`entrypoint.sh`의 주요 흐름은 다음과 같다.

1. 이미지 variant와 실제 host driver 정보를 출력한다.
2. `USER_ID`, `UID`, `GID`, `USER_GROUP`을 검증하고 기존 홈의 owner와 맞춘다.
3. supplemental group과 sudo 정책을 구성한다.
4. SSH 로그인과 Jupyter 설정을 만든다.
5. Kerberos 모드이면 전달받은 ccache를 사용자 환경에 연결한다.
6. Jupyter를 시작하고, 요청된 경우에만 VNC/noVNC를 시작한다.

호스트 mount가 `root_squash`인 환경에서는 root가 사용자 홈을 임의로 `chown`할 수 없다. 그래서 이미 결정된 UID/GID로 사용자를 실행하고, 홈에 쓸 수 없는 root 작업을 최소화한다. 사용자 홈의 기존 비밀번호와 VNC password 파일도 재시작 때 보존한다.

3.4 제한적 sudo와 공유 helper

기본 `DECS_USER_SUDO_MODE=restricted`는 패키지 설치에 필요한 제한된 sudo는 허용하지만 UID 전환, mount, 권한 변경, root shell과 우회 가능한 interpreter 실행을 막는다.

사용자 그룹 공유는 관리자 sudo 대신 `group-dir-share` helper로 제공한다. 사용자 자신의 권한으로 디렉터리를 만들고 `2770`과 default ACL을 설정하므로 공유 기능과 identity 격리를 함께 유지한다.

Kerberos keytab과 ccache를 분리한 이유와 container credential 경계는 [Kerberos/NFS의 keytab과 ccache 모델](#)을 따른다.

3.5 GPU 호환성 표시와 강제 모드

이미지는 시작할 때 build에 기록된 최소 NVIDIA driver와 `nvidia-smi` 결과를 출력한다. 기본값은 경고 중심이며 `STRICT_CUDA_COMPAT=true`일 때만 최소 driver보다 낮은 host에서 시작을 실패시킨다.

기본을 강제 실패로 두지 않은 이유는 GPU가 없는 검증 환경이나 긴급 진단 container까지 막지 않기 위해서다. 운영 배포에서 호환성을 반드시 보장해야 하면 strict mode를 명시한다.

4. 사용자 생명주기와의 계약

`uidctl create-container`는 image 이름과 version tag, 최종 UID/GID, 포트와 home mount를 Docker 환경변수/옵션으로 전달한다. 이미지가 기대하는 주요 값은 다음과 같다.

값	의미
<code>USER_ID</code>	컨테이너 사용자 이름
<code>UID</code> , <code>GID</code>	DB·AD·NFS와 이미 일치해 검증된 숫자 identity
<code>USER_GROUP</code>	primary group 이름
<code>ENABLE_VNC</code>	VNC/noVNC opt-in

값	의미
KRB5CCNAME	컨테이너에 보이는 ccache 경로
DECS_KERBEROS_HOST_KEYTAB	호스트 관리 ticket을 기다리는 모드
DECS_USER_SUDO_MODE	disabled, restricted, allowed 중 하나

TARGET_UID 나 TARGET_GID 같은 두 번째 identity 체계를 만들지 않는다. 하나의 UID / GID 만 사용해야 container, NFS와 DB가 어긋나는 오류를 조기에 발견할 수 있다.

container-images 운영

개요 · 설계

1. 빌드와 배포 흐름

```
image-variants.json
|
+--> variant_matrix.py --> GitHub Actions matrix --> build/push
|
+--> build_variants.py -----> local build/push
|
+--> test_image_variants.py -----> smoke test
```

전체 명령 확인:

```
cd /home/jy/server_manage/container-images
python3 scripts/build_variants.py --dry-run
```

특정 variant build:

```
python3 scripts/build_variants.py \
  --variant cuda12.5-tf2.20-ubuntu22.04
```

registry push는 build 결과와 tag 목록을 확인한 뒤 --push 를 추가한다. 날짜 tag는 재현 가능한 배포 지점을 만들고, stable / latest alias는 현재 권장 variant를 가리킨다. 운영 기록에는 alias보다 날짜가 포함된 tag를 남기는 것이 좋다.

2. 테스트 전략

빠른 정적/회귀 테스트:

```
cd /home/jy/server_manage/container-images
bash tests/test_image_build_config.sh
bash tests/test_entrpoint_root_squash.sh
```

이미지 smoke test:

```
python3 scripts/test_image_variants.py \
  --variant cuda12.5-tf2.20-ubuntu22.04
```

실제 GPU까지 확인할 때만 `--gpu` 를 사용한다. 사용자 생성 계약은 실제 DB를 바꾸지 않는 `dry-run/print` 경로로 검증한다.

```
python3 scripts/test_uid_create_container.py \
  --variant cuda12.5-tf2.20-ubuntu22.04 \
  --print-only
```

3. 변경 가이드

새 variant 추가

1. `image-variants.json` 에 버전, base image와 최소 driver를 추가한다.
2. `python3 scripts/variant_matrix.py` 결과의 tag를 확인한다.
3. build `dry-run`과 shell 회귀 테스트를 실행한다.
4. 실제 image smoke test와 가능하면 대상 GPU host test를 수행한다.
5. 검증 전에는 `support: experimental` 과 명시적인 alias를 사용한다.

공통 런타임 변경

`Dockerfile` 변경과 `entrpoint.sh` 변경을 구분한다. package/파일은 build 시점에 넣고, 사용자 UID나 mount처럼 컨테이너마다 달라지는 항목만 `entrpoint` 에서 처리한다. `entrpoint` 변경은 재시작 시 idempotent한지, 기존 홈의 파일을 덮어쓰지 않는지, `root_squash` 환경에서 동작하는지를 함께 확인한다.

4. 운영 안전 수칙

- 실제 사용자 비밀번호를 image layer나 manifest에 넣지 않는다.
- keytab을 image 또는 container mount로 제공하지 않는다.

- `latest` 만 기록하지 말고 장애 분석이 가능한 날짜 tag를 남긴다.
- `experimental variant`를 `stable alias`로 바꾸기 전에 실장비 GPU test를 한다.
- `sudo` 완화는 `ccache`와 `host bind mount`에 대한 우회 가능성을 먼저 검토한다.
- 로컬 source 변경과 이미 registry에 push된 image를 구분해서 진단한다.

Kerberos/NFS

원본: md/system/kerberos-nfs/index.md, md/system/kerberos-nfs/design.md, md/system/kerberos-nfs/operations.md, md/system/kerberos-nfs/debugging/index.md, md/system/kerberos-nfs/debugging/lab9-why-sec-krb5.md, md/system/kerberos-nfs/debugging/nfs-v41-session-slot-stuck.md, md/system/kerberos-nfs/debugging/nfs-v41-session-slot-stuck-remediation.md

역할: FARM/LAB에서 AD Kerberos로 사용자를 인증하고, 같은 UID/GID로 NFS 공유 스토리지에 접근하게 하는 구조와 운영 기준을 설명한다.

이 문서는 Kerberos/NFS 문서의 시작점이다. 처음 읽는 사람은 **설계**에서 전체 인증 흐름을 먼저 이해한 뒤 **운영**에서 생성·점검·복구 명령을 확인한다. 과거 장애의 증거와 재현 실험은 **디버깅 로그**에서 확인한다. 환경별 실제 값과 최종 실행 절차는 인프라 저장소의 FARM/LAB canonical runbook에서 확인한다.

문서 구성

문서	읽어야 하는 경우	핵심 내용
현재 페이지	전체 범위와 문서 위치를 빠르게 확인할 때	목적, 원칙, 관리 범위
설계	인증이 어느 서버와 프로세스를 거치는지 이해할 때	FARM/LAB NFS version·principal 차이, keytab 발급, RPCSEC_GSS, 권한 판정
FARM/LAB 설정 가이드	각 환경을 처음 구성하거나 기준값을 다시 확인할 때	FARM Synology NAS와 LAB Linux storage의 canonical runbook PDF
운영	계정/컨테이너 생성, timer 확인, mount·KVNO 장애를 처리할 때	Synology/Linux storage 차이, 명령, 점검표, 수동 repair/rotation
디버깅 로그	재현된 장애의 원인과 실험 조건·명령·증거를 확인할 때	장애별 증상, 가설, 재현, 판정, 복구·예방

한눈에 보는 구조

관리 서버(uidctl)

- AD DC: 사용자·RFC2307·keytab 생성
- 계산 호스트: root-only keytab + ccache refresh timer
- NAS/storage: NFS service keytab + 사용자 home
- container: keytab이 아닌 ccache만 사용

사용자 I/O

container process -> host kernel NFS client -> rpc.gssd
-> AD KDC ticket -> NAS svcgssd/NFS -> UID/GID 권한 판정

현재 구성에서 지켜야 할 원칙은 다음과 같다.

- 사용자 keytab은 장기 자격증명이므로 계산 호스트의 `/etc/decs-krb/keytabs/<username>.keytab`에 `root:root0400`으로만 둔다.
- 컨테이너에는 keytab을 넣지 않는다. 호스트가 만든 `/run/user/<uid>/krb5cc`와 읽기 전용 `/etc/krb5.conf`만 bind mount한다.
- 컨테이너가 시작되기 전에 사용자별 `decs-krb-refresh@<username>.timer`를 활성화하고 유효한 ccache를 한 번 발급한다.
- NFS mount source는 IP가 아니라 NFS service principal과 일치하는 FQDN을 쓴다. FARM의 source는 `nas.farm.decs.internal:/volume1/share`다.
- `addr=100.100.100.120`은 FQDN이 해석된 실제 전송 주소로 사용할 수 있다. source 자체를 `100.100.100.120:/...`로 바꾸면 안 된다.
- 기본 security flavor는 `sec=krb5`다. `krb5i`와 `krb5p`는 무결성·암호화가 명시적으로 필요한 경로에서 성능 비용과 함께 선택한다.

Identity 불변 조건

한 사용자에게 대해 다음 숫자가 같아야 한다.

UID DB ubuntu_uid

- = AD RFC2307 uidNumber
- = NFS client에서 관측한 home owner UID
- = container UID

primary GID도 같은 원칙 적용

이 조건을 확인하지 않고 NAS에서 `chown`만 반복하면 AD, DB, NAS의 identity가 더 어긋날 수 있다. 계정 생성 흐름은 Docker와 DB를 확정하기 전에 AD/NAS/host identity와 실제 Kerberos NFS 쓰기를 검증한다.

모듈별 관리 범위

작업	담당 모듈
AD 사용자·그룹, 사용자 keytab, host ccache timer, container 생성	user-lifecycle
공통 host Kerberos/NFS desired state와 fstab	server-state
NAS NFS service account·SPN·service keytab의 수동 변경	kerberos-nfs
GSS readiness, keytab drift 관측, mount 상태, canary, forensics	monitoring
ccache 소비, 시작 전 credential 확인, restricted sudo	container-images

관측 실패가 곧바로 AD password 변경이나 NAS service 재시작으로 이어지지 않게 읽기 전용 monitoring과 상태 변경 작업을 분리한다.

기준 문서와 코드

- FARM canonical runbook
- LAB canonical runbook
- user-lifecycle Kerberos 구현
- container 생성 구현
- host Kerberos/NFS desired state
- Kerberos/NFS keytab health check
- NFS GSS monitoring

실제 endpoint, principal, mount option이 바뀌면 이 위키보다 canonical runbook과 코드 설정을 먼저 갱신하고, 검증 시각과 대상 host를 함께 기록한다.

Kerberos/NFS 설계

이 문서는 keytab이 만들어지는 시점부터 사용자가 NFS 파일을 읽고 쓰는 시점까지, 어느 서버의 어떤 객체와 프로세스가 관여하는지를 설명한다.

자세한 설정 방법은 아래 문서를 통해 확인할 수 있다.

- FARM/LAB 설정 가이드

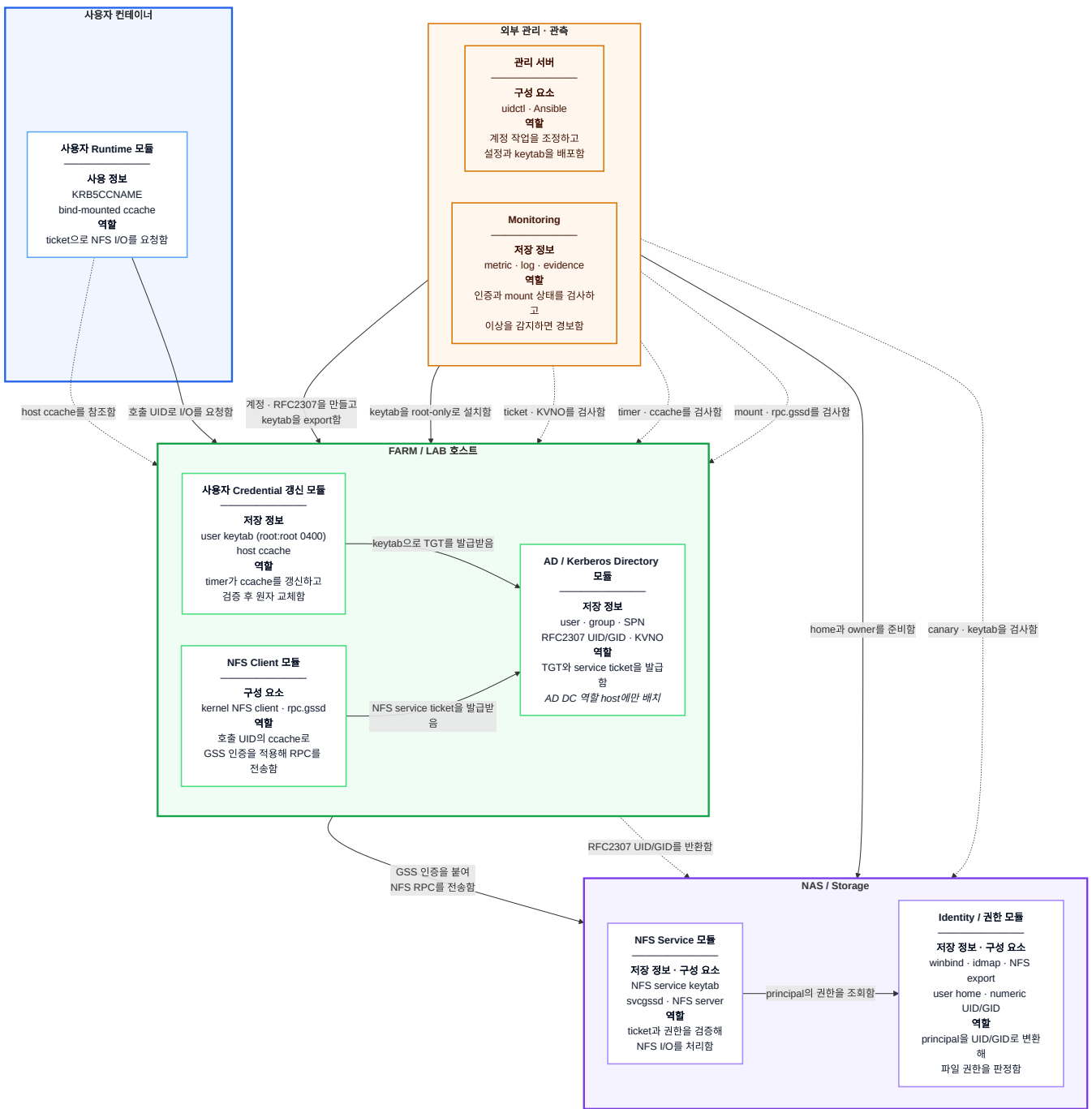
1. 설계 목표

- 비밀번호를 컨테이너에 저장하지 않고 사용자별 Kerberos 인증을 제공한다.
- DB, AD, 계산 호스트, NAS와 컨테이너가 같은 numeric UID/GID를 사용한다.

- NFS 서버는 FQDN 기반 service principal로 인증하고 client는 `sec=krb5` 로 RPCSEC_GSS context를 만든다.
- keytab 같은 장기 자격증명의 노출 범위를 host root로 제한한다.
- ticket 갱신과 컨테이너 lifecycle을 분리하여 컨테이너 재시작 중에도 credential을 안정적으로 유지한다.
- monitoring은 drift를 관측하되 공유 NAS와 AD를 자동으로 변경하지 않는다.

2. 전체 구성

다이어그램의 **굵은 바깥 제목**은 배치 영역이고, 영역 안 각 카드의 **굵은 첫 줄은 모듈 이름**이다. 카드 안의 **저장 정보 / 구성 요소**에는 명사만 적고, **역할**에는 **주체가 무엇을 하는지 동사로** 적었다. 화살표에는 출발 모듈이 요청하거나 제공하는 동작을 표시했으며, 점선은 credential 참조나 읽기 전용 조회·관측을 나타낸다. AD/KDC는 모든 계산 host에 있는 것이 아니라 FARM/LAB 중 **AD DC 역할을 맡은 host에만** 있다. 다이어그램에는 핵심 값과 동작만 표시하고 상세 책임은 아래 표에서 설명한다.



구성요소별 책임

영역	모듈	내부 구성 요소	역할
외부 관리	계정 생성 조정	<code>uidctl</code> , Ansible runner	계정 생성 절차를 조정하고 AD, host, storage에 필요한 상태를 준비
FARM/LAB 호스트	AD/Kerberos Directory	Samba AD DC, KDC, 사용자·그룹·SPN·RFC2307 객체	principal과 Unix identity 저장, keytab export, TGT/service ticket 발급; AD DC 역할 host에만 배치
FARM/LAB 호스트	사용자 Credential 갱신	user keytab, <code>decs-krb-refresh@.service/.timer</code> , host ccache	root-only keytab으로 ccache를 만들고 검증 후 원자 교체
FARM/LAB 호스트	NFS Client	kernel NFS client, <code>rpc.gssd</code>	호출 UID의 ccache로 RPCSEC_GSS context를 만들고 NFS RPC 전송
사용자 컨테이너	사용자 Runtime	사용자 process, bind-mounted ccache, <code>KRB5CCNAME</code>	keytab 없이 host가 갱신한 ticket을 사용하여 NFS 파일 I/O 수행
NAS/Storage	NFS Service	service keytab, <code>svcgssd</code> , NFS server	<code>nfs/<fqdn>@<realm></code> service ticket을 수락하고 NFS 요청 처리
NAS/Storage	Identity/권한	Samba, winbind, idmap, NFS export와 filesystem	Kerberos principal을 RFC2307 UID/GID로 해석하여 최종 파일 권한 판정
외부 관측	Monitoring	readiness, KVNO checker, mount probe, canary, forensics	AD/host/storage 상태를 읽기 전용으로 수집하고 drift와 장애를 경보

3. FARM과 LAB의 현재 설정 차이

FARM과 LAB은 “사용자 keytab은 host root만 보관하고 container에는 ccache만 전달한다”는 credential 모델은 같다. 그러나 realm, NFS server 구현, mount source, service principal과 NFS protocol version은 서로 다르다. 한 환경의 값을 다른 환경에 그대로 복사하면 KDC가 잘못된 service ticket을 발급하거나 NFS server가 ticket을 복호화하지 못한다.

3.1 Identity와 endpoint

항목	FARM	LAB
Kerberos realm	<code>FARM.DECS.INTERNAL</code>	<code>LAB.DECS.INTERNAL</code>
DNS domain / NetBIOS	<code>farm.decs.internal</code> / FARM	<code>lab.decs.internal</code> / LAB
AD/KDC	<code>dc1.farm.decs.internal</code> ; farm2가 primary이고 farm6·farm7을 replica 경로로 사용	<code>dc1.lab.decs.internal</code> ; lab2
NFS server 구현	Synology NAS가 FARM AD domain member로 동작	일반 Linux storage host가 LAB AD domain member와 NFS server로 동작

항목	FARM	LAB
관리 SSH	192.168.2.30:6954	192.168.1.20:6953
NFS data 주소	100.100.100.120	100.100.100.100
운영 mount source	nas.farm.decs.internal:/volume1/share	lab-storage.lab.decs.internal:/294t/dcloud/share
계산 host mount target	/home/tako<번호>/share	/home/tako<번호>/share
사용자 home 원본	/volume1/share/user-share/<username>	/294t/dcloud/share/user-share/<username>

관리 주소는 SSH와 장비 관리에만 사용한다. NFS source에는 반드시 service principal의 hostname을 사용한다. LAB의 /294t/dcloud/share/test_krb 와 /294t/health/nfs-gss-canary 는 각각 격리 PoC와 canary용 export이며 운영 사용자 mount source인 /294t/dcloud/share 와는 용도가 다르다.

3.2 NFS와 Kerberos service identity

항목	FARM	LAB
현재 NFS version	vers=4.0	vers=4.1
기본 security flavor	sec=krb5	sec=krb5
server export flavor	production storage IP에는 sec=krb5:krb5i:krb5p ; client가 krb5 를 선택	현재 LAB baseline은 sec=krb5
NFS service principal	nfs/nas.farm.decs.internal@FARM.DECS.INTERNAL	nfs/lab-storage.lab.decs.internal@LAB.DECS.INTERNAL
추가 principal	nfs/NAS@FARM.DECS.INTERNAL , nfs/nas.ai lab.dgu@AILAB.DGU 를 NAS keytab에서 함께 보존	LAB profile에는 추가 acceptor 설정 없음
NFS SPN 등록 계정	전용 AD user svc-nfs-farm ; Synology machine account NAS\$ 와 분리	Linux storage computer account LAB-STORAGE\$
NFS service keytab	/etc/nfs/krb5.keytab 이 ticket 검증 기준이며 /etc/krb5.keytab 도 checker가 비교	/etc/krb5.keytab
server GSS process	Synology의 /usr/sbin/svcgssd ; 여러 realm의 nfs/* 를 받아야 하므로 -p 로 하나를 고정하지 않음	Linux rpc-svcgssd 또는 NFS server가 관리하는 rpc.svcgssd
client machine principal 예	FARM8\$@FARM.DECS.INTERNAL	LAB9\$@LAB.DECS.INTERNAL

항목	FARM	LAB
GSS canary	전용 export가 없어 현재 비활성	lab-storage.lab.decs.internal:/294t/health/nfs-gss-canary , vers=4.1,sec=krb5 로 활성

사용자 principal도 realm에 따라 `<username>@FARM.DECS.INTERNAL` 또는 `<username>@LAB.DECS.INTERNAL` 이 된다. 사용자 keytab, host machine keytab과 NFS service keytab은 서로 다른 principal과 lifecycle을 가지므로 한 파일로 합쳐서는 안 된다.

3.3 NFS version이 다른 이유

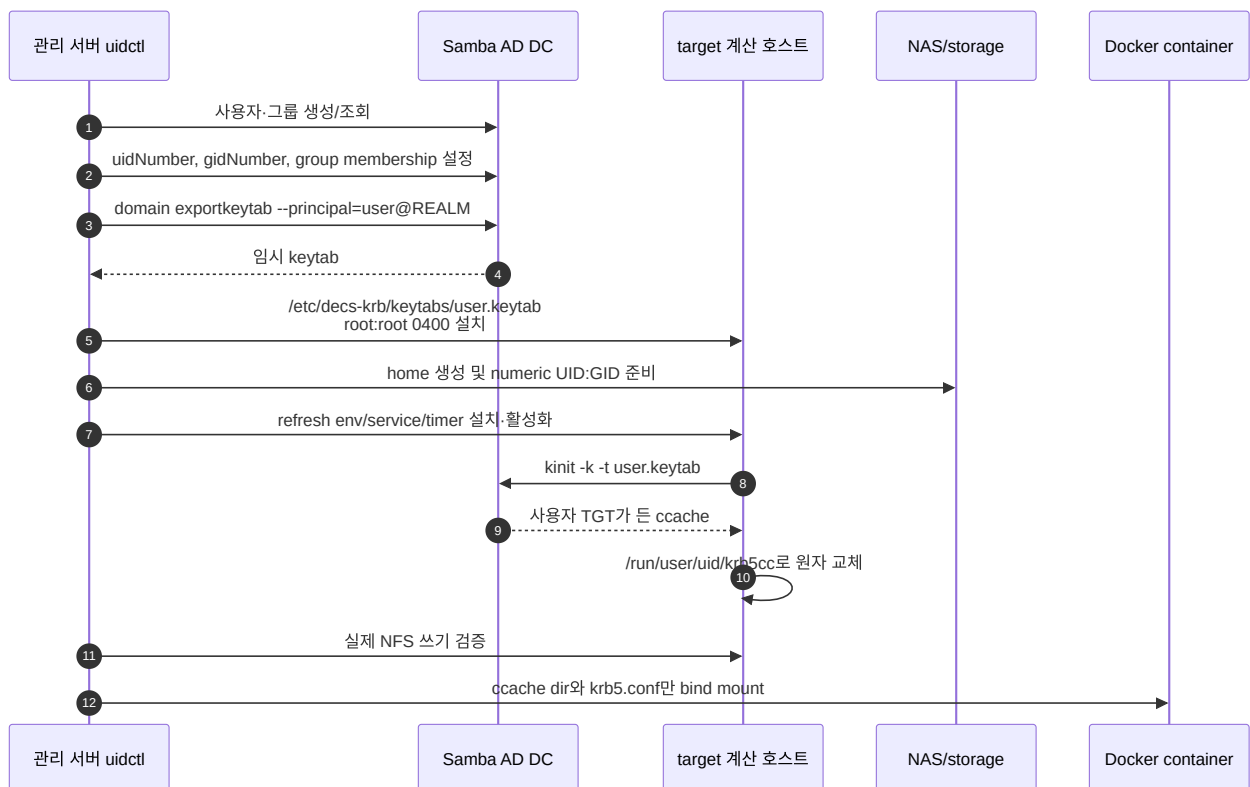
FARM의 Synology NAS에는 NFSv4.1 기능이 활성화되어 있지만, rollout에서 `vers=4.1` 과 `RPCSEC_GSS`를 함께 사용했을 때 timeout 또는 access denied가 발생했다. 따라서 현재 운영 baseline은 `vers=4.0,sec=krb5,hard,timeo=600,retrans=2` 다. fstab에서 1 MiB `rsize/wsize` 를 요청해도 실제 연결에서는 Synology와 128 KiB로 협상될 수 있다.

LAB은 Linux storage의 NFSv4.1을 사용한다. NFSv4.1 session/slot 동작은 server kernel 구현에 따라 달라지므로, 구형 kernel의 idmap deferral과 slot 고착 문제는 [NFSv4.1 session slot 고착](#)에서 별도로 관리한다. 두 환경의 version은 각각 검증된 값이므로 LAB을 임의로 v4.0으로 낮추거나 Synology NAS를 검증 없이 v4.1로 올리지 않는다.

현재 설정값은 [server-state Kerberos/NFS playbook](#)에 정의되어 있다. Monitoring에서 기대하는 값은 `exporter` 변수에서 확인한다.

4. 사용자 keytab 발급 흐름

keytab은 컨테이너 안에서 만들지 않는다. AD DC에서 export한 뒤 target 계산 호스트의 root-only 경로에 설치한다.



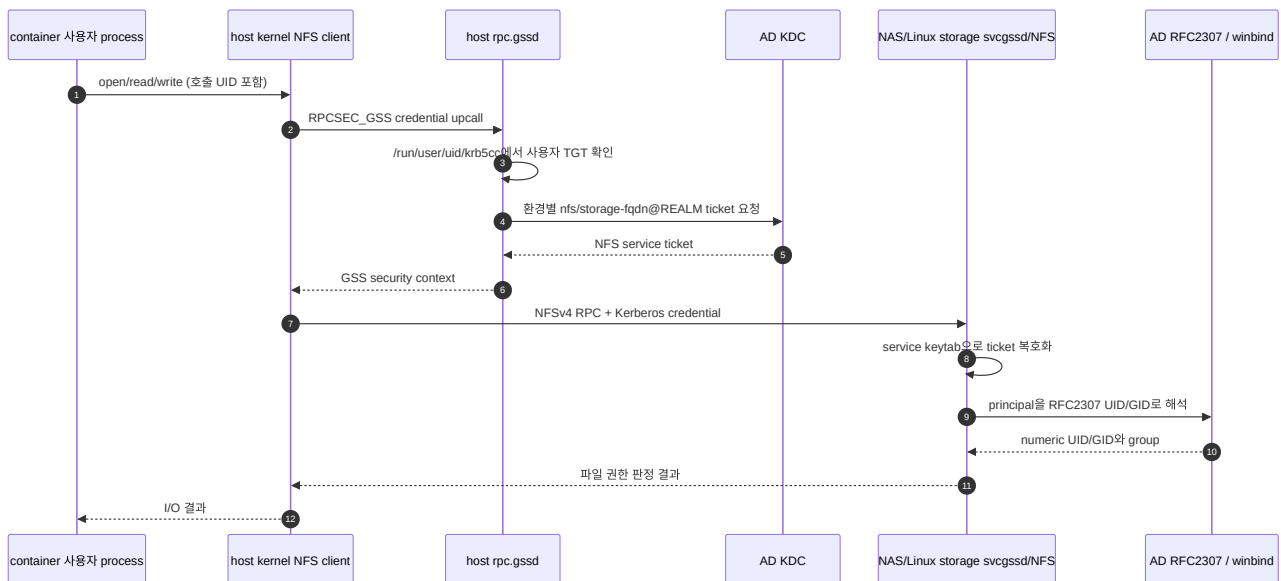
실제 생성 구현은 다음 순서로 동작한다.

1. DB/기존 AD/storage 값을 사용해 UID/GID를 선택한다.
2. AD user/group과 RFC2307 `uidNumber`, `gidNumber` 를 보장한다.
3. AD DC에서 사용자 keytab을 임시 파일로 export하고 `0400` 으로 만든다.
4. target이 다른 host이면 Ansible `fetch` 와 `copy` 로 root-only keytab을 옮긴다.
5. NAS/storage home을 동일 UID/GID로 준비한다.
6. target host에 ccache directory, refresh helper, service/timer를 만든다.
7. host에서 사용자 ccache를 발급하고 NFS owner와 실제 write/delete를 검증한다.
8. 검증 후에만 컨테이너와 DB record를 확정한다.

관련 코드는 [AD identity](#)와 [keytab 생성](#), [container 생성 transaction](#), [credential 경로 모델](#)에서 확인한다.

5. 실제 NFS 인증 흐름

keytab 발급과 매 파일 접근의 인증은 서로 다른 흐름이다. 평상시 파일 I/O에서 컨테이너가 keytab을 직접 읽는 일은 없다.



FARM에서는 `nfs/nas.farm.decs.internal@FARM.DECS.INTERNAL`, LAB에서는 `nfs/lab-storage.lab.decs.internal@LAB.DECS.INTERNAL` ticket을 요청한다. 이 값은 mount source의 hostname에서 결정되며 단순 표시용 설정이 아니다.

인증이 실패한 단계에 따라 증상이 달라진다.

실패 위치	대표 증상
사용자 keytab/ccache	<code>kinit</code> 또는 <code>klist</code> 실패, 사용자 접근만 <code>Permission denied</code>
DNS/FQDN/SPN	<code>kvno nfs/<fqdn></code> 실패, IP source mount에서 principal 불일치
host machine keytab/ <code>rpc.gssd</code>	부팅 시 mount 실패, readiness의 keytab/kinit/rpc-gssd stage 실패

실패 위치	대표 증상
NAS service keytab/KVNO	여러 client가 동시에 GSS 실패, <code>kvno -k</code> 복호화 실패
RFC2307/ldap	인증은 되지만 owner가 <code>nobody</code> 이거나 UID/GID가 달라 쓰기 거부
NFS transport/kernel	<code>not responding</code> , D-state, mount probe timeout

6. Keytab과 ccache를 분리한 이유

구분	keytab	ccache
의미	principal의 장기 비밀키	이미 발급된 TGT/service ticket 묶음
새 ticket 발급	가능	제한된 renew 기간 안에서만 가능
유출 영향	rotation할 때까지 반복 악용 가능	ticket lifetime/renew lifetime에 제한
저장 위치	host root-only	<code>/run/user/<uid>/krb5cc</code> , 사용자 <code>0600</code>
container 전달	금지	bind mount하여 전달

컨테이너 삭제만으로 유출된 keytab을 무효화할 수 없기 때문에 image, Docker volume, Kubernetes Secret에 사용자 keytab을 그대로 넣지 않는다. 호스트 refresh service는 임시 ccache에서 `klist` 검증을 끝낸 뒤 소유권 `uid:gid`, mode `0600` 을 적용하고 최종 경로로 원자 교체한다.

현재 코드가 구현하는 대상은 Docker container다. Kubernetes Pod도 같은 원칙을 적용해야 하지만, Pod가 다른 node로 이동할 수 있으므로 단순 hostPath와 특정 node의 systemd timer에 의존해서는 안 된다. Pod 지원을 추가할 때는 node 배치, credential 발급 controller/CSI, rotation과 Pod 종료 시 정리를 별도 설계해야 한다.

7. Ticket 갱신 설계

`decs-krb-refresh@<username>.timer` 는 기본적으로 boot 2분 뒤 시작하고 사용자별 설정 주기(현재 일반적으로 1시간)마다 oneshot service를 실행한다.

유효 ccache 있음

- └─ renew deadline이 margin보다 멀 -> 임시 복사본에서 `kinit -R`
- └─ deadline 임박/renew 실패 -> root-only keytab으로 새 ticket 발급

발급 결과 -> `klist` 검증 -> `uid:gid 0600` -> 최종 ccache로 atomic move

기본 재발급 여유는 `DECS_KRB_REISSUE_BEFORE_SECONDS=86400` (24시간)이다. 기존 ticket의 PAC/group 정보는 renew 시 유지될 수 있으므로 AD group 변경 뒤에는 keytab을 이용한 fresh login을 한 번 강제해야 한다.

8. NFS service identity와 KVNO

FARM NFS SPN은 Synology domain member의 machine account `NAS$` 가 아니라 전용 AD service account `svc-nfs-farm` 에 등록되어 있다.

```
nfs/nas.farm.decs.internal@FARM.DECS.INTERNAL
nfs/NAS@FARM.DECS.INTERNAL
```

과거에는 `NAS$` machine password가 주기적으로 변경될 때 KVNO도 바뀌어 NAS의 정적 NFS keytab과 어긋날 수 있었다. 이를 service identity와 domain membership의 lifecycle 결합 문제로 보고 NFS SPN을 전용 계정으로 분리했다.

현재 `svc-nfs-farm`에는 자동 password expiration/rotation timer가 없다. 따라서 “NAS KVNO가 지금도 주기적으로 바뀐다”가 아니라 **과거 자동 변경 문제를 전용 계정으로 제거했고, 현재 KVNO는 관리자가 명시적으로 rotation할 때만 바뀐다**가 정확한 운영 모델이다. 별도 5분 checker는 변경을 만들지 않고 AD KVNO와 두 NAS keytab의 일치만 확인한다.

NAS의 `svcgssd`는 FARM과 AILAB principal을 한 keytab에서 받아들이므로 `-p`로 FARM principal 하나만 강제하지 않는다. keytab repair 시에도 AILAB acceptor를 보존한다.

LAB은 Synology NAS용 전용 service account와 keytab merge 절차를 사용하지 않는다. 현재 LAB keytab checker profile은 `LAB-STORAGE$` computer account의 KVNO와 Linux storage의 `/etc/krb5.keytab` 안 `nfs/lab-storage.lab.decs.internal@LAB.DECS.INTERNAL`을 비교한다. FARM repair나 rotation script를 LAB storage에 실행하면 안 된다.

9. FQDN mount와 service principal

Kerberos의 NFS service identity는 host-based principal이다. FARM mount source는 다음처럼 principal의 hostname과 같아야 한다.

```
mount source: nas.farm.decs.internal:/volume1/share
service SPN:  nfs/nas.farm.decs.internal@FARM.DECS.INTERNAL
transport:    addr=100.100.100.120
```

`100.100.100.120:/volume1/share`를 source로 쓰면 client가 IP 기반 service identity를 찾으려 하거나 기대한 FQDN SPN과 연결하지 못할 수 있다. 반면 source를 FQDN으로 유지한 상태에서 runtime option에 `addr=100.100.100.120`이 보이는 것은 정상이다. 관리 SSH 주소 `192.168.2.30`은 NFS transport로 사용하지 않는다.

LAB도 같은 규칙을 적용한다.

```
mount source: lab-storage.lab.decs.internal:/294t/dcloud/share
service SPN:  nfs/lab-storage.lab.decs.internal@LAB.DECS.INTERNAL
transport:    100.100.100.100
```

LAB 관리 SSH 주소 `192.168.1.20` 이나 data IP `100.100.100.100:/294t/dcloud/share` 를 mount source로 직접 쓰지 않는다.

10. 보안 flavor와 권한 모델

- `sec=krb5` : 사용자/서비스 인증. RPC payload 무결성·암호화 wrapping 없음.
- `sec=krb5i` : 인증 + RPC payload 무결성.
- `sec=krb5p` : 인증 + 무결성 + privacy encryption.

현재 내부 FARM/LAB 기본은 `sec=krb5` 다. packet capture에 payload가 포함될 수 있으므로 NFS forensics 산출물은 root-only로 보관한다.

Kerberos 인증 뒤 최종 파일 권한은 numeric UID/GID와 mode/ACL로 결정된다. 컨테이너 root가 host credential 경계를 우회하지 못하도록 Kerberos mode에서는 `DECS_USER_SUDO_MODE=restricted` 를 함께 사용한다. 다만 restricted sudo는 완전한 sandbox가 아니므로 강한 격리가 필요하면 sudo와 `SYS_ADMIN` , host bind mount 범위도 별도로 줄여야 한다.

11. 설정과 구현 위치

관심사	기준 코드/문서
FARM endpoint, SPN, mount option, NAS keytab	FARM runbook
LAB topology와 rollout gate	LAB runbook
사용자 AD/keytab/ccache 명령 생성	commands.py
keytab·ccache·home 경로	paths.py
create-container transaction과 Docker bind	create_container.py
container credential 대기와 restricted sudo	entrypoint.sh
host machine keytab/readiness/fstab	kerberos_nfs_client_recovery.yml
NFS GSS readiness와 recovery	cluster-monitor-exporter scripts
NAS service keytab drift checker	check-nfs-keytab.sh

Kerberos/NFS 운영

이 문서는 현재 Docker 기반 FARM/LAB 환경의 일상 점검과 장애 대응 절차를 설명한다. 환경별 endpoint와 적용 상태는 반드시 canonical runbook에서 다시 확인한다.

1. 운영 원칙

1. 관측과 변경을 분리한다. 5분 keytab checker는 자동 repair/rotation을 하지 않는다.
2. 사용자 keytab은 host root-only로 두고 container에는 ccache만 전달한다.
3. 컨테이너를 시작하기 전에 refresh timer와 최초 ccache 발급을 완료한다.
4. mount source는 FQDN을 사용하고 IP는 `addr=` transport 값으로만 사용한다.
5. `findmnt`, `klist`, `kvno`, readiness 순서로 원인을 좁힌 뒤 service restart나 remount를 마지막 수단으로 사용한다.
6. UID/GID 불일치는 AD·DB·NFS 숫자를 비교한 뒤 고친다. 먼저 `chown` 하지 않는다.

2. Kerberos container 생성 (추후 자동화 시스템으로 대체될 예정)

대표 CLI 형태는 다음과 같다. 실제 image/version, 사용자 정보와 대상 server는 요청에 맞게 지정한다.

```
cd /home/jy/server_manage/user-lifecycle/script

python3 -B -m uid_manager.cli create-container \
  --server-id FARM8 \
  --name "사용자 이름" \
  --username <username> \
  --group <groupname> \
  --image <image> \
  --version <version> \
  --created-by <operator> \
  --email <email> \
  --phone <phone> \
  --enable-kerberos
```

먼저 `--dry-run` 으로 UID/GID, target host, mount source, container command를 확인하는 것을 권장한다. 기존 사용자 password/key를 의도적으로 바꿀 때만 `--rotate-kerberos-keytab` 을 추가한다. 이 옵션은 AD user password와 KVNO를 바꾸므로 단순 재배포 옵션으로 사용하지 않는다.

생성 transaction은 다음 조건을 모두 통과한 후 Docker/DB를 확정해야 한다.

- AD `uidNumber/gidNumber` 가 선택한 DB UID/GID와 일치한다.
- target host keytab이 `root:root 0400` 이다.
- ccache timer가 enabled/active이고 최초 ccache가 유효하다.
- NAS/storage home의 numeric owner가 기대한 UID/GID다.
- 새로 만든 사용자 credential로 host NFS write/delete가 성공한다.
- Docker에는 ccache directory와 읽기 전용 `krb5.conf` 만 전달된다.

구현은 `create_container.py`와 Kerberos command builder에서 확인한다.

3. Host keytab과 ccache 확인

파일 권한

```
sudo stat -c '%U:%G %a %n' \  
    /etc/decs-krb/keytabs/<username>.keytab \  
    /etc/decs-krb/refresh.d/<username>.env  
  
sudo stat -c '%U:%G %a %n' /run/user/<uid> /run/user/<uid>/krb5cc
```

기대값은 다음과 같다.

```
keytab:      root:root 400  
refresh env: root:root 600  
ccache dir:  <uid>:<gid> 700  
ccache:      <uid>:<gid> 600
```

timer와 ticket

```
systemctl list-timers 'decs-krb-refresh@*.timer'  
systemctl status 'decs-krb-refresh@<username>.timer' --no-pager  
sudo systemctl start 'decs-krb-refresh@<username>.service'  
sudo journalctl -u 'decs-krb-refresh@<username>.service' -n 100 --no-pager  
  
sudo -u '#<uid>' env KRB5CCNAME=FILE:/run/user/<uid>/krb5cc \  
    klist -c FILE:/run/user/<uid>/krb5cc
```

컨테이너 생성 때 timer unit을 설치·enable/start하고, 기존 ccache가 유효하지 않으면 oneshot service를 즉시 실행하도록 구현되어 있다. timer만 존재하고 최초 ccache가 없는 상태에서 컨테이너를 먼저 시작하면 Kerberized home 접근이 실패할 수 있다.

AD group을 변경했다면 renew만 기다리지 말고 fresh ticket을 발급한다.

```
sudo rm -f /run/user/<uid>/krb5cc  
sudo systemctl start 'decs-krb-refresh@<username>.service'  
sudo -u '#<uid>' klist -c FILE:/run/user/<uid>/krb5cc
```

4. 컨테이너 credential 확인

```
docker inspect <container> --format '{{range .Config.Env}}{{println .}}{{end}}' |  
grep -E '^(KRB5CCNAME|DECS_KRB5_PRINCIPAL|DECS_KERBEROS_ENABLED)='  
  
docker exec --user <username> <container> \  
sh -lc 'klist -c "$KRB5CCNAME" && touch ~/.__krb_test && rm ~/.__krb_test'
```

다음은 잘못된 상태다.

- container mount나 image 안에 *.keytab 이 있음
- KRB5CCNAME 이 host와 공유되지 않은 임시 경로를 가리킴
- container UID가 DB/AD UID와 다름
- Kerberos mode인데 unrestricted sudo와 광범위한 host mount를 함께 허용함

현재 저장소에는 Kubernetes Pod용 credential controller가 구현되어 있지 않다. Pod 운영을 추가한다면 Pod 생성 시 “어느 node에서 누가 keytab을 보관하고 ccache를 갱신할지”부터 구현해야 하며, Docker용 systemd timer 명령을 그대로 복사해 완료로 간주하면 안 된다.

5. NFS mount source 확인

환경마다 source, NFS version과 service principal이 다르다.

FARM의 올바른 형태는 다음과 같다.

```
nas.farm.decs.internal:/volume1/share /home/tako8/share nfs4  
defaults,vers=4.0,rsize=1048576,wsize=1048576,sec=krb5,proto=tcp,hard,timeo=600,retrans=2,addr=100.100.100.120,  
_netdev,exec,nouser 0 0
```

LAB의 올바른 형태는 다음과 같다. 실제 fstab에는 rpc-gssd 와 decs-kerberos-nfs-ready.service 에 대한 x-systemd.requires/after option도 playbook이 함께 추가한다.

```
lab-storage.lab.decs.internal:/294t/dcloud/share /home/tako9/share nfs4  
defaults,vers=4.1,sec=krb5,proto=tcp,hard,_netdev,exec,nouser 0 0
```

핵심은 source가 각 realm의 NFS service principal과 일치하는 FQDN이라는 점이다.

FARM 올바른: `nas.farm.decs.internal:/volume1/share`
FARM 잘못된: `100.100.100.120:/volume1/share` 또는 `192.168.2.30:/volume1/share`
FARM 정상: runtime option의 `addr=100.100.100.120`

LAB 올바른: `lab-storage.lab.decs.internal:/294t/dcloud/share`
LAB 잘못된: `100.100.100.100:/294t/dcloud/share` 또는 `192.168.1.20:/294t/dcloud/share`
LAB 정상: FQDN을 해석한 실제 transport 주소 `100.100.100.100`

IP source를 쓰면 `nfs/nas.farm.decs.internal@FARM.DECS.INTERNAL` service principal 또는 `nfs/lab-storage.lab.decs.internal@LAB.DECS.INTERNAL` 과 이름이 맞지 않는다. `192.168.2.30` 과 `192.168.1.20` 은 관리 SSH 주소이며 운영 NFS data path가 아니다.

점검 명령:

```
MOUNT_TARGET=/home/tako9/share
STORAGE_FQDN=lab-storage.lab.decs.internal

findmnt -T "$MOUNT_TARGET" -o TARGET,SOURCE,FSTYPE,OPTIONS
nfsstat -m | sed -n "\\|${MOUNT_TARGET}|,+4p"
getent hosts "$STORAGE_FQDN"
```

FARM host에서는 값을 `/home/tako<번호>/share` 와 `nas.farm.decs.internal` 로 바꾼다. `findmnt` 의 `vers=4.0` 은 FARM, `vers=4.1` 은 LAB이어야 하고 두 환경 모두 `sec=krb5` 여야 한다.

`fstab` 설정은 [server-state playbook](#)에서 관리한다. 이미 mount된 legacy superblock은 유지보수 창에 `clean unmount/remount`한다. NFS caller가 D-state이면 강제 unmount를 반복하지 말고 포렌식 보존과 host reboot 판단으로 전환한다.

6. Machine credential과 NFS service ticket 확인

계산 호스트의 root mount에는 host machine keytab과 `rpc.gssd` 가 필요하다. realm과 NFS principal을 host 이름에 맞춰 선택한다.


```

short=$(hostname -s | tr '[:lower:]' '[:upper:]')

case "$short" in
    FARM*) KRB_REALM=FARM.DECS.INTERNAL
           KRB_NFS_PRINCIPAL=nfs/nas.farm.decs.internal@FARM.DECS.INTERNAL ;;
    LAB*) KRB_REALM=LAB.DECS.INTERNAL
          KRB_NFS_PRINCIPAL=nfs/lab-storage.lab.decs.internal@LAB.DECS.INTERNAL ;;
    *)    printf '지원하지 않는 host: %s\n' "$short" >&2; exit 1 ;;
esac
KRB_MACHINE_PRINCIPAL="${short}\${KRB_REALM}"

sudo klist -kte /etc/krb5.keytab
sudo KRB5CCNAME=FILE:/run/decs-host-check.ccache \
    kinit -k -t /etc/krb5.keytab "$KRB_MACHINE_PRINCIPAL"
sudo KRB5CCNAME=FILE:/run/decs-host-check.ccache \
    kvno "$KRB_NFS_PRINCIPAL"
sudo rm -f /run/decs-host-check.ccache

systemctl status rpc-gssd --no-pager
systemctl cat rpc-gssd
pgrep -a rpc.gssd

```

`rpc.gssd`를 `-n`으로 시작하지 않는다. root mount는 host machine credential을 사용해야 하며, `-n`은 UID 0 사용자 credential을 찾게 하여 ticket이 없을 때 mount를 잃게 만들 수 있다.

7. Synology NAS와 Linux storage 운영 차이

FARM의 Synology NAS와 LAB의 Linux storage는 같은 NFS/Kerberos 개념을 사용하지만 설정을 관리하는 방식, service 이름과 keytab lifecycle이 다르다. client에서 확인하는 `findmnt`, `klist`, `kvno` 절차는 비슷해도 server 변경 명령은 서로 바뀌 실행하면 안 된다.

7.1 운영 경계 비교

항목	FARM Synology NAS	LAB Linux storage
관리 접속	<code>jy@192.168.2.30:6954</code>	<code>jy@192.168.1.20:6953</code>
AD join 도구	DSM UI 또는 <code>/usr/syno/sbin/synowin</code>	표준 Samba <code>net ads join</code>
Samba/winbind	Synology SMB package 경로와 <code>pkg-synosamba-*</code> unit	<code>/etc/samba/smb.conf</code> , 표준 <code>winbind.service</code>

항목	FARM Synology NAS	LAB Linux storage
RFC2307 NSS	<code>files winbind syno</code> 순서와 Synology idmap 설정을 함께 확인	<code>files systemd winbind</code> 와 Samba <code>idmap config</code> LAB : <code>backend = ad</code> 확인
NFS export root	<code>/volume1/share</code>	<code>/294t/dcloud/share</code> ; PoC는 별도 <code>test_krb</code> export
production export	storage IP별 <code>root_squash</code> , <code>sec=krb5:krb5i:krb5p</code> ; Synology <code>anonuid/anongid</code> , <code>insecure</code> option 포함	Linux <code>/etc/exports</code> 에서 <code>root_squash</code> , <code>sec=krb5</code> , <code>no_subtree_check</code> 를 명시
NFS service	Synology <code>/usr/sbin/svcgssd</code> 와 <code>/usr/sbin/idmapd</code>	<code>nfs-server</code> 와 <code>rpc-svcgssd</code> 또는 배포판의 socket-activated GSS service
NFS keytab	<code>/etc/nfs/krb5.keytab</code> 이 ticket 검증 기준; <code>/etc/krb5.keytab</code> 도 동기화 검사	<code>/etc/krb5.keytab</code> 한 경로
NFS SPN 등록 계정	전용 user <code>svc-nfs-farm</code> ; <code>NAS\$</code> 와 분리	computer account <code>LAB-STORAGE\$</code>
추가 acceptor	같은 keytab의 AILAB <code>nfs/nas.ai lab.dgu@AILAB.DGU</code> 를 반드시 보존	현재 없음
운영 NFS version	v4.0	v4.1
keytab checker	<code>--profile farm</code> , 5분 user timer 운영	<code>--profile lab</code> ; profile은 준비됐지만 PoC 단계에서는 timer를 기본 활성화하지 않음

Synology에서 `getent` , `id` 또는 `wbinfo` 가 DECS UID/GID 대신 `96470xxx` 같은 내부 ID를 반환하면 RFC2307 경로가 정상인 상태가 아니다. 이때 symbolic `chown -R 'FARM\user':'FARM\group'` 을 실행하면 잘못 해석된 숫자가 filesystem에 저장된다. 먼저 DB-AD와 NAS의 numeric UID/GID가 일치하는지 확인하고, ownership 복구도 검증된 숫자로 수행한다.

FARM NAS에는 관리·호환성 목적으로 남은 `192.168.2.0/24 sec=sys,no_root_squash` export가 있을 수 있다. 이것은 production Kerberos data path가 아니며 FARM host나 container가 이 `sec=sys` export를 사용하도록 변경하면 안 된다.

7.2 읽기 전용 점검

FARM Synology NAS에서는 vendor 경로와 service 이름을 사용한다.

```
ssh -p 6954 jy@192.168.2.30

/usr/syno/sbin/synowin -getWorkgroup
SMB=/usr/local/packages/@appstore/SMBService/usr/bin
sudo "$SMB/net" ads testjoin
sudo "$SMB/wbinfo" --online-status

sudo klist -kte /etc/nfs/krb5.keytab
sudo klist -kte /etc/krb5.keytab
sudo exportfs -v | sed -n '/\/volume1\/share/,+8p'
ps -ef | grep -E 'svcgssd|idmapd' | grep -v grep
```

LAB Linux storage에서는 표준 Samba/NFS systemd unit과 keytab을 확인한다.

```
ssh -p 6953 jy@192.168.1.20

sudo net ads testjoin
wbinfo --online-status
getent passwd '<username>'

sudo klist -kte /etc/krb5.keytab \
| grep -F 'nfs/lab-storage.lab.decs.internal@LAB.DECS.INTERNAL'
systemctl is-active winbind nfs-server
systemctl --no-pager --full status rpc-svcgssd.service 2>/dev/null || true
sudo exportfs -v | sed -n '/\/294t\/dcloud\/share/,+8p'
```

LAB 배포판에서는 `rpc-svcgssd` 가 독립 unit이 아니라 socket 또는 `nfs-server` 의 일부로 관리될 수 있다. unit 이름 하나만으로 실패를 판정하지 말고 process, service keytab과 실제 `kvno` /canary 결과를 함께 본다.

7.3 변경 작업에서 지킬 차이

- Synology 설정을 바꾸기 전에는 DSM/Samba/NFS 설정과 두 keytab을 먼저 backup한다. DSM 또는 SMB package가 관리하는 파일·unit을 일반 Ubuntu와 같다고 가정하지 않는다.
- FARM keytab repair는 AILAB principal과 이전 FARM KVNO를 보존하는 전용 `repair-farm-nas-nfs-keytab.sh` 을 사용한다. `svcgssd` 를 FARM principal 하나로 제한하는 `-p` 나 nameless acceptor `-n` 으로 시작하지 않는다.
- LAB storage는 표준 Samba join, `/etc/krb5.keytab` , `nfs-server` / `rpc-svcgssd` 와 Ansible desired state를 기준으로 한다. FARM repair/rotation script를 LAB에 실행하지 않는다.
- 두 환경 모두 identity/export 변경 후 `exportfs -ra` 와 필요한 NFS/RPC cache flush를 수행할 수 있지만, 먼저 활성 client와 D-state를 확인하고 유지보수 시간에만 실행한다.
- Synology 관리 IP와 LAB storage 관리 IP는 NFS source가 아니다. 변경 후 client `findmnt` 에서 각각 `nas.farm.decs.internal` 과 `lab-storage.lab.decs.internal` 이 유지되는지 확인한다.

환경별 keytab 상태는 같은 checker를 profile만 바꿔 읽기 전용으로 확인한다.

```
cd /home/jy/server_manage/monitoring
health-checks/kerberos-nfs-keytab/script/check-nfs-keytab.sh --profile farm
health-checks/kerberos-nfs-keytab/script/check-nfs-keytab.sh --profile lab
```

server-side 전체 설정 절차는 [FARM canonical runbook](#)과 [LAB canonical runbook](#)을 각각 따른다.

8. FARM NAS service account와 KVNO 운영

현재 정책

- NFS SPN 등록 계정: `svc-nfs-farm`
- `NAS$`: domain membership용 `HOST/*`, `RestrictedKrbHost/*` 만 유지
- 자동 password expiration/rotation: 없음
- keytab drift 관측: 5분마다 읽기 전용
- repair/rotation: 관리자 명시 실행만 허용

과거 `NAS$` machine password의 주기적 변경으로 KVNO와 NAS NFS keytab이 어긋났기 때문에 전용 service account를 만들었다. 운영 문서에는 “KVNO가 주기적으로 바뀌어서 매번 새 계정을 만든다”가 아니라, **한 번 만든 전용 계정으로 자동 machine-account rotation에서 NFS SPN을 분리했다고** 기록한다.

읽기 전용 점검

```
cd /home/jy/server_manage/monitoring
health-checks/kerberos-nfs-keytab/script/check-nfs-keytab.sh --profile farm

systemctl --user status check-nfs-keytab@farm.timer --no-pager
systemctl --user list-timers 'check-nfs-keytab@farm.timer'
```

checker가 확인하는 항목:

- AD `svc-nfs-farm`의 현재 `msDS-KeyVersionNumber`
- NAS `/etc/krb5.keytab`, `/etc/nfs/krb5.keytab`에 현재 KVNO와 NFS principal 존재
- 공유 keytab에 AILAB acceptor가 보존되어 있는지
- 실제 `kvno -k` service-ticket 복호화 성공 여부
- `svcgssd` 실행 여부와 잘못된 `-p` 제한 여부

코드는 공용 checker, profile 값은 [FARM config](#)에서 확인한다.

Drift 수동 복구

```
cd /home/jy/server_manage/kerberos-nfs

./script/keytab/repair-farm-nas-nfs-keytab.sh --check
./script/keytab/repair-farm-nas-nfs-keytab.sh --repair
```

`--repair` 는 새 key를 export·검증하고 기존 AILAB principal과 이전 FARM KVNO를 보존하면서 NAS keytab을 원자 교체한다. 필요하다면 `svcgssd` 를 `-p` 없이 재시작하므로 활성 client 영향과 되돌릴 backup을 먼저 확인한다. 구현은 [repair script](#)에 있다.

계획된 rotation

```
./script/keytab/rotate-farm-nfs-service-key.sh --check

# 유지보수 창, AD replication, NAS/client 상태를 확인한 뒤에만 실행
./script/keytab/rotate-farm-nfs-service-key.sh --rotate --apply
```

rotation은 random password 설정, AD KVNO 변경, 새 key export, NAS keytab merge, `svcgssd` 반영, replica sync를 한 작업으로 수행한다. 이전 KVNO는 설정된 24시간 ticket lifetime보다 긴 최소 48시간 보존한다. 구현은 [rotation script](#)에 있다.

9. 모니터링에서 확인할 항목

무엇을 확인하는가	대표 상태/지표	코드
AD KVNO와 NAS keytab 일치	profile status JSON, checker exit 0/1/2	keytab health check
host keytab→kinit→kvno→rpc-gssd	<code>cluster_monitor_nfs_gss_ready</code> 와 stage	readiness script
실제 service path	<code>cluster_monitor_nfs_gss_canary_success</code>	canary probe
운영 mount	<code>cluster_monitor_host_mount_up</code> , <code>..._responsive</code> , probe seconds	cluster collector
missing mount 복구	<code>cluster_monitor_nfs_gss_recovery_*</code>	guarded recovery
D-state/lease/hung task	<code>cluster_monitor_host_dstate_*</code> , <code>cluster_monitor_nfs_*</code>	NFS forensics collector
경보 조건	<code>ClusterMonitorNFSGSS*</code> , <code>ClusterMonitorNFS*</code> , mount alerts	FARM Prometheus values

상세한 Prometheus/Grafana 운영은 [monitoring 운영 문서](#)를 참고한다.

10. 장애 진단 순서

사용자 한 명만 실패

1. DB UID/GID와 AD `uidNumber/gidNumber` 를 비교한다.
2. user keytab permission과 principal을 확인한다.
3. refresh service journal과 ccache `klist` 를 확인한다.
4. 동일 UID로 host NFS write를 시험한다.
5. container UID, `KRB5CCNAME` , ccache bind mount를 확인한다.
6. AD group 변경 직후라면 fresh ticket을 발급한다.

여러 host/user가 동시에 실패

1. `getent hosts <storage-fqdn>` 과 storage RPC 연결을 확인한다.
2. keytab checker에서 AD KVNO/NAS keytab drift를 확인한다.
3. `kvno nfs/<storage-fqdn>@<REALM>` 을 확인한다.
4. NAS `svcgssd` 와 service keytab을 확인한다.
5. mount source가 IP나 관리망 주소로 바뀌지 않았는지 확인한다.
6. AD replication 실패 시 실제 쓰기 대상 DC와 NAS가 조회하는 DC를 확인한다.

Mount가 없거나 응답하지 않음

1. `findmnt` 로 존재/source/security를 확인한다.
2. readiness 상태에서 처음 실패한 stage를 확인한다.
3. D-state, lease expiry, hung-task와 forensics snapshot을 확인한다.
4. mount가 **없는 경우에만** guarded recovery timer 상태를 확인한다.
5. 이미 healthy한 mount를 자동/수동으로 재마운트하지 않는다.
6. D-state caller가 남아 있으면 forced unmount 반복 대신 증거 보존 후 reboot를 검토한다.

11. 변경 전후 체크리스트

변경 전

- 대상 realm, DC, storage FQDN과 NFS principal 확인
- 활성 client와 유지보수 창 확인
- AD replication health 확인
- NAS keytab backup과 AILAB principal 확인
- 현재 `findmnt` , `klist` , checker status 보존

변경 후

- AD current KVNO와 NAS keytab KVNO 일치
- `kvno -k` 복호화 성공
- `svcgssd` 가 `-p` 없이 실행
- host readiness와 실제 사용자 NFS write 성공
- refresh timer enabled/active, ccache 유효
- Prometheus target/rules와 Grafana dashboard 정상
- 이전 KVNO를 최소 48시간 보존

12. 문서에 추가로 유지할 내용

설계와 운영이 다시 섞이지 않도록 다음 내용을 계속 분리해 기록한다.

- 설계 문서: trust boundary, principal naming, UID/GID invariant, ticket lifetime, FQDN 선택 근거, failure domain, Docker와 향후 Pod의 credential 모델
- 운영 문서: 현재 endpoint/KVNO가 아니라 확인 명령, timer/SLO, rotation 승인 조건, rollback, incident 진단 순서, 마지막 검증 날짜
- canonical runbook: 실제 host별 mount, 현재 적용 상태, 예외와 잔여 위험
- 실험 결과: 당시 kernel/NFS/security flavor와 재현 조건; 현재 default와 분리

keytab, password, local environment, incident 개인정보와 packet capture는 위키와 Git에 넣지 않는다.

Kerberos/NFS 디버깅 로그

이 문서는 Kerberos/NFS에서 발생한 장애와 재현 실험을 축적하는 인덱스다. 정상 상태를 만드는 절차는 [운영 문서](#)에 두고, 여기에는 장애가 어떻게 관측됐고 어떤 가설을 어떤 조건에서 검증했는지 기록한다.

기록 목록

검증일	상태	문제	핵심 결론
2026-07-23	영구 조치 적용 · 운영 검증 완료	NFSv4.1 session slot 고착 해결 및 운영 적용	LAB storage를 6.12.0-250.el10으로 교체하고 LAB8 운영 mount의 15초 idmap 지연 시험에서도 영구 slot 고착이 없음을 확인함
2026-07-20	관측, 인과 미확정	LAB9에서 sec=krb5를 사용하는 이유	krb5p에서 krb5로 전환한 뒤 D-state 지속 시간, SUNRPC 적체와 mount probe 지연이 크게 감소함
2026-07-16	재현 완료, 과거 장애 원인 후보	NFSv4.1 session slot 고착	구형 storage kernel에서 지연된 idmap decode가 NFSv4.1 slot 하나를 영구 INUSE 상태로 남기는 경로를 재현함

상태는 다음 의미로 사용한다.

- **관측**: 증상과 증거는 있지만 원인 경로를 재현하지 못했다.
- **재현 완료**: 통제된 조건에서 같은 kernel 또는 protocol 경로를 다시 만들었다.
- **원인 후보**: 재현된 경로가 과거 장애를 설명하지만 장애 발생 시점의 직접 trace가 없어 동일 원인이었다고 확정할 수 없다.
- **원인 확정**: 장애가 시작된 시점의 증거와 재현 결과가 같은 경로를 가리킨다.

장애 발생 시 먼저 할 일

NFS **hard** mount 관련 process가 D-state에 들어간 경우에는 새로운 **find**, **du**, **stat**, canary I/O를 반복하지 않는다. 각 명령이 새로운 NFS request와 D-state process를 추가할 수 있다.

먼저 NFS 경로를 읽지 않는 local 상태만 수집한다.

```
date -Ins
uname -r
findmnt -rn -o TARGET,SOURCE,FSTYPE,OPTIONS

ps -e -o state=,pid=,ppid=,etimes=,comm=,wchan:48=,args= \
| awk '$1 ~ /^D/ {print}'

cat /proc/net/rpc/nfs 2>/dev/null || true
cat /run/decs-nfs-forensics/status.json 2>/dev/null || true
cat /var/lib/decs-nfs-gss/recovery.state 2>/dev/null || true
cat /var/lib/decs-nfs-gss/canary.state 2>/dev/null || true

systemctl --no-pager --full status rpc-gssd.service
journalctl -u rpc-gssd.service -b --no-pager -n 200
```

forensics가 배포된 host에서는 자동 snapshot이 생성됐는지 먼저 확인한다. 자동 trigger가 없었다면 local 상태만 읽는 수동 snapshot을 한 번 실행할 수 있다.

```
sudo systemctl start decs-nfs-forensics-snapshot.service
sudo cat /var/lib/decs-nfs-forensics/last_snapshot.state
```

이미 mount된 경로에 D-state caller가 있으면 **rpc-gssd** restart, forced unmount, 반복 remount를 먼저 실행하지 않는다. 증거를 보존한 뒤 storage·network 상태와 kernel wait path를 기준으로 복구 범위를 결정한다.

새 디버깅 문서 작성 형식

새 문제는 한 파일에 다음 내용을 남긴다.

1. **상태와 영향 범위**: 발생 시각, host, kernel, mount source/target/security

2. **증상**: 사용자 증상, process state, wait channel, alert
3. **확인한 사실과 가설**: 관측 사실과 아직 증명하지 못한 추론을 분리
4. **실험 목적**: 어떤 가설을 반증하거나 확인하려 했는지
5. **재현 조건**: server/client, version, cache, 동시성, fault와 안전장치
6. **실행 명령**: 사전 확인, fault 주입, workload, 관측, cleanup 순서
7. **결과와 판정 기준**: 성공/실패를 판단한 trace와 metric
8. **복구와 예방**: 임시 containment, 영구 수정, monitoring 변경
9. **원본 증거**: 결과 README, curated evidence, script와 checksum 링크

운영 환경에서 fault를 주입한 명령은 일반 runbook처럼 제시하지 않고 반드시 실행 일자와 승인·안전 조건을 붙인 **과거 실행 기록**으로 표시한다. 반복 실험은 가능하면 격리 VM harness로 옮긴다.

LAB9에서 **sec=krb5** 를 사용하는 이유

상태: LAB9에서 **sec=krb5p** 전환 전 24시간과 **sec=krb5** 전환 후 약 51시간을 비교한 운영 관측이다. 전환 뒤 NFS 지연과 적체가 크게 감소했지만, 관측 기간이 다르고 security flavor만 통제된 실험이 아니므로 **krb5p** 를 단일 원인으로 확정하지는 않는다.

1. 결론

LAB9의 Kerberos NFS 기본값은 **sec=krb5** 로 유지한다. 이 선택은 Kerberos principal 인증을 유지하면서 RPC payload 암호화 비용을 제외하고, LAB9에서 실제로 관측된 D-state 지속·SUNRPC 적체·mount probe 지연을 낮추기 위한 운영 기준이다.

krb5p 가 잘못된 방식이라는 뜻은 아니다. NFS payload 기밀성이 반드시 필요한 경로에서는 **krb5p** 를 사용해야 한다. 다만 현재 LAB9 사용자 공유의 기본 profile은 가용성과 지연 안정성을 우선해 **krb5** 를 사용한다. **krb5i** 또는 **krb5p** 는 요구사항이 있고 부하 검증을 통과한 경로에서만 명시적으로 선택한다.

2. security flavor의 차이

flavor	제공하는 보호	LAB9에서의 위치
sec=krb5	Kerberos principal 인증	기본값
sec=krb5i	인증 + RPC payload 무결성	변조 방지가 명시적으로 필요한 경로에서 검토
sec=krb5p	인증 + 무결성 + RPC payload 암호화	기밀성이 필요한 경로에서 성능 검증 후 사용

`sec=krb5`에서는 사용자와 NFS service가 Kerberos로 서로 인증되지만, NFS payload 자체는 암호화되지 않는다. 따라서 패킷 캡처와 forensics evidence는 root만 읽을 수 있게 보관하고, 보호되지 않은 네트워크에서 기밀 데이터를 전송해야 한다면 이번 성능 결과만으로 `krb5`를 선택해서는 안 된다.

3. 왜 전환했나

`sec=krb5p` 사용 중 LAB9에서는 NFS 관련 D-state가 관측 기간 내내 존재했고, SUNRPC task와 transport reply 대기가 누적됐다. mount responsiveness probe도 설정된 10초 timeout 상한에 도달했다. 이 상태에서는 home 경로를 읽는 SSH나 사용자 process가 NFS 대기에 연쇄적으로 묶일 수 있다.

`krb5p`는 각 RPC payload에 무결성 처리와 암호화를 추가한다. 이 비용이 장애의 단일 원인이라고 입증한 것은 아니지만, storage·kernel·network에서 이미 발생한 지연을 더 크게 보이게 하는 증폭 요인인지 확인할 운영상 이유가 있었다. 그래서 principal 인증은 유지하면서 privacy 처리를 제외하는 `sec=krb5`로 전환하고 동일한 NFS health 지표를 계속 관측했다.

4. 전환 전후 결과

측정 대상은 LAB9다.

지표	<code>krb5p</code> 전환 전 24시간	<code>krb5</code> 전환 후 약 51시간
NFS D-state 존재 비율	100%	약 4.1%
D-state 프로세스 최대	28개	8개
최장 연속 D-state	83,339초(약 23.1시간)	1,236초(약 20분 36초)
<code>xprt_pending</code> 평균	5.35	1.29
RPC task 최대	3,696	525
마운트 probe 최대	10초	0.429초
lease 만료	없음	없음

변화 폭은 다음과 같다.

비교 항목	변화
NFS D-state 존재 비율	95.9%p 감소
D-state 프로세스 최대	약 71.4% 감소
최장 연속 D-state	약 98.5% 감소
<code>xprt_pending</code> 평균	약 75.9% 감소
RPC task 최대	약 85.8% 감소

비교 항목	변화
마운트 probe 최대	약 95.7% 감소

가장 중요한 변화는 D-state가 “항상 존재하는 상태”에서 짧고 간헐적인 상태로 바뀐 점이다. `xprt_pending` 평균과 RPC task 최대도 함께 감소했으므로 단순히 D-state process 이름만 달라진 것이 아니라 NFS transport 적체 자체가 줄어든 방향과 일치한다.

전환 전 mount probe의 10초는 `cluster-monitor-exporter`의 command timeout 상한과 같다. 따라서 이 값은 정상 응답에 10초가 걸렸다가보다 probe가 timeout 또는 기존 in-flight probe 상태에 도달했을 가능성이 크다. 전환 후 최대 0.429초는 관측 구간에서 10초 상한에 도달하지 않았음을 보여준다.

두 구간 모두 lease 만료가 없었다. 따라서 이번 비교에서 확인한 개선은 lease 만료 복구 여부보다 D-state 지속, RPC queue와 mount 응답성에 관한 것이다.

5. 지표의 의미

지표	판정 방법	구현
NFS D-state 존재 비율	유효 sample 중 NFS/RPC 관련 D-state process가 하나 이상인 sample 비율	<code>cluster_monitor_nfs_forensics_d_state_processes</code>
D-state 프로세스 최대	관측 구간의 NFS/RPC 관련 D-state process 최대값	같은 metric의 <code>max_over_time</code>
최장 연속 D-state	가장 오래 연속 관측된 NFS/RPC 관련 D-state process의 시간	<code>cluster_monitor_nfs_forensics_d_state_oldest_seconds</code>
<code>xprt_pending</code> 평균	transport reply를 기다리는 SUNRPC task 수의 구간 평균	<code>cluster_monitor_nfs_xprt_pending</code>
RPC task 최대	NFS RPC client task 수의 구간 최대	<code>cluster_monitor_nfs_rpc_tasks</code>
마운트 probe 최대	required mount에 수행한 bounded <code>statfs</code> 시간의 구간 최대	<code>cluster_monitor_host_mount_probe_seconds</code>
lease 만료	local NFSv4 lease deadline을 넘긴 시간	<code>cluster_monitor_nfs_lease_expired_seconds</code>

수집 경로는 다음과 같다.

```
LAB9 kernel:/proc 상태
-> NFS forensics watcher status.json
-> cluster-monitor-exporter metrics
-> FARM Prometheus
-> 같은 server="LAB9" label의 전환 전후 구간 집계
```

구현은 `forensics watcher`, `forensics metric adapter`, `mount/D-state collector`에서 확인한다.

6. 이 결과로 말할 수 있는 것과 없는 것

말할 수 있는 범위는 다음과 같다.

- LAB9에서 **krb5** 전환 뒤 모든 주요 NFS 적체 지표가 같은 방향으로 개선됐다.
- 약 51시간의 전환 후 구간에는 24시간 전환 전 구간과 같은 장시간 고착이 관측되지 않았다.
- Kerberos 인증을 유지한 **krb5** 가 LAB9의 현재 운영 baseline으로 더 안정적이었다.

아직 확정할 수 없는 범위는 다음과 같다.

- **krb5p** 의 암호화 비용만으로 전환 전 D-state가 발생했다는 인과관계
- 같은 시기에 달라졌을 수 있는 workload, storage queue, network, kernel 상태의 영향
- FARM/LAB의 다른 host에서도 감소 폭이 동일할 것이라는 일반화
- 51시간보다 긴 기간이나 peak workload에서도 같은 결과가 유지된다는 보장

따라서 이 기록의 판정은 **LAB9 운영 선택을 지지하는 강한 전후 관측**이며, 통제된 benchmark나 원인 확정 실험은 아니다. 원인을 더 좁히려면 동일 workload와 동일 시간대에 **krb5 / krb5p** 를 반복 교차하는 격리 실험이 필요하다.

7. 운영 기준과 확인 방법

LAB9의 desired mount option은 **sec=krb5** 다. mount source는 IP가 아니라 NFS service principal과 일치하는 FQDN을 사용한다. D-state가 이미 존재할 때는 확인을 위해 mount를 반복해서 읽거나 즉시 remount하지 말고 먼저 local forensics 상태를 보존한다.

현재 mount의 security flavor는 다음처럼 정확히 확인한다.

```
LAB9_MOUNT_TARGET=/home/tako9/share

findmnt -T "$LAB9_MOUNT_TARGET" -n -o TARGET,SOURCE,FSTYPE,OPTIONS
findmnt -T "$LAB9_MOUNT_TARGET" -n -o OPTIONS \
| tr ' ' '\n' \
| grep -Fx 'sec=krb5'
```

grep -F 'sec=krb5' 만 사용하면 **sec=krb5p** 도 일치하므로 option을 쉼표 단위로 분리해 exact match한다. 전환 이후에도 최소한 다음 조건을 함께 확인한다.

- NFS D-state 존재 비율과 최장 연속 시간이 다시 증가하지 않는가
- **xprt_pending** 평균과 RPC task 최대가 전환 전 수준으로 돌아가지 않는가
- mount probe가 10초 timeout 상한에 다시 도달하지 않는가
- **cluster_monitor_nfs_lease_expired_seconds** 가 계속 0인가
- payload 기밀성 요구가 새로 생겨 **krb5p** 가 필요한 경로가 되지 않았는가

기본 security policy는 Kerberos/NFS 운영 저장소, 배포 값은 **monitoring exporter** 변수를 기준으로 한다. 상세 장애 채증과 D-state 대응은 **디버깅 로그 개요**의 절차를 따른다.

8. 다음 측정에서 함께 남길 값

현재 비교표에는 구간 길이만 있고 정확한 시작·종료 timestamp와 원본 query/export artifact가 없다. 다음 재측정 부터는 아래 값을 함께 보관해야 같은 결과를 재계산할 수 있다.

- 전환 시각과 두 구간의 시작·종료 시각(타임존 포함)
- LAB9의 source FQDN, target, NFS version과 전체 mount option
- client/server kernel, storage 상태와 구간별 workload
- 사용한 PromQL 또는 `samples.tsv` 집계 script와 원본 결과
- exporter command timeout과 scrape interval
- 전환과 동시에 수행한 다른 설정 변경

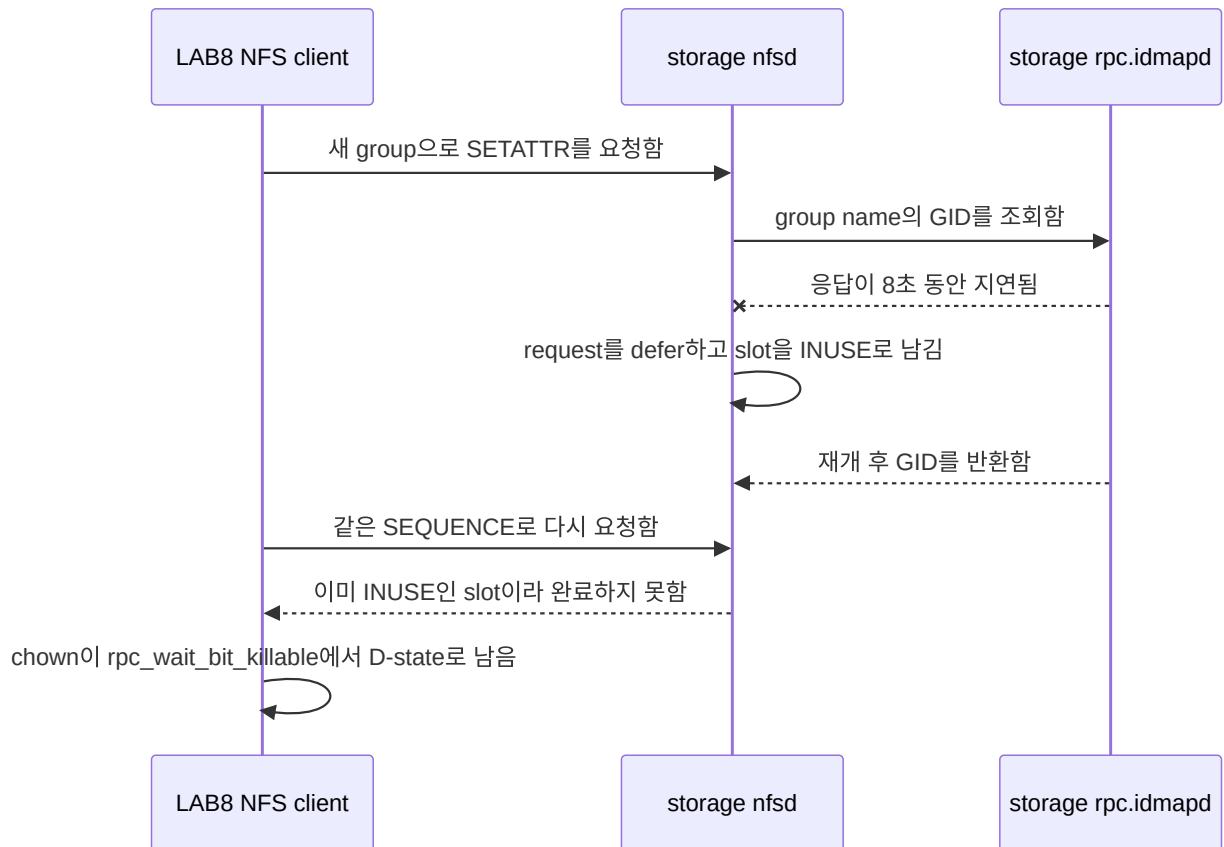
원본 artifact를 저장소에 추가하면 이 문서의 표 아래에 commit 고정 링크를 추가한다.

NFSv4.1 session slot 고착

상태: 2026-07-16에 slot leak 경로 재현 완료. 2026-06-29 LAB5 장애의 강한 원인 후보이지만, LAB5 장애 시작 시점의 storage trace가 없어 과거 장애의 최초 원인으로 확정하지는 않는다.

1. 요약

LAB storage의 구형 kernel `3.10.0-862.el7.x86_64` 에서 NFSv4 `SETATTR` 가 새로운 group 이름을 UID/GID로 해석하던 중 `rpc.idmapd` 응답이 지연되면 request가 defer될 수 있다. 이때 NFSv4.1 `SEQUENCE` 는 slot을 `INUSE` 로 표시하지만 response encode와 `nfsd4_sequence_done()` 이 실행되지 않아 slot이 반환되지 않는 경로를 실제로 재현했다.



한 slot만 leak한 최소 재현에서는 `ForeChannel` waiter가 0일 수도 있다. 따라서 waiter가 없다는 사실만으로 slot 문제를 배제하면 안 된다. session reset이나 recovery가 모든 slot의 drain을 기다리는 상황에서는 고착된 한 slot이 전체 복구를 막을 수 있다.

2. 어떤 증상이었나

2026-07-16 재현에서는 LAB8의 `chown` worker가 173초 이상 D-state에 남았다.

```

chown
-> nfs4_proc_setattr
-> nfs4_call_sync_sequence
-> rpc_wait_bit_killable
  
```

storage trace에서는 첫 처리와 재방문이 다음처럼 달랐다.

단계	관측
첫 idmap decode	<code>RQ_USEDEFERRAL</code> , <code>RQ_DROPME</code> 가 설정됨
첫 SEQUENCE 확인	<code>seqid=0x5f</code> , <code>slot_seqid=0x5e</code> , <code>INUSE=0</code>
첫 request 종료	dispatcher가 0을 반환했고 encode와 <code>sequence_done</code> 이 실행되지 않음
<code>rpc.idmapd</code> 재개 후	같은 request가 <code>seqid=0x5f</code> , <code>slot_seqid=0x5f</code> , <code>INUSE=1</code> 로 재방문

단계	관측
이후 retry	약 15초마다 같은 <code>INUSE=1</code> 을 확인하고 완료하지 못함

3. 왜 이 실험을 했나

조사는 다음 순서로 가설을 좁혔다.

1. 2026-06-29 LAB5에서는 NFS user home을 읽는 `sshd: ... [priv]` 가 D-state에 누적됐다.
2. 2026-07-10 LAB8에서 TCP/2049 transport를 일시 차단하자 같은 `rpc_wait_bit_killable` 계열 증상을 재현했지만, network가 복구된 뒤 process도 자연 복구됐다.
3. keytab, `rpc-gssd`, readiness와 mount 순서를 바로잡은 clean boot에서도 구형 storage kernel의 session state가 영구 고착될 수 있는지는 별도 검증이 필요했다.
4. upstream의 idmap deferral 수정 내용이 “NFSv4 decode 중 request drop으로 `NFSD4_SLOT_INUSE` 가 해제되지 않는다”는 경로를 설명하므로, 실제 LAB kernel과 mount에서 그 primitive를 최소 workload로 확인했다.

실험 목적은 많은 client를 동시에 멈추게 하는 것이 아니라, **idmap miss 하나가 slot 하나를 반환 불가능하게 만들 수 있는지**를 확인하는 것이었다.

4. 실험 결과

4.1 실제 LAB 환경의 최소 재현

첫 pass에서 `RQ_DROPME` 가 설정된 뒤 slot이 `INUSE` 로 바뀌었지만 encode와 `nfdsd4_sequence_done()` 이 호출되지 않았다. 8초 뒤 `rpc.idmapd` 가 재개된 후에도 같은 sequence는 이미 사용 중인 slot으로 판단됐다. client retry도 동일 상태를 반복했고 worker는 D-state에서 끝나지 않았다.

따라서 다음 좁은 결론은 확정할 수 있다.

- 당시 storage kernel에서는 idmap decode deferral이 NFSv4.1 session slot 하나를 leak할 수 있다.
- keytab 발급, `rpc-gssd` 시작 순서나 client mount 순서만 고쳐서는 이 server-side kernel 경로를 제거할 수 없다.
- 한 slot leak은 재현했지만 LAB5에서 나중에 관측한 전체 session recovery/drain stall까지 이 최소 실험 하나로 재현한 것은 아니다.
- LAB5 장애 시작 시점의 trace가 없으므로 어떤 실제 operation이 최초 idmap miss를 만들었는지는 알 수 없다.

4.2 A-G: 8초 idmap 지연 kernel matrix

최소 재현과 같은 `idmap-setattr`, NFSv4.1, `sec=krb5p` 조건에서 server와 client kernel 조합을 바꿔 각 5회 실행했다. client OS는 모두 Ubuntu 22.04.5 LTS이며, F/G만 client kernel을 `6.18.38` 로 올렸다.

Case	NFS server OS	Server kernel	Client kernel	결과
A	CentOS Linux 7.9.2009	<code>3.10.0-862.el7.x86_64</code>	<code>6.8.0-134-generic</code>	slot 고착 5/5

Case	NFS server OS	Server kernel	Client kernel	결과
B	CentOS Linux 7.9.2009	3.10.0-1160.el7.x86_64	6.8.0-134-generic	slot 고착 5/5
C	CentOS Stream 10	6.12.74	6.8.0-134-generic	slot 고착 5/5
D	CentOS Stream 10	6.12.75	6.8.0-134-generic	자동 복구 5/5
E	CentOS Stream 10	6.12.0-248.el10.x86_64	6.8.0-134-generic	자동 복구 5/5
F	CentOS Linux 7.9.2009	3.10.0-862.el7.x86_64	6.18.38	slot 고착 5/5
G	CentOS Linux 7.9.2009	3.10.0-1160.el7.x86_64	6.18.38	slot 고착 5/5

구형 server kernel에서는 client kernel을 6.18.38 로 올린 F/G도 모두 고착됐다. 반대로 idmap/slot fix가 포함된 D와 signed Stream kernel E는 모두 자동 복구했다. 이 결과는 문제와 복구 여부를 가르는 주된 변수가 client가 아니라 **NFS server kernel의 idmap deferral 처리**임을 지지한다.

4.3 D/E: 120·300·600초 장시간 idmap 지연 추가 실험

8초 지연만으로는 수정된 kernel이 짧은 지연에서만 우연히 복구한 것인지 판단하기 어려웠다. 그래서 자동 복구한 D/E만 대상으로 idmap cache miss 응답을 120초, 300초, 600초까지 지연하고 각 조건을 3회씩 다시 실행했다.

Case	Server kernel	idmap 지연	원래 chgrp	신규 RPC	최종 snapshot D / FC / lease
D-120s	6.12.75-nfs-slot	120초	성공 3/3	성공 3/3	모두 0 / 0 / 0*
D-300s	6.12.75-nfs-slot	300초	성공 3/3	성공 3/3	모두 0 / 0 / 0
D-600s	6.12.75-nfs-slot	600초	EINVAL 3/3	성공 3/3	모두 0 / 0 / 0
E-120s	6.12.0-248.el10.x86_64	120초	성공 3/3	성공 3/3	모두 0 / 0 / 0
E-300s	6.12.0-248.el10.x86_64	300초	성공 3/3	성공 3/3	모두 0 / 0 / 0
E-600s	6.12.0-248.el10.x86_64	600초	EINVAL 3/3	성공 3/3	모두 0 / 0 / 0

여기서 FC는 NFSv4.1 ForeChannel waiter 수다. 600초 조건의 EINVAL은 원래 chgrp 요청의 application 결과이며, slot 고착 판정과 분리해야 한다. 같은 run에서 idmap 지연을 해제한 뒤 실행한 신규 RPC는 모두 성공했고, 최종 snapshot의 D-state, ForeChannel waiter와 lease_expired도 모두 0이었다. 따라서 600초에서 원래 작업이 실패했더라도 **session slot은 반환됐고 후속 요청을 막지 않았다**.

* D-120s 2회차의 2초 sampler는 종료 직전 D-state 1을 한 번 기록했다. 그러나 직후 final snapshot은 D-state 0이었고, 신규 RPC 성공, ForeChannel 0, lease_expired=0, analyzer verdict=pass였다. 지속된 NFS slot 고착으로 판정하지 않았다.

5. 실제 재현 조건

항목	2026-07-16 조건
client	LAB8, 100.100.100.108
server	lab-storage.lab.decs.internal , 100.100.100.100
mount	/home/tako8/share , NFSv4.1, sec=krb5p , hard
storage kernel	3.10.0-862.el7.x86_64
client 시작 상태	D-state 0, NFS RPC task 0, readiness ready
test identity	LAB8 local group decs_idmap_repro_424242 , GID 424242
test object	/home/tako8/share/.decs-idmap-slot-repro-20260716
fault	storage rpc.idmapd 에 8초간 SIGSTOP
workload	test file에 GID 424242 를 지정하는 chown(2) 한 번
안전장치	storage-local 8초·20초 SIGCONT timer를 먼저 예약
금지한 복구	재현 후 client reboot, unmount, signal, service restart를 하지 않음

boot 순서도 별도로 확인했다. LAB8은 rpc-gssd active → Kerberos/NFS readiness 완료 → mount 순서였고, 기존 /etc/krb5.keytab 의 KVNO는 2였다. 따라서 이번 결과를 keytab 생성 누락이나 잘못된 mount 시작 순서로 설명할 수 없다.

6. 당시 실행한 명령

경고: 아래 명령은 2026-07-16의 승인된 실제 실행 기록이다. storage의 rpc.idmapd 를 멈추므로 일반 점검이나 운영 재현 절차로 다시 실행하면 안 된다. 재실험은 8절의 격리 VM harness를 사용한다. 특히 kprobe 인자 register는 당시 x86_64 kernel symbol에만 맞춘 값이라 다른 kernel에서 그대로 사용하지 않는다.

6.1 client 사전 확인과 test 값 준비

LAB8에서 D-state와 RPC task가 없는지 확인한 뒤, storage에 없던 group 이름을 client가 encode하도록 local group과 test file을 준비했다.

```

ps -eo stat= | awk '$1 ~ /^D/ {n++} END {print "d_state=" (n+0)}'
grep -c '^RPC task' /proc/net/rpc/nfs 2>/dev/null || true
findmnt -T /home/tako8/share -n -o TARGET,SOURCE,FSTYPE,OPTIONS

sudo groupadd -g 424242 decs_idmap_repro_424242

sudo bash -c '
p=/home/tako8/share/.decs-idmap-slot-repro-20260716
rm -f "$p"
touch "$p"
chmod 0666 "$p"
stat -c "path=%n uid=%u gid=%g mode=%a ino=%i" "$p"
'

```

fault 전에 GID 424241 을 사용한 정상 `chown` 을 실행해 같은 경로가 idmap 지연 없이는 encode와 `sequence_done` 까지 완료되는 것을 먼저 확인했다.

```

sudo python3 -c 'import os; p="/home/tako8/share/.decs-idmap-slot-repro-20260716"; print("before", os.stat(p).st_gid); os.chown(p, -1, 424241); print("after", os.stat(p).st_gid)'

```

6.2 storage trace 설정

당시 storage에서 idmap decode, slot 검사, dispatcher return, response encode와 slot 반환 여부를 한 trace instance에 기록했다.

```

T=/sys/kernel/debug/tracing
I="$T/instances/decs-idmap-slot-repro"
sudo mkdir -p "$I"

for event in uid gid check_slot dispatch dispatch_ret encode sequence sequence_done; do
    echo "-:decs_idmap/$event" | sudo tee -a "$T/kprobe_events" >/dev/null || true
done

echo 'p:decs_idmap/uid nfsd_map_name_to_uid rqstp=%di name=+0(%si):string namelen=%dx:u32' \
    | sudo tee -a "$T/kprobe_events" >/dev/null
echo 'p:decs_idmap/gid nfsd_map_name_to_gid rqstp=%di name=+0(%si):string namelen=%dx:u32' \
    | sudo tee -a "$T/kprobe_events" >/dev/null
echo 'p:decs_idmap/check_slot check_slot_seqid seqid=%di:u32 slot_seqid=%si:u32 slot_inuse=%dx:u32' \
    | sudo tee -a "$T/kprobe_events" >/dev/null
echo 'p:decs_idmap/dispatch nfsd_dispatch rqstp=%di' \
    | sudo tee -a "$T/kprobe_events" >/dev/null
echo 'r:decs_idmap/dispatch_ret nfsd_dispatch ret=$retval:s32' \
    | sudo tee -a "$T/kprobe_events" >/dev/null
echo 'p:decs_idmap/encode nfs4svc_encode_compoundres rqstp=%di' \
    | sudo tee -a "$T/kprobe_events" >/dev/null
echo 'p:decs_idmap/sequence nfsd4_sequence rqstp=%di cstate=%si seq=%dx' \
    | sudo tee -a "$T/kprobe_events" >/dev/null
echo 'p:decs_idmap/sequence_done nfsd4_sequence_done resp=%di' \
    | sudo tee -a "$T/kprobe_events" >/dev/null

echo 0 | sudo tee "$I/tracing_on" >/dev/null
echo 0 | sudo tee "$I/events/enable" >/dev/null
sudo sh -c ": > '$I/trace'"
echo 8192 | sudo tee "$I/buffer_size_kb" >/dev/null

for event in \
    decs_idmap/uid decs_idmap/gid decs_idmap/check_slot decs_idmap/dispatch \
    decs_idmap/dispatch_ret decs_idmap/encode decs_idmap/sequence \
    decs_idmap/sequence_done sunrpc/svc_process sunrpc/svc_defer \
    sunrpc/svc_revisit_deferred sunrpc/svc_drop sunrpc/svc_send
do
    echo 1 | sudo tee "$I/events/$event/enable" >/dev/null
done
echo 1 | sudo tee "$I/tracing_on" >/dev/null

```

6.3 fault와 단일 workload 실행

storage에서 반드시 자동 재개 timer 두 개가 active인지 확인한 뒤 `rpc.idmapd`를 멈췄다.

```
pid="$(pgrep -xo rpc.idmapd)"

sudo systemd-run --unit=decs-idmapd-auto-resume-8s \
  --on-active=8s --timer-property=AccuracySec=1s \
  /bin/kill -CONT "$pid"
sudo systemd-run --unit=decs-idmapd-auto-resume-20s \
  --on-active=20s --timer-property=AccuracySec=1s \
  /bin/kill -CONT "$pid"

sudo systemctl is-active --quiet decs-idmapd-auto-resume-8s.timer
sudo systemctl is-active --quiet decs-idmapd-auto-resume-20s.timer
sudo kill -STOP "$pid"
```

그 직후 LAB8의 별도 systemd worker에서 `chown(2)` 한 번만 실행했다.

```
sudo systemd-run --unit=decs-idmap-slot-repro-worker --no-block \
  /usr/bin/python3 -c \
  "import os; os.chown('/home/tako8/share/.decs-idmap-slot-repro-20260716', -1, 424242)"
```

6.4 결과 확인과 trace 보존

새로운 NFS I/O를 만들지 않고 worker, local kernel state와 이미 동작 중이던 forensics state를 확인했다.

```
systemctl --no-pager --full status decs-idmap-slot-repro-worker.service
ps -eo pid=,ppid=,stat=,comm=,wchan:32=,args= \
  | awk '$3 ~ /^D/ {print}'
grep -E 'lease_expired|RPC task|xprt_pending|ForeChannel' \
  /proc/net/rpc/nfs 2>/dev/null || true

cat /run/decs-nfs-forensics/status.json
cat /var/lib/decs-nfs-gss/recovery.state
cat /var/lib/decs-nfs-gss/canary.state
```

storage trace는 다음 pattern을 기준으로 추렸다.

```
I=/sys/kernel/debug/tracing/instances/decs-idmap-slot-repro
sudo sh -c "echo 0 > '$I/tracing_on'"
sudo cp "$I/trace" /var/tmp/decs-idmap-slot-repro-20260716.trace

sudo grep -E \
  'decs_idmap_gid|svc_defer|svc_revisit_deferred|svc_drop:|check_slot|sequence_done|dispatch_ret' \
  /var/tmp/decs-idmap-slot-repro-20260716.trace
```

7. Monitoring과 운영 대응

재현 당시 guardrail은 의도대로 동작했다.

- forensics watcher가 `nfs_d_state` 이유로 incident snapshot을 보존했다.
- recovery는 `status=blocked`, `diagnosis=nfs_d_state_detected`, `retry_inhibited=1`로 바뀌었다.
- canary도 같은 진단으로 차단돼 추가 NFS request를 만들지 않았다.
- packet ring과 kernel trace는 계속 증거를 수집했다.

운영에서 같은 증상이 보이면 다음 순서를 따른다.

1. NFS path를 읽는 `find`, `du`, `stat`, canary를 추가 실행하지 않는다.
2. D-state stack, `/proc/net/rpc/nfs`, forensics status와 snapshot을 보존한다.
3. mount가 이미 있으면 자동 remount와 `rpc-gssd` 반복 restart를 차단한다.
4. storage kernel, `rpc.idmapd`, network와 NFS session 증거를 함께 확인한다.
5. 상태가 자연 복구되지 않으면 유지보수 창외 client reboot 또는 storage 조치를 수동 결정한다. reboot는 상태를 지울 뿐 원인 수정은 아니다.
6. 영구 조치는 idmap deferral fix가 포함된 storage kernel로 올리고 격리 회귀 실험을 통과시키는 것이다.

운영 mount를 `soft`로 바꾸는 방식은 timeout을 application error로 노출하고 data integrity 위험을 만들 수 있어 해결책으로 사용하지 않는다.

8. 격리 VM에서 다시 검증하는 방법

반복 실험은 LAB8 local disk의 `decs-nfs-slot-isolated` network에서 수행한다. 이 network는 forwarding이 없고 `100.100.100.0/24`, `192.168.1.0/24`와 운영 storage hostname을 거부한다. VM disk, trace와 result도 NFS share가 아니라 LAB8 local filesystem에 둔다.

kernel 비교 기준은 다음과 같다.

Case	Server kernel	Client kernel	목적
A	CentOS 7 3.10.0-862	Ubuntu 6.8.0-134	운영 취약 baseline
B	CentOS 7 3.10.0-1160	Ubuntu 6.8.0-134	CentOS 7 최종 kernel 비교

Case	Server kernel	Client kernel	목적
C	upstream 6.12.74	Ubuntu 6.8.0-134	fix 이전 positive control
D	upstream 6.12.75	Ubuntu 6.8.0-134	idmap/slot fix 비교 control
E	signed Stream 6.12.0-248.el10	Ubuntu 6.8.0-134	운영 후보
F	CentOS 7 3.10.0-862	kernel.org 6.18.38	새 client kernel 비교
G	CentOS 7 3.10.0-1160	kernel.org 6.18.38	새 client kernel 비교

준비와 격리는 VM lab README의 절차를 따른다.

```
cd /home/jy/server_manage/kerberos-nfs/test/lab-kerberos-poc/vm-slot-regression

sudo ./bin/provision_lab8_vms.sh "$PWD"
./bin/configure_vm_stack.sh
sudo ./bin/prefetch_kernel_matrix.sh
sudo ./bin/verify_kernel_sources.sh
sudo ./bin/build_upstream_kernels_vm.sh
sudo ./bin/prepare_centos7_domains.sh "$PWD"
sudo ./bin/isolate_test_vms.sh
```

단일 비교는 같은 idmap-setattr, sec=krb5p, 8초 조건을 case별로 실행한다.

```
./bin/switch_kernel_case.sh A
NFS_SLOT_EXPECT_VULNERABLE=1 \
./bin/run_single_case.sh A krb5p idmap-setattr 8 1

./bin/switch_kernel_case.sh D
NFS_SLOT_EXPECT_VULNERABLE=0 \
./bin/run_single_case.sh D krb5p idmap-setattr 8 1
```

먼저 실행 수만 확인하려면 plan-only mode를 쓴다.

```
MATRIX_PLAN_ONLY=1 \
MATRIX_CASES='A C D E' \
MATRIX_SECURITY='krb5p' \
MATRIX_SCENARIOS='idmap-setattr' \
MATRIX_DURATIONS='8' \
MATRIX_REPETITIONS=3 \
./bin/run_matrix.sh
```

취약 case가 고착되면 harness는 evidence를 archive한 뒤 **격리 VM만** reset한다. 그 reset은 자동 복구 성공으로 계산하지 않는다. 운영 LAB8 host와 운영 NFS mount는 이 harness의 복구 대상이 아니다.

9. 원본 증거와 코드

- 2026-07-16 실제 재현 결과
- curated trace evidence
- 2026-07-10 NFS transport fault 실험
- 격리 VM slot regression lab
- single-case harness
- result analyzer
- upstream stable patch 설명
- NFS forensics 구현

보존된 storage raw trace의 SHA-256은 `5fe7fbab2b8aa712ff71afe5156438b4e880edcc39cf16a00592e22d578bac59` 다.

NFSv4.1 session slot 고착 해결 및 운영 적용

상태: 2026-07-21~22 LAB storage의 OS와 kernel을 교체하고 Kerberos/NFS 구성을 복원했다. 2026-07-23 LAB8 운영 mount에서 `rpc.idmapd` 를 15초 지연한 직접 시험에서도 session slot이 영구 고착되지 않았다.

이 문서는 **NFSv4.1 session slot 고착**에서 확인한 server-side kernel 문제를 실제 LAB storage에서 어떻게 제거하고 운영 구성을 복원했는지 기록한다. 원인 재현 과정과 fault-injection 명령은 기존 문서에 두고, 여기에는 해결 조치와 안전한 사후 검증만 정리한다.

1. 해결 결론

이 장애의 근본 조치는 **NFS server인 lab-storage**의 kernel을 `idmap deferral` 수정이 포함된 kernel 계열로 교체하는 것이다. client 재부팅, NFS remount 또는 `rpc-gssd` 재시작은 이미 고착된 상태를 지울 수는 있지만 server-side slot leak 경로를 없애지 못한다.

항목	변경 전	변경 후
작업 서버	<code>lab-storage.lab.decs.internal</code>	동일
OS	CentOS Linux 7	CentOS Stream 10
kernel	<code>3.10.0-862.el7.x86_64</code>	<code>6.12.0-250.el10.x86_64</code>
데이터	<code>/294t</code> XFS	기존 <code>/dev/sda1</code> 을 포맷하지 않고 <code>/294t</code> 에 재연결

항목	변경 전	변경 후
AD/KDC	LAB2 Samba AD	동일, storage keytab은 LAB2에서 재생성
NFS 보안	Kerberos NFS	SSSD·gssproxy·NFS export를 복원

격리 VM 비교에서는 upstream 6.12.74 까지 slot 고착이 발생했고 6.12.75 에서 자동 복구했다. CentOS Stream 10 의 signed kernel 6.12.0-248.el10 도 같은 시험을 통과했다. 현재 운영 kernel 6.12.0-250.el10 은 그보다 뒤에 설치한 같은 배포판 kernel이다.

여기서 250.el10 은 upstream Linux 6.12.250 을 뜻하지 않고 CentOS/RHEL 계열의 package release 번호다. 초기 운영 적용은 격리 VM 회귀 결과와 교체 후 실제 서비스 상태를 근거로 판정했고, 2026-07-23에는 승인된 15초 지연 시험을 LAB8 운영 mount에서 한 번 추가해 현재 kernel에서도 영구 slot 고착이 없음을 확인했다.

2. 서버별 역할

서버	이번 조치에서의 역할
lab-storage.lab.decs.internal (100.100.100.100)	kernel 문제를 제거한 NFS server. OS, network, data mount, SSSD, keytab, gssproxy와 export를 복원한 대상
LAB2 / 100.100.100.102	운영 Samba AD DC, DNS/KDC. storage machine account와 NFS service principal의 원본
LAB1~LAB10	NFS client. 각 서버의 기존 /etc/fstab 으로 /home/takoN/share 를 다시 mount
LAB8 host	/home/tako8/share 운영 mount에서 15초 idmap 지연을 직접 검증한 client
LAB8의 격리 VM	kernel matrix와 장시간 지연 회귀 시험 전용

keytab은 예전 파일을 backup해서 되돌리지 않았다. LAB2 AD에 등록된 storage machine account와 SPN을 기준으로 새 keytab을 만든 뒤 lab-storage 의 /etc/krb5.keytab 에 설치했다. 따라서 keytab 생성 작업은 LAB2에서, 설치와 검증은 lab-storage 에서 수행한다.

3. 실제 적용 내용

3.1 lab-storage : OS와 kernel 교체

2026-07-21 22:38 KST에 kernel-core-6.12.0-250.el10.x86_64 를 설치했고 22:49 KST부터 이 kernel로 부팅해 운영 중이다. hostname과 두 network를 다음 상태로 복원했다.

용도	interface	주소
관리망	enp24s0f1	192.168.1.20/24
storage망	enp101s0f0	100.100.100.100/24

hostname은 `lab-storage.lab.decs.internal`, timezone은 `Asia/Seoul`이며 NTP 동기화 상태는 `yes`다.

3.2 lab-storage : 기존 데이터 디스크 재연결

RAID volume이 처음 보이지 않은 이유

OS 설치 직후에는 294TB 데이터 RAID가 정상적인 `/dev/sda`로 나타나지 않았다. 이 장비의 controller는 다음 Areca ARC-1284다.

```
17:00.0 RAID bus controller: Areca Technology Corp. ARC-12x4
Subsystem: Areca Technology Corp. ARC-1284 24 Port
PCI ID: 17d3:1214
```

CentOS Stream 10의 운영 kernel 설정을 확인하면 ARC-1284용 `arcmsr`가 빠져 있다.

```
grep CONFIG_SCSI_ARCMSR /boot/config-"$(uname -r)"
```

당시 결과는 다음과 같았다.

```
# CONFIG_SCSI_ARCMSR is not set
```

먼저 `kernel-modules-extra`를 설치했지만 이 kernel용 `arcmsr.ko`는 들어 있지 않았다.

```
sudo dnf install -y kernel-modules-extra
modinfo arcmsr
```

ELRepo의 `kmod-arcmsr`도 확인했지만 EL10용 package가 없었다. 최종 해결은 ELRepo package 설치가 아니라 **현재 실행 kernel에 맞춰 `arcmsr`를 외부 module로 직접 빌드하는 것이었다.**

`arcmsr` 외부 module 빌드와 설치

2026-07-21에 실제로 설치한 build dependency는 다음과 같다. `kernel-devel`의 version은 반드시 module을 사용할 kernel과 정확히 같아야 한다.

```
KVER=$(uname -r)

sudo dnf install -y \
    "kernel-devel-$KVER" \
    gcc make elfutils-libelf-devel xz
```

Linux 6.12의 `drivers/scsi/arcmsr`에서 `Makefile`, `arcmsr.h`, `arcmsr_attr.c`, `arcmsr_hba.c`를 임시 build directory로 가져왔다. Stream 10 kernel에 backport된 API에 맞게 다음 호환 조정을 적용한 뒤 외부 module로 빌드했다.

- `slave_configure` 를 `sdev_configure` 와 `struct queue_limits` signature로 변경
- `bios_param` 의 `struct block_device` 를 `struct gendisk` 로 변경
- `del_timer_sync()` 를 `timer_delete_sync()` 로 변경
- `from_timer()` 를 `timer_container_of()` 로 변경
- `sysfs bin_attribute` 인자를 `const` 로 변경
- PCI device ID table을 `const` 로 변경

실제 build 형태는 다음과 같다. upstream 6.12 source를 수정하지 않고 그대로 빌드하면 Stream 10 kernel API와 맞지 않으므로 위 호환 조정이 필요하다.

```
KVER=$(uname -r)
BUILD_DIR=$(mktemp -d /tmp/arcmsr-build.XXXXXX)

# BUILD_DIR에 호환 조정을 마친 다음 파일을 준비한다.
# Makefile arcmsr.h arcmsr_attr.c arcmsr_hba.c

make -C "/usr/src/kernels/$KVER" \
    M="$BUILD_DIR" \
    CONFIG_SCSI_ARCMSR=m \
    modules

modinfo "$BUILD_DIR/arcmsr.ko"
```

먼저 `insmod` 로 현재 boot에서 controller와 RAID volume이 정상적으로 인식되는지 확인했다.

```
sudo insmod "$BUILD_DIR/arcmsr.ko"

lspci -nnk -s 17:00.0
lsblk -e7 -o NAME,SIZE,TYPE,FSTYPE,UUID,MOUNTPOINTS,MODEL
```

`ARC-1284-VOL#000` 과 partition UUID가 확인된 후 module을 영구 경로에 설치하고 boot 때 자동으로 load되도록 등록했다.

```
KVER=$(uname -r)

sudo install -D -m 0644 "$BUILD_DIR/arcmsr.ko" \
    "/lib/modules/$KVER/extra/arcmsr.ko"
sudo depmod -a "$KVER"

printf 'arcmsr\n' | sudo tee /etc/modules-load.d/arcmsr.conf >/dev/null
```

현재 설치된 module은 다음 상태다.

```
path=/lib/modules/6.12.0-250.el10.x86_64/extra/arcmsr.ko
version=v1.51.00.14-20230915
sha256=8c78e3bb666abc49a17419904ab5911e0fed0c9cb885f0e2c18333b118d7865e
driver=arcmsr
volume=ARC-1284-VOL#000
```

이 module은 RPM 소유 파일이 아니며 서명되지 않은 외부 module이다. 현재 server는 Secure Boot가 disabled라 load됐고 kernel log에는 out-of-tree module taint가 기록된다.

kernel 업데이트 주의: 새 kernel을 설치하면 그 version에 맞는 `kernel-devel` 로 `arcmsr.ko` 를 다시 빌드해 새 kernel의 `/lib/modules/<새-version>/extra/` 에 설치하고 `depmod` 를 실행해야 한다. 이를 준비하지 않고 새 kernel로 reboot하면 ARC-1284 volume이 나타나지 않아 `/294t` mount가 실패할 수 있다.

XFS mount 복원

driver load 후 RAID volume은 `/dev/sda` , 기존 XFS partition은 `/dev/sda1` 로 나타났다. 데이터 디스크를 포맷하지 않고 다음 fstab 항목으로 다시 연결했다.

```
UUID=2a417935-7537-4301-bbcf-5ab2ca836af5 /294t xfs defaults 1 2
```

2026-07-22 확인 결과는 `/dev/sda1` → `/294t` , XFS, `rw` 다. 이 조치에서는 `/294t` 를 다시 포맷하지 않았다.

```
lsblk -e7 -o NAME,SIZE,FSTYPE,UUID,MOUNTPOINTS,MODEL
findmnt -rn -o TARGET,SOURCE,FSTYPE,OPTIONS /294t
lspci -nnk -s 17:00.0
lsmod | grep '^arcmsr'
```

2026-07-22 이후 kernel log에는 일부 allocation group의 AGFL을 reset했고 block이 leak됐다는 XFS warning도 기록됐다. 이는 `arcmsr` load와 `/294t` mount 성공 여부와 별개인 filesystem metadata 점검 항목이다. 운영 export가 연결된 상태에서 `xfs_repair` 를 실행하지 말고, 별도 유지보수 창에 NFS를 중단하고 `/294t` 를 unmount한 뒤 offline 점검·복구해야 한다.

3.3 LAB2와 lab-storage : AD 설정과 SSSD·keytab 복원

LAB2: AD server와 storage principal 준비

LAB2는 `LAB.DECS.INTERNAL` realm의 Samba AD DC와 KDC 역할을 한다. storage망 주소는 `100.100.100.102` 이며, `lab-storage` 의 Kerberos 설정에서 KDC와 admin server로 사용한다.

LAB2 AD에서 `lab-storage` machine account와 NFS service principal을 확인한 뒤 keytab을 새로 생성했다. OS 재설치 전 keytab을 backup해서 복원하지 않고 AD를 원본으로 재발급한 것이다.

```
LAB-STORAGE$@LAB.DECS.INTERNAL  
nfs/lab-storage.lab.decs.internal@LAB.DECS.INTERNAL
```

생성된 keytab의 machine principal과 NFS service principal KVNO는 모두 2였다.

lab-storage : SSSD와 keytab 설치·검증

lab-storage 는 Kerberos realm을 LAB.DECS.INTERNAL , KDC와 admin server를 LAB2의 100.100.100.102 로 설정했다. SSSD는 다음 정책으로 AD의 RFC2307 UID/GID를 그대로 조회한다.

```
[domain/lab.decs.internal]  
ad_domain = lab.decs.internal  
krb5_realm = LAB.DECS.INTERNAL  
id_provider = ad  
access_provider = permit  
ldap_id_mapping = False  
use_fully_qualified_names = False
```

LAB2에서 재생성한 keytab은 lab-storage 의 /etc/krb5.keytab 에 root 전용 권한으로 설치했다.

lab-storage 에서 machine principal로 TGT를 받은 뒤 NFS principal을 keytab과 대조한 결과는 다음과 같다.

```
nfs/lab-storage.lab.decs.internal@LAB.DECS.INTERNAL: kvno = 2, keytab entry valid
```

NFS service principal 자체를 client principal처럼 kinit -k 하는 것은 이 구성의 검증 방법이 아니다. 아래 5.1절처럼 machine principal로 먼저 kinit 한 뒤 kvno -k 를 사용한다.

3.4 lab-storage : NFS service와 export 복원

nfs-server , sssd , gssproxy 를 활성화하고 NFSv4 port 2049/tcp 와 관련 firewall service를 복원했다. 2026-07-22 확인 시 nfsd thread는 16개였고 nfs-server , sssd , gssproxy 가 모두 active였다.

Kerberos 대상 export는 다음 상태다.

경로	client	security flavor
/294t/dcloud/share	100.100.100.101 ~ 110	krb5p:krb5i:krb5
/294t/share/test-krb	지정 시험 client	krb5p
/294t/health/nfs-gss-canary	100.100.100.101 ~ 110	krb5 , read-only

기존 sec=sys 전용 경로는 별도 호환 export로 유지했다. Kerberos export가 정상이라는 이유로 기존 sec=sys export를 임의로 삭제하지 않는다.

3.5 LAB1~LAB10: 기존 fstab mount 복원

client별 mount 설정을 새로 생성하지 않고 각 서버에 이미 있던 `/etc/fstab` 을 사용했다. mount가 없는 client에서 만 `sudo mount -a` 를 실행해 `/home/takoN/share` 를 다시 연결했다.

2026-07-22 재점검에서 LAB1~LAB10 모두 다음 공통 상태를 보였다.

```
source=lab-storage.lab.decs.internal:/294t/dcloud/share
fstype=nfs4
vers=4.1
hard
sec=krb5
addr=100.100.100.100
```

최초 병렬 점검 중 LAB7에서 짧은 D-state process 1개가 관측됐지만 즉시 수행한 local 상태 재확인에서는 사라졌다. 같은 시점에 NFS RPC task, transport pending, ForeChannel waiter와 NFS D-state가 모두 0이었고 recovery 상태도 `healthy` 였다. 이를 지속된 session slot 고착으로 판정하지 않았다.

3.6 LAB8 운영 mount: 15초 `rpc.idmapd` 지연 시험

과거 실행 기록: 아래 내용은 안전장치를 준비한 뒤 한 번 수행한 운영 검증 결과다. 일반 점검 절차가 아니며 이 문서만 보고 운영 storage에서 반복 실행하지 않는다.

격리 VM 결과와 별도로 현재 운영 중인 storage kernel에서 같은 idmap 지연 경로가 영구 slot 고착을 만들지 않는지 직접 확인했다.

항목	시험 조건
NFS client	LAB8 host의 운영 mount <code>/home/tako8/share</code>
mount	NFSv4.1, <code>sec=krb5</code> , <code>hard</code>
NFS server	<code>lab-storage.lab.decs.internal</code>
storage kernel	<code>6.12.0-250.el10.x86_64</code>
workload	시험 container 내부에서 새 GID <code>424242</code> 로 test file에 <code>chown</code>
idmap fault	storage의 <code>rpc.idmapd</code> 응답을 15초 지연
안전장치	15초 자동 재개 timer와 독립적인 30초 이중 자동 재개 timer

지연 중에는 `chown` 이 `rpc_wait_bit_killable` 경로에서 일시적으로 D-state에 들어갔다. 15초 자동 재개 후 worker는 더 이상 D-state에 남지 않고 종료됐다. 원래 `chown` 의 application 결과는 `EINVAL` 이었다.

```
/bin/chown: changing group of '.../chown-target': Invalid argument
```

이 `EINVAL` 은 “`chown` 성공”을 의미하지 않는다. 하지만 취약 kernel에서처럼 동일 요청이 `INUSE` slot에 계속 걸려 영구 D-state로 남지도 않았다. 이후 test file 삭제 RPC가 성공했고 client reboot, unmount, NFS service restart 없이 정상 상태로 돌아왔다. 후속 확인 결과는 다음과 같았다.

판정 항목	결과
지속 NFS D-state	0
NFS RPC task	0
transport pending	0
ForeChannel waiter	0
<code>lease_expired</code>	0
후속 NFS 작업	test file 삭제 성공

따라서 이 시험은 운영 kernel `6.12.0-250.el10` 에서 15초 idmap 지연이 일시적인 RPC 대기를 만들 수는 있지만 **NFSv4.1 session slot**을 영구 고착시키지는 않았음을 보여준다. 원래 operation의 application 결과와 session recovery 판정은 분리해야 한다.

4. 해결 판정 기준

이번 조치는 다음을 모두 만족해 운영 적용 완료로 판정했다.

- `lab-storage` 가 취약한 `3.10.0-862.el7` 이 아니라 `6.12.0-250.el10` 으로 부팅돼 있다.
- `/294t` 가 기존 XFS data disk에서 `rw` 로 mount돼 있다.
- LAB2 machine principal의 TGT 발급과 NFS service principal의 keytab 검증이 성공한다.
- SSSD에서 AD 사용자의 RFC2307 UID/GID가 조회된다.
- `nfs-server` , `sssd` , `gssproxy` 가 active이고 NFSv4 port가 listen한다.
- Kerberos NFS export가 LAB1~LAB10 storage망 주소에 열려 있다.
- LAB1~LAB10에서 기존 fstab의 NFSv4.1 `sec=krb5` mount가 확인된다.
- 지속되는 NFS D-state, RPC task, ForeChannel waiter 또는 lease expiry가 없다.
- LAB8 운영 mount의 15초 idmap 지연 후에도 workload가 영구 D-state에 남지 않고 후속 NFS RPC가 성공한다.

이 판정은 “2026-06-29 LAB5 장애의 최초 원인이 반드시 이 bug였다”는 뜻은 아니다. 당시 시작 시점의 storage trace가 없어 과거 장애 원인은 여전히 강한 후보로 남는다. 이번에 확정한 것은 구형 storage kernel에 실제 slot leak 경로가 있었고, 그 kernel을 운영에서 제거했다는 사실이다.

5. 안전한 사후 검증

5.1 작업 서버: `lab-storage`

```

hostname -f
uname -r
findmnt -rn -o TARGET,SOURCE,FSTYPE,OPTIONS /294t

systemctl is-active nfs-server sssd gssproxy
sudo exportfs -v
sudo ss -lnt | grep ':2049 '

DECS_KRB5CC_DIR=$(mktemp -d /tmp/decs-storage-kvno.XXXXXX)
DECS_KRB5CC="FILE:$DECS_KRB5CC_DIR/ccache"

sudo env KRB5CCNAME="$DECS_KRB5CC" \
  kinit -k -t /etc/krb5.keytab 'LAB-STORAGE$@LAB.DECS.INTERNAL'
sudo env KRB5CCNAME="$DECS_KRB5CC" \
  kvno -k /etc/krb5.keytab \
  nfs/lab-storage.lab.decs.internal@LAB.DECS.INTERNAL
sudo env KRB5CCNAME="$DECS_KRB5CC" kdestroy
rmdir "$DECS_KRB5CC_DIR"

```

기대 kernel은 6.12.0-250.el10.x86_64 이상이며, kvno 결과에는 keytab entry valid 가 나와야 한다.

5.2 작업 서버: 각 NFS client LAB1~LAB10

아래의 N 은 현재 접속한 LAB 서버 번호로 바꾼다. 이미 mount돼 있으면 반복 remount하지 않는다. mount가 없고 D-state도 없을 때만 기존 fstab으로 mount한다.

```

ps -e -o state=,pid=,etimes=,comm=,wchan:48=,args= \
  | awk '$1 ~ /^D/ {print}'

findmnt -T /home/takoN/share -o TARGET,SOURCE,FSTYPE,OPTIONS

# findmnt에 결과가 없고 D-state process도 없을 때만 실행
sudo mount -a
findmnt -T /home/takoN/share -o TARGET,SOURCE,FSTYPE,OPTIONS

```

container의 실제 사용자 권한까지 확인할 때는 container 안에서 해당 사용자로 실행한다. 다른 사용자의 홈이나 파일에 임의로 chown 하지 않는다.

```
id
ls -al "$HOME/share" | sed -n '1,40p'

probe="$HOME/share/.nfs-post-upgrade-check.${hostname -s}.${}"
printf 'nfs post-upgrade check\n' >"$probe"
grep -Fx 'nfs post-upgrade check' "$probe"
ls -ln "$probe"
rm -f "$probe"
```

`ls -al`의 소유자·그룹이 `nobody`가 아니라 AD의 사용자·그룹 이름으로 보이고, 생성·읽기·삭제가 모두 성공해야 한다.

6. 같은 증상이 다시 보일 때

NFS `hard` mount의 process가 D-state에 들어가면 NFS 경로를 읽는 `find`, `du`, `stat`, 반복 canary를 추가 실행하지 않는다. 새로운 NFS request와 D-state process를 더 만들 수 있다.

1. 먼저 client에서 `uname -r`, D-state stack, `/proc/net/rpc/nfs`, forensics와 recovery state를 local filesystem에서 수집한다.
2. `lab-storage`가 실수로 구형 kernel로 부팅되지 않았는지 확인한다.
3. 자동 remount, 강제 unmount, `rpc-gssd` 반복 restart를 중단한다.
4. storage의 network, nfsd, gssproxy와 session trace를 함께 확인한다.
5. 2026-07-23의 승인된 단일 시험을 일반 점검처럼 반복하지 않는다. 추가 재실험은 LAB8의 격리 VM harness에서 수행한다.
6. client reboot는 고착된 client state를 지우는 복구 수단일 뿐 근본 해결로 기록하지 않는다.

운영 mount를 `soft`로 바꾸는 것도 해결책으로 사용하지 않는다. timeout을 application error로 노출하고 data integrity 위험을 만들 수 있다.

7. 증거와 관련 문서

- 원인 재현과 kernel matrix
- 실제 LAB8 재현 결과
- 격리 VM 회귀 시험 절차
- LAB storage OS 재설치 후 복원 절차

Monitoring

원본: md/system/monitoring/index.md, md/system/monitoring/design.md, md/system/monitoring/operations.md

역할: FARM/LAB 서버, NFS, GPU와 사용자 container의 실행 상태를 반복 관측하고, Prometheus·Grafana·Alertmanager를 통해 시각화와 경보를 제공한다.

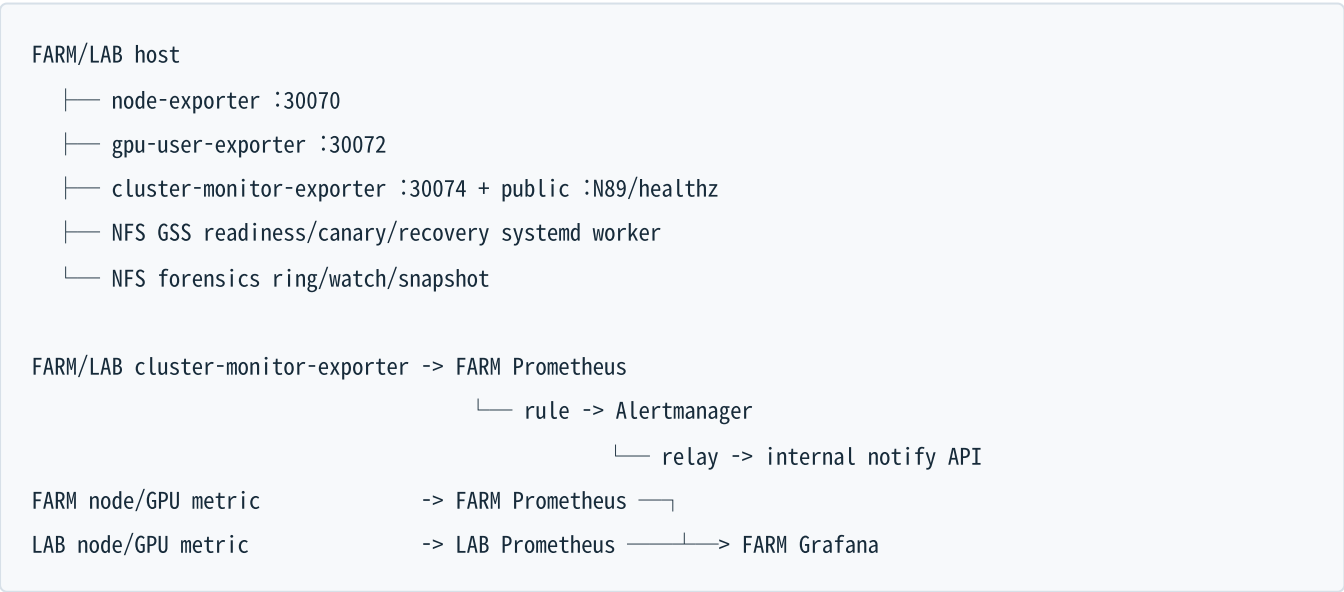
이 문서는 monitoring 문서의 시작점이다. **설계**는 수집 데이터가 어떤 구성 요소를 거쳐 metric과 alert가 되는지 설명하고, **운영**은 배포·점검·장애 대응 명령과 코드 위치를 설명한다.

문서 구성

문서	핵심 내용
현재 페이지	책임 범위, 핵심 구성 요소와 운영 원칙
설계	exporter, Prometheus, Grafana, Alertmanager, GSS health와 forensics의 데이터 흐름
운영	배포, endpoint 점검, metric/alert 코드 링크, 장애별 진단 순서

처음 보는 사람은 **설계 2절의 전체 구성**에서 배치 영역과 데이터 흐름을 먼저 확인하고, 실제 점검이나 배포가 필요할 때 **운영 문서**로 이동한다. 장애 대응 중 설계 문서의 구성 요소 설명과 운영 문서의 명령을 번갈아 읽지 않도록, 반복해서 사용하는 endpoint·metric·진단 순서는 운영 문서에 모았다.

한눈에 보는 구조



핵심 구성 요소

구성 요소	endpoint/산출물	역할
node-exporter	:30070/metrics	CPU, memory, filesystem, disk와 network 같은 일반 host 자원 노출
gpu-user-exporter	:30072/metrics , :30072/-/healthy	GPU process를 DB의 실제 사용자와 container에 연결
cluster-monitor-exporter	:30074/metrics , :30074/healthz	mount, GSS, GPU, Docker, container, 연결성, D-state 수집
public health listener	서버별 N89/healthz	외부에서 NAT와 host 도달 가능성 확인
NFS GSS worker	/run · /var/lib state	readiness, canary와 guarded missing-mount recovery
NFS forensics	status JSON, local incident snapshot	제한된 packet/kernel trace를 보존
keytab health check	profile status JSON	AD KVNO, storage keytab과 GSS service drift를 읽기 전용 확인
FARM Prometheus/Alertmanager	FARM Kubernetes release	모든 cluster-monitor-exporter 를 중앙 수집하고 rule 평가와 alert routing 수행
LAB Prometheus	LAB Kubernetes release, NodePort 30073	LAB node/GPU metric을 별도 저장; Alertmanager와 Grafana는 비활성화
Grafana	FARM NodePort 30080	FARM/LAB Prometheus datasource를 한 UI에서 조회

운영 경계

- 지속 관측은 `monitoring` , 부팅 orchestration은 `remote-operations` , host desired state는 `server-state` 에서 관리한다.
- `cluster-monitor-exporter` 가 직접 mount하지 않는다. 별도 systemd recovery worker가 fstab, DNS/RPC, Kerberos readiness, canary와 D-state gate를 통과한 **missing mount**만 복구한다.
- keytab checker는 AD password, NAS keytab, service와 mount를 변경하지 않는다.
- forensics는 증거를 수집할 뿐 mount/restart/reboot를 수행하지 않는다.
- 자동 복구는 stopped container 시작, SSH 시작, 안전성이 확인된 NVML symlink repair처럼 영향 범위가 제한된 작업에만 사용한다.
- FARM과 LAB Prometheus는 장애와 저장소를 분리하고, Grafana dashboard만 공통 UI에서 datasource UID와 cluster label로 구분한다.

주요 코드

- `cluster-monitor-exporter`

- [gpu-user-exporter](#)
- [Prometheus/Alertmanager 설정](#)
- [Grafana dashboards](#)
- [monitoring Ansible](#)
- [NFS forensics](#)
- [Kerberos/NFS keytab checker](#)

Kerberos credential과 FQDN mount 원칙은 [Kerberos/NFS 설계](#), KVNO·timer·mount 장애 절차는 [Kerberos/NFS 운영](#)을 함께 참고한다.

Monitoring 설계

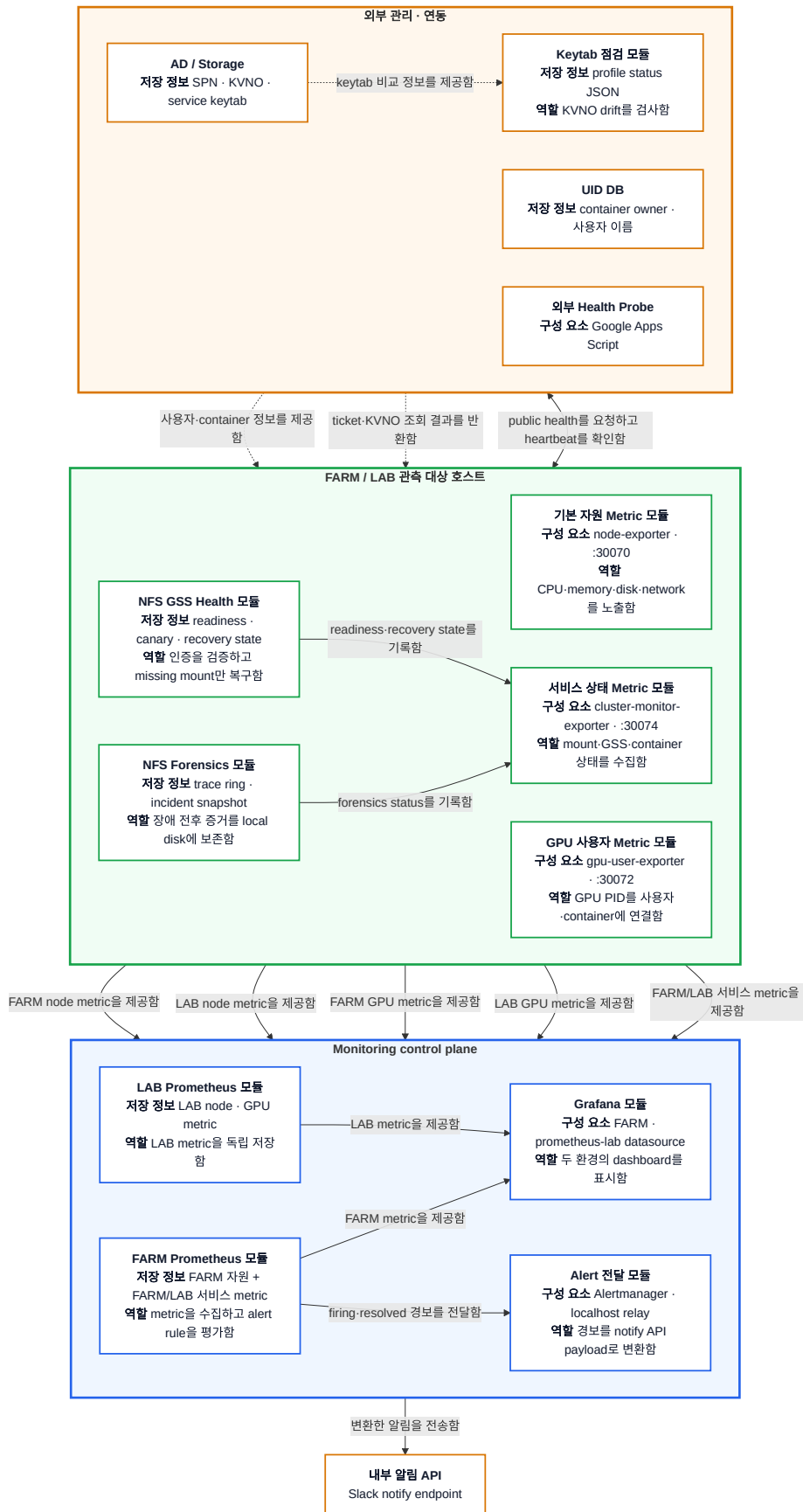
이 문서는 host의 원시 상태가 metric, alert와 dashboard가 되기까지의 흐름과 자동 복구의 안전 경계를 설명한다.

1. 설계 목표

- FARM/LAB host, GPU, container와 storage 상태를 동일한 metric 체계로 관측한다.
- HTTP process가 살아 있는지와 실제 collection이 갱신되는지를 구분한다.
- NFS **hard** mount와 D-state 상황에서도 monitoring loop 전체가 멈추지 않게 한다.
- alert에는 원인 진단에 필요한 stage와 label을 포함하되 cardinality를 제한한다.
- 관측, 증거 수집과 상태 변경을 분리하여 monitoring이 장애 반경을 넓히지 않는다.
- FARM/LAB Prometheus failure domain은 분리하고 공통 Grafana에서 함께 조회한다.

2. 전체 데이터와 알림 흐름

다이어그램의 **굵은 바깥 제목은 배치 영역**이고, 각 카드의 **굵은 첫 줄은 모듈 이름**이다. 카드 안의 **저장 정보 / 구성 요소**에는 명사만 적고, **역할**에는 주체가 무엇을 하는지 동사로 적었다. 화살표에는 출발 모듈이 요청하거나 제공하는 동작을 표시했으며, 점선은 외부 값을 읽기 전용으로 조회하는 흐름이다.



구성요소별 책임

배치 영역	모듈	내부 구성 요소·정보	역할
FARM/LAB host	기본 자원 Metric	<code>node-exporter</code> , <code>:30070</code>	CPU, memory, filesystem, disk와 network metric 노출
FARM/LAB host	GPU 사용자 Metric	<code>gpu-user-exporter</code> , <code>nvidia-smi</code> , Docker, UID DB cache	GPU process를 실제 사용자·container·GPU에 연결
FARM/LAB host	서비스 상태 Metric	<code>cluster-monitor-exporter</code> , <code>:30074</code> , public <code>:N89</code>	mount, GSS, GPU, Docker, container, 연결성과 D-state 수집
FARM/LAB host	NFS GSS Health	readiness, canary, guarded recovery worker와 state	인증 stage를 검증하고 안전 조건을 통과한 missing mount만 복구
FARM/LAB host	NFS Forensics	packet/ftrace ring, watcher, snapshot	user share를 읽거나 복구하지 않고 장애 전후 증거 보존
관리 host	Keytab 점검	profile checker, user systemd timer, status JSON	AD KVNO, storage keytab과 service ticket drift를 읽기 전용 검사
FARM control plane	FARM Prometheus/Alert	Prometheus, rule, Alertmanager, relay	FARM 자원과 FARM/LAB 서비스 metric을 수집하고 중앙 경고 전달
LAB control plane	LAB Prometheus	Prometheus, local PV, <code>:30073</code>	LAB node/GPU metric을 별도 저장; Grafana와 Alertmanager는 실행하지 않음
FARM control plane	Grafana	FARM/LAB datasource와 dashboard	두 Prometheus의 시계열을 공통 UI에서 조회

데이터 plane과 control plane

종류	예	설계 원칙
관측 data plane	<code>/proc</code> , <code>nvidia-smi</code> , Docker inspect, mountinfo, state JSON	bounded timeout, read-only 우선
metric plane	exporter <code>/metrics</code> → Prometheus	scrape 실패와 collection stale을 별도 표시
alert plane	Prometheus rule → Alertmanager → relay	secret은 Kubernetes Secret, alert label로 credential 전달 금지
visualization plane	FARM/LAB datasource → Grafana	datasource UID와 cluster label로 환경 구분
제한된 control plane	container start, SSH start, NVML repair, missing mount worker	명시적 enable, 사전조건, retry/inhibit 필요

3. Host metric exporter 구성

3.1 node-exporter: 일반 host 자원

`node-exporter` 는 CPU, memory, filesystem, disk I/O와 network 같은 일반 host 자원을 `:30070/metrics` 로 노출한다. FARM과 LAB Prometheus가 각 환경의 node metric을 저장하며, 서비스별 진단은 `cluster-monitor-exporter` 가 담당한다. 따라서 CPU 사용량이 보인다는 사실만으로 NFS readiness나 container SSH가 정상이라고 판단하지 않는다.

`node-exporter` 는 host systemd 서비스로 배포하는 두 custom exporter와 달리 `kube-prometheus-stack` 의 DaemonSet/service 설정으로 배포된다. 포트와 scrape 설정은 FARM/LAB Prometheus values의 `prometheus-node-exporter` 에서 확인한다.

3.2 cluster-monitor-exporter: 서비스 상태

Background collection과 freshness

exporter는 HTTP 요청 때마다 무거운 host 명령을 실행하지 않고 background loop의 마지막 결과를 text format으로 제공한다. `/healthz` 는 process 응답뿐 아니라 마지막 성공 collection이 `metrics_stale_after` 이내인지 확인한다.

이 구분이 필요한 이유는 HTTP server는 살아 있지만 child command나 collection goroutine이 멈춘 상태를 `up=1` 만으로는 찾을 수 없기 때문이다. 대표 지표는 `cluster_monitor_exporter_last_collection_timestamp_seconds` 와 `cluster_monitor_exporter_metrics_stale_after_seconds` 다.

수집/렌더링 구현은 `main.go`에서 확인한다.

관측 범위

- required mount의 source/target/filesystem/security option과 응답 시간
- storage host 및 peer 연결성, mount failure diagnosis
- 모든 thread를 포함한 host D-state process와 command별 count
- kernel hung-task 증가, NFS lease와 forensic 상태
- host `nvidia-smi` , Docker daemon
- 대상 image container의 running/SSH/GPU readiness
- NVML driver/library mismatch와 제한된 repair 결과
- 외부 connectivity heartbeat와 public inbound health
- NFS GSS readiness/canary/recovery state와 `rpc-gssd` journal

Container 실행 상태와 GPU alert gating

container 내부 GPU readiness는 container가 실행 중일 때만 의미가 있다. collector는 정지한 대상 container에 대해 `cluster_monitor_container_running=0` 과 `cluster_monitor_container_gpu_up=0` 을 함께 노출하지만, 이때 `gpu_up=0` 은 GPU 장애가 아니라 검사를 실행할 수 없다는 뜻이다.

따라서 GPU alert는 다음처럼 running metric을 선행조건으로 결합한다. `instance` , `server` , `container` , `image` label로 vector를 매칭하여 다른 host의 동명 container가 결합되지 않게 한다.

```
cluster_monitor_container_gpu_up{job="cluster-monitor-exporter"} == 0
and on (instance, server, container, image)
cluster_monitor_container_running{job="cluster-monitor-exporter"} == 1
```

Container 상태	running	gpu_up	발생해야 하는 alert
정지	0	0	container stopped alert만 발생
실행 중, GPU 정상	1	1	없음
실행 중, GPU 실패가 5분 지속	1	0	container GPU alert 발생

이 gating은 하나의 원인인 container 정지가 stopped와 GPU failure로 중복 통지되는 것을 막고, container가 실제로 실행 중인데 `nvidia-smi` 가 실패하는 경우만 GPU 경보로 분류한다.

D-state와 명령 timeout

NFS `hard` mount에서 child process가 D-state에 들어가면 `SIGKILL` 에도 즉시 끝나지 않고 `cmd.Wait()` 가 무기한 block될 수 있다. exporter는 외부 명령마다 timeout을 두고 collection loop가 한 NFS probe 때문에 완전히 멈추지 않게 한다.

D-state는 발생 즉시 total과 command별 metric으로 기록하되 alert는 30분 연속 지속을 기다린다. `sshd`, `find`, `du`, `chown`, `chmod`, `codex`, `claude`, `python`, `python3` 은 0인 시계열도 유지하고, 다른 command는 실제 관측될 때만 동적으로 노출한다. hung-task alert는 과거 누적 line이 아니라 최근 5분 증가분을 사용한다.

제한된 container 복구

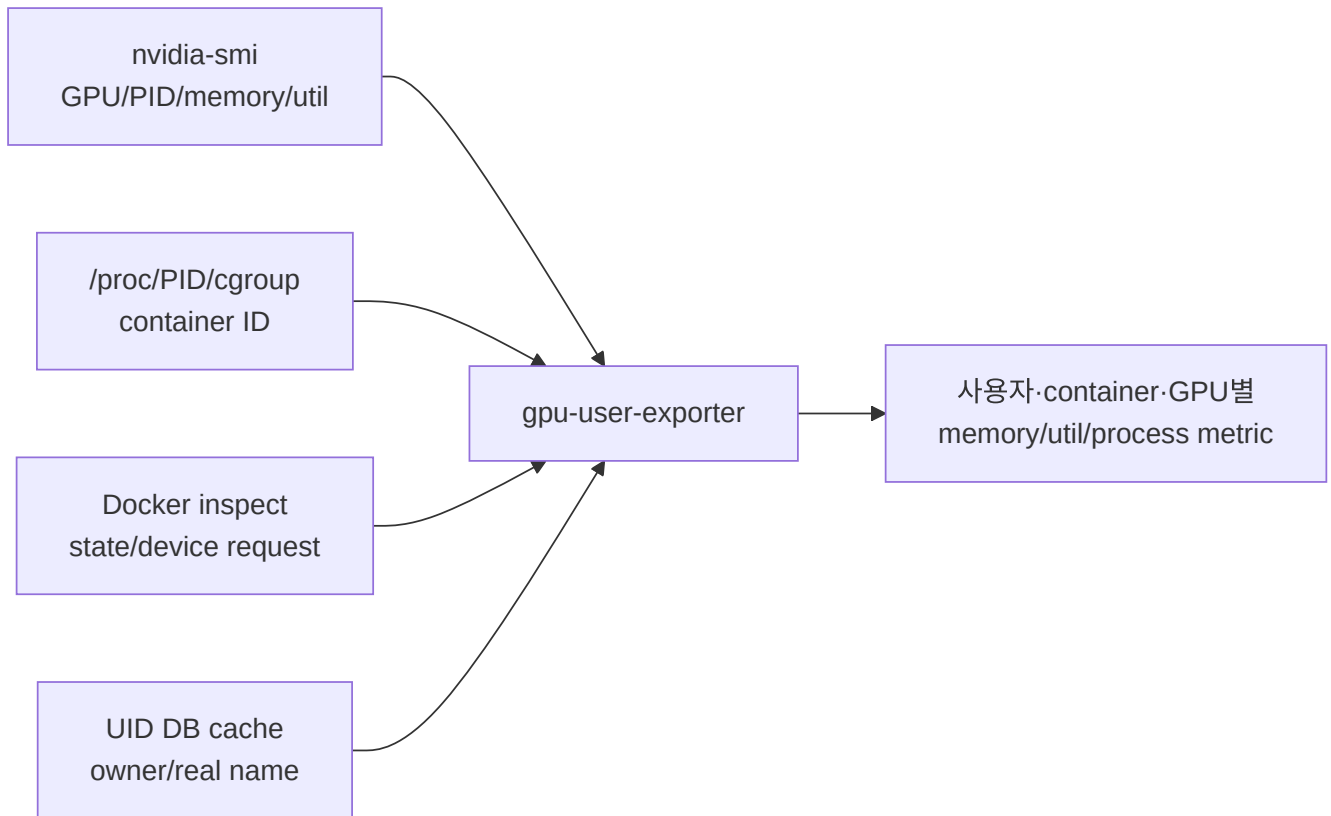
설정으로 허용된 경우에만 다음을 수행한다.

- stopped target container 시작
- container 내부 SSH service 시작
- host driver version 검증 뒤 `libnvidia-ml.so.1` symlink repair

임의 image/container, user data, host mount를 변경하지 않는다. 복구 시도와 결과는 별도 metric으로 남겨 “정상”과 “복구되어 정상”을 구분한다.

4. gpu-user-exporter

`nvidia-smi` 는 PID와 GPU 사용량만 제공하므로 실제 사용자와 container를 알기 위해 여러 정보를 join한다.



주요 지표:

- `docker_gpu_user_memory_used_bytes`
- `docker_gpu_user_sm_utilization_percent`
- `docker_gpu_user_process_count`
- device 전체 utilization/memory
- ignored process count
- DB cache entry/refresh success
- exporter scrape success/duration

DB에 없는 process를 임의의 사용자와 연결하지 않고 ignored count로 분리한다. 매 scrape마다 DB를 조회하지 않도록 활성 container owner를 cache하고 refresh 성공과 cache 크기를 metric으로 노출한다. 구현은 [gpu-user-exporter main.go](#)에 있다.

5. NFS GSS readiness, canary와 recovery

Readiness

부팅 시 NFS mount가 Kerberos host identity보다 먼저 실행되는 race를 막기 위해 다음 stage를 순서대로 확인한다.

```

host keytab -> machine kinit -> NFS service kvno
-> rpc_pipefs -> rpc-gssd -> (환경별 canary)
  
```


각 stage와 diagnosis는 state file에 기록되고 exporter가 metric으로 변환한다. 구현은 [readiness script](#)와 [NFS GSS collector](#)에 있다.

Canary

canary는 실제 user share가 아니라 전용 read-only export에서 service path를 확인한다. LAB은 canary를 사용하며 FARM은 전용 export가 준비되기 전까지 canary gate만 비활성화하고 나머지 readiness를 유지한다.

Guarded missing-mount recovery

exporter는 recovery state를 관측하고 loopback API로 worker를 queue할 수 있지만 실제 `mount` 는 별도 systemd service가 수행한다. worker gate는 다음과 같다.

1. 기대 source/filesystem/security로 이미 mount되어 있으면 즉시 healthy로 종료한다.
2. mount가 없을 때만 fstab 기대 entry를 확인한다.
3. NFS/RPC D-state가 있으면 새 mount 시도를 보류한다.
4. DNS와 storage RPC를 확인한다.
5. Kerberos readiness를 확인한다.
6. 환경에서 요구하는 canary를 확인한다.
7. retry/backoff/inhibit가 허용할 때만 `mount <target>` 을 실행한다.

healthy mount를 unmount/remount하지 않으며, mount timeout이나 관련 D-state가 생기면 현재 boot에서 반복 시도를 inhibit할 수 있다. 구현은 [mount recovery script](#)에 있다.

6. NFS forensics

NFS 장애는 재시작 후 증거가 사라질 수 있으므로 recovery와 독립된 passive forensics를 둔다.

- TCP/2049 packet의 크기 제한 ring
- 저용량 ftrace instance
- 5초 간격 kernel/NFS/D-state 상태 관측
- incident threshold에서 pre-incident ring을 local snapshot으로 보존
- PID와 process start time을 함께 사용해 PID 재사용으로 duration이 이어지지 않게 함

forensics는 mount/unmount, user share 읽기, NFS/GSS restart, recovery, reboot를 하지 않는다. exporter는 `/run/decs-nfs-forensics/status.json` 만 읽는다. `sec=krb5` 는 payload privacy를 제공하지 않으므로 capture와 incident directory는 root-only다.

구현은 [trace buffer](#), [watcher](#), [snapshot](#), [metric adapter](#)에서 확인한다.

7. Keytab health check

AD/NAS service keytab은 exporter의 자동 복구 대상이 아니다. 별도 user systemd timer가 profile별 checker를 실행한다.

```
AD KVN0 + required SPN
vs storage keytab KVN0/principal
+ 실제 service ticket 복호화
+ GSS service process/forbidden option
-> profile status JSON
```

exit code는 정상 0, 점검 오류 1, drift 2 다. keytab이나 credential 원문은 status에 기록하지 않는다. 코드는 [check-nfs-keytab.sh](#)에 있다.

8. Prometheus, Alertmanager와 Grafana

Cluster 분리

- FARM: kube-prometheus-stack, Prometheus/Grafana local PV, 공통 Grafana 제공
- LAB: LAB8 control plane의 독립 Prometheus, NodePort 30073
- Grafana: LAB datasource UID prometheus-lab 으로 LAB dashboard 조회

retention, storage, target, Alertmanager routing과 control-plane failure domain이 다르므로 Prometheus release와 values를 분리한다. dashboard는 운영자가 한 UI에서 비교하므로 공통 디렉터리에 둔다.

Alert delivery

Prometheus rule은 Alertmanager로 전달되고, Alertmanager는 localhost relay sidecar에 native payload를 보낸다. relay가 내부 notify API contract로 변환하여 /api/internal/slack/notify 로 전달한다. Grafana password와 Slack webhook은 values가 아니라 Kubernetes Secret으로 관리한다.

FARM과 LAB의 ClusterMonitor.* 경보는 cluster별 receiver로 나누지 않고 하나의 receiver를 사용한다. cluster label은 grouping과 진단 표시에만 사용하며 Slack channel 선택에는 사용하지 않는다. relay가 모든 경보에 cluster-monitor-slack-webhook-farm Secret을 적용하므로 LAB 전용 receiver와 webhook Secret은 필요하지 않다. relay는 alert fingerprint별 마지막 firing/resolved 상태를 persistent state에 기록하여 동일 상태의 반복 전송도 제거한다.

canonical FARM rule과 routing은 prometheus-farm-values.yaml, relay 구현은 Alertmanager relay ConfigMap에서 확인한다.

9. 설정과 코드 위치

관심사	기준 파일
exporter 공통 기본값, FQDN mount/SPN, GSS/forensics option	group_vars/exporters.yaml
exporter build/install/systemd config	deploy_exporters.yaml
GSS readiness/canary/recovery 설치	deploy_nfs_gss_health.yaml

관심사	기준 파일
forensics 설치	deploy_nfs_forensics.yml
FARM Prometheus/Alertmanager rules	prometheus-farm-values.yaml
LAB Prometheus values	prometheus-lab-values.yaml
GPU/NIC/storage dashboards	grafana/dashboards

10. 설계 변경 시 확인할 항목

- 새 metric의 이름, type, help, label cardinality와 실패 시 값
- command timeout과 NFS D-state에서 collection loop가 유지되는지
- alert `for` 시간이 일시적 부팅/배포 지연보다 충분히 긴지
- recovery의 enable flag, precondition, retry/backoff, inhibit와 audit metric
- user data를 읽지 않는 canary/probe 경로
- FARM/LAB datasource, job, label과 dashboard query의 일치
- credential, webhook, DB password가 Git/metric/label/log에 노출되지 않는지

Monitoring 운영

이 문서는 monitoring 구성 요소를 배포하고 endpoint, metric, alert와 incident를 점검하는 방법을 설명한다.

운영 원칙

- `up=1` 과 실제 collection freshness를 구분한다. HTTP process가 응답해도 마지막 성공 collection이 오래됐으면 정상으로 판단하지 않는다.
- metric 수집, 증거 보존과 상태 변경을 분리한다. exporter는 mount하지 않고, forensics는 service restart나 reboot를 수행하지 않는다.
- NFS caller가 D-state이면 `find`, `du`, `stat`, canary나 recovery를 반복 실행하지 않고 local state와 이미 수집된 snapshot을 먼저 보존한다.
- 자동 복구 결과는 원래 정상 상태와 구분해 metric과 state에 남긴다. inhibit 또는 backoff가 설정된 worker를 endpoint 호출로 우회하지 않는다.
- 배포는 `monitoring/ansible_playbook/` 을 기준으로 하고 target host의 개별 install script를 운영 entry point로 사용하지 않는다.
- DB password, Grafana password, webhook과 Kerberos credential은 metric, alert label, Git values와 일반 journal에 기록하지 않는다.

1. 배포 순서

새 host 또는 NFS GSS monitoring을 처음 적용할 때는 다음 순서를 권장한다.

1. inventory와 host FQDN/SPN/mount source 확인
2. NFS GSS readiness/canary/recovery worker 설치
3. NFS forensics 설치
4. exporter 배포
5. Prometheus target/rule 반영
6. Grafana datasource/dashboard 확인
7. alert delivery test

운영 배포 entry point는 exporter 내부 개별 shell script가 아니라 `monitoring/ansible_playbook/` 이다.

2. Exporter 배포

```
cd /home/jy/server_manage/monitoring
```

두 exporter를 한 host에 배포:

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_exporters.yml \  
-e exporter_hosts=farm8
```

FARM 전체 또는 FARM/LAB 전체:

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_exporters.yml \  
-e exporter_hosts=FARM
```

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_exporters.yml
```

한 exporter만 전체 재배포:

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_cluster_monitor_exporter_all.yml
```

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_gpu_user_exporter_all.yml
```

target에는 다음 systemd service와 endpoint가 생긴다.

```
cluster-monitor-exporter.service -> :30074/metrics, :30074/healthz, :N89/healthz
gpu-user-exporter.service        -> :30072/metrics, :30072/-/healthy
```

배포 기준과 요구 조건은 [Ansible README](#), 실제 task는 [deploy_exporters.yml](#)에서 확인한다.

3. NFS GSS와 forensics 배포

NFS GSS readiness/canary/recovery worker:

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \
ansible-playbook ansible_playbook/deploy_nfs_gss_health.yml \
-e nfs_gss_health_hosts=lab8
```

NFS forensics:

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \
ansible-playbook ansible_playbook/deploy_nfs_forensics.yml
```

중요한 systemd unit과 state:

```
decs-kerberos-nfs-ready.service
decs-nfs-gss-canary-probe.service/.timer
decs-nfs-gss-mount-recovery.service/.timer
/run/decs-nfs-gss/ready.state
/var/lib/decs-nfs-gss/canary.state
/var/lib/decs-nfs-gss/recovery.state

decs-nfs-forensics-buffer.service
decs-nfs-forensics-watch.service/.timer
decs-nfs-forensics-snapshot.service
/run/decs-nfs-forensics/status.json
/var/lib/decs-nfs-forensics/
```

FARM은 전용 read-only canary export가 준비될 때까지 canary만 비활성화하며 keytab/kinit/kvno/rpc-gssd readiness와 guarded recovery는 유지한다. LAB canary는 `sec=krb5` 전용 read-only export를 사용한다.

4. Keytab checker 배포

FARM 읽기 전용 checker:

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_kerberos_nfs_keytab_check.yml
```

LAB을 상시 점검 대상으로 전환한 뒤에만 두 profile을 활성화한다.

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_kerberos_nfs_keytab_check.yml \  
-e '{"keytab_check_profiles":["farm","lab"]}'
```

```
systemctl --user list-timers 'check-nfs-keytab@*.timer'  
systemctl --user status check-nfs-keytab@farm.service --no-pager
```

checker는 drift를 고치지 않는다. KVNO history, repair와 rotation은 [Kerberos/NFS 운영](#)을 따른다.

5. Prometheus/Alertmanager 배포

먼저 server-side render와 cluster precondition만 검증한다.

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_prometheus.yml
```

변경 적용:

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_prometheus.yml \  
-e prometheus_dry_run=false
```

한 환경만 배포하려면 `prometheus_farm_enabled=false` 또는 `prometheus_lab_enabled=false` 를 사용한다. playbook은 kubeconfig 대상 cluster, node Ready/DiskPressure, 필수 Secret 이름을 확인한 후 Helm을 실행한다.

FARM에서 필요한 Secret:

- `monitoring-grafana-admin`: `admin-user`, `admin-password`
- `cluster-monitor-slack-webhook-farm`: `url`

값은 Git values에 넣지 않는다. Alertmanager는 Slack webhook으로 직접 보내지 않고 localhost relay를 거쳐 internal notify API로 보낸다.

현재 control plane의 역할은 다음처럼 구분한다.

- FARM Prometheus는 FARM 자원 metric과 FARM/LAB의 `cluster-monitor-exporter` 를 수집하고 중앙 alert rule을 평가한다.

- FARM Alertmanager와 relay는 FARM/LAB 서비스 경보를 내부 notify API로 보낸다.
- LAB Prometheus는 LAB node/GPU metric을 독립 저장한다. LAB release의 Grafana와 Alertmanager는 비 활성화되어 있다.
- FARM Grafana는 기본 FARM datasource와 `prometheus-lab` datasource를 함께 사용한다.

6. 배포 후 endpoint 확인

target host에서:

```
systemctl status cluster-monitor-exporter gpu-user-exporter --no-pager
journalctl -u cluster-monitor-exporter -n 100 --no-pager
journalctl -u gpu-user-exporter -n 100 --no-pager

curl -fsS http://127.0.0.1:30074/healthz
curl -fsS http://127.0.0.1:30074/metrics | head
curl -fsS http://127.0.0.1:30072/-/healthy
curl -fsS http://127.0.0.1:30072/metrics | head
curl -fsS http://127.0.0.1:30070/metrics | head
```

:30070의 `node-exporter`는 `kube-prometheus-stack`에서 배포하므로 앞의 두 custom exporter처럼 host systemd unit을 확인하지 않는다. endpoint가 없으면 그 cluster의 DaemonSet, Service와 node scheduling 상태를 확인한다.

public health port는 서버 번호 `N`에 대해 `9000 + N*100 - 11`이다. 예를 들어 FARM8은 `9789/healthz`다. 이 endpoint는 exporter process/collection freshness와 외부 NAT 도달 경로를 확인하는 용도이며, 모든 dependency가 준비됐음을 보장하지는 않는다.

NFS GSS host-only API:

```
curl -fsS http://127.0.0.1:30074/nfs-gss/ready
curl -fsS http://127.0.0.1:30074/nfs-gss/health
curl -fsS -X POST http://127.0.0.1:30074/nfs-gss/probe
curl -fsS -X POST http://127.0.0.1:30074/nfs-gss/recover
```

`probe`와 `recover`는 loopback 요청만 허용하며 worker를 queue한 뒤 즉시 응답한다. `recover`를 반복 호출하기 전에 recovery state와 inhibit reason을 확인한다.

7. 무엇을 어디에서 보는가

관측 대상	대표 metric/alert	구현/설정 링크
CPU, memory, filesystem, disk, network	<code>node_cpu_*</code> , <code>node_memory_*</code> , <code>node_filesystem_*</code> , <code>node_disk_*</code> , <code>node_network_*</code>	FARM values, LAB values
exporter process와 freshness	<code>up</code> , <code>cluster_monitor_exporter_last_collection_timestamp_seconds</code> , <code>ClusterMonitorExporter*</code>	collector, rules
required mount와 latency	<code>cluster_monitor_host_mount_up</code> , <code>..._responsive</code> , <code>..._probe_seconds</code>	mount collector, storage dashboards
Kerberos NFS readiness	<code>cluster_monitor_nfs_gss_ready</code> , <code>ClusterMonitorNFSGSSReadinessFailed</code>	readiness, GSS collector
canary/recovery	<code>cluster_monitor_nfs_gss_canary_*</code> , <code>..._recovery_*</code>	canary, recovery
D-state, lease, hung task	<code>cluster_monitor_host_dstate_*</code> , <code>cluster_monitor_nfs_*</code> , <code>ClusterMonitorNFS*</code>	forensics adapter, forensics scripts
host GPU/Docker/container	<code>cluster_monitor_host_gpu_up</code> , <code>...docker_daemon_up</code> , <code>...container_*</code>	cluster collector
사용자별 GPU	<code>docker_gpu_user_memory_used_bytes</code> , <code>...sm_utilization_percent</code> , <code>...process_count</code>	gpu-user-exporter, GPU dashboard
외부 연결	<code>cluster_monitor_external_connectivity_*</code> , public health	exporter, Apps Script
AD/NAS keytab	checker JSON/exit code	keytab checker

canonical alert rule은 [prometheus-farm-values.yaml](#)이다. exporter 내부 [prometheus/cluster_monitor_alerts.yml](#) 은 metric 이름을 보여 주는 reference이며 실제 FARM release와 값이 다를 수 있다.

8. Alert별 진단 순서

Exporter down 또는 stale

1. Prometheus target에서 connection error와 scrape timestamp를 구분한다.
2. target systemd status/journal을 확인한다.
3. `:30074/healthz` 와 `/metrics` 를 각각 확인한다.
4. process는 살아 있지만 stale이면 마지막 실행 command와 D-state를 확인한다.
5. 변경된 env/config와 binary 배포 시간을 확인한 뒤 exporter만 재배포한다.

Required mount down/slow/unresponsive

1. `findmnt` 로 mount 존재, FQDN source, filesystem과 `sec` 를 확인한다.
2. mount probe와 storage ping/peer diagnosis label을 확인한다.
3. GSS readiness에서 처음 실패한 stage를 확인한다.
4. D-state, lease expiry, hung-task와 local forensic snapshot을 확인한다.
5. mount가 없을 때만 recovery state를 확인한다.
6. mount가 이미 있거나 D-state caller가 있으면 강제 remount를 반복하지 않는다.

Kerberos source는 IP가 아니라 FQDN이어야 한다. 자세한 기준은 [Kerberos/NFS 운영의 mount 절](#)을 따른다.

GSS readiness/canary/recovery

```
systemctl status decs-kerberos-nfs-ready.service --no-pager
systemctl status decs-nfs-gss-canary-probe.timer --no-pager
systemctl status decs-nfs-gss-mount-recovery.timer --no-pager
journalctl -u decs-kerberos-nfs-ready.service -n 100 --no-pager
cat /run/decs-nfs-gss/ready.state
cat /var/lib/decs-nfs-gss/canary.state
cat /var/lib/decs-nfs-gss/recovery.state
```

- keytab/kinit stage: host machine credential 확인
- kvno stage: DNS/FQDN/SPN/KDC 확인
- rpc-gssd stage: service와 잘못된 `-n` override 확인
- canary stage: 전용 export와 user share를 혼동하지 않았는지 확인
- inhibited recovery: D-state 또는 mount timeout 증거를 보존하고 원인을 처리

GPU 사용자 mapping 이상

1. `nvidia-smi` 와 exporter scrape success를 확인한다.
2. ignored process count가 증가했는지 확인한다.
3. PID의 cgroup container ID와 Docker inspect를 비교한다.
4. DB cache refresh success와 cache age/entry를 확인한다.
5. DB record와 실제 running container가 어긋났다면 user-lifecycle에서 관리하는 절차로 고친다.

Container SSH/GPU alert

1. container가 running인지 먼저 확인한다.
2. exporter가 start/SSH/NVML recovery를 시도했는지 metric과 journal을 확인한다.
3. container image가 configured regex 대상인지 확인한다.
4. NVML mismatch가 아니라 application/GPU allocation 문제면 자동 symlink repair를 반복하지 않는다.

9. Grafana 점검

- LAB dashboard datasource UID가 `prometheus-lab` 인지 확인한다.
- dashboard query의 `cluster`, `server_id`, `job` label이 scrape config와 일치하는지 확인한다.
- storage dashboard에서 mount probe, responsive, D-state, blocked process, hung-task를 같은 시간축으로 비교한다.
- GPU dashboard에서 DB-known 사용자 metric과 device 전체 metric을 함께 본다.
- 새 server 추가 시 dashboard에 hard-coded server/GPU panel 누락이 없는지 확인한다.

dashboard 원본:

- [FARM storage latency](#)
- [LAB storage latency](#)
- [GPU usage](#)
- [Network traffic](#)

10. 테스트

```
cd /home/jy/server_manage/monitoring

(cd prometheus/exporters/cluster-monitor-exporter && go test ./...)
(cd prometheus/exporters/gpu-user-exporter && go test ./...)
bash nfs-forensics/tests/test_watch.sh
bash prometheus/exporters/cluster-monitor-exporter/tests/test_nfs_gss_ready.sh
bash prometheus/exporters/cluster-monitor-exporter/tests/test_nfs_gss_mount_recovery.sh
python3 prometheus/config/tests/test_slack_notify_relay.py
```

변경 위험에 맞게 최소한 다음을 함께 검증한다.

- metric 이름/type/help와 0/failure path
- recovery가 healthy mount를 건드리지 않는지
- D-state에서 mount attempt가 차단되지만 existing mount는 healthy로 처리되는지
- Alertmanager relay가 secret을 출력하지 않고 internal API payload로 변환하는지
- FARM/LAB dashboard datasource가 바뀌지 않았는지

11. 새 서버 추가 체크리스트

1. `ansible_playbook/inventory.ini` 에 `farmN` / `labN` 형식으로 추가
2. `group_vars/exporters.yml` 의 mount source, SPN과 환경 기본값 확인
3. host share target와 public `N89` port 계산 확인

4. GSS readiness/forensics/exporter 배포
5. Prometheus `:30070`, `:30072`, `:30074` scrape target 추가
6. 방화벽/NAT public health 확인
7. Grafana dashboard server/GPU panel 확인
8. alert rule label과 Alertmanager route 확인
9. endpoint와 실제 alert delivery 검증

12. 운영 문서에 계속 추가할 내용

설계와 운영을 분리한 뒤 다음 정보를 운영 문서에 누적하면 좋다.

- 서비스별 owner, endpoint, systemd/Kubernetes resource와 dependency
- metric별 정상 범위, alert `for` 와 runbook link
- 최근 배포 version/commit과 마지막 검증 시각
- FARM/LAB retention, PV 용량과 capacity threshold
- alert silence 승인/만료 기준과 false-positive 기록
- 자동 복구별 시도 횟수, inhibit 해제와 rollback 절차
- incident snapshot 보존 기간, 접근 권한과 개인정보 취급
- 새 server/metric/dashboard 추가 checklist

설계 문서에는 구성 요소의 관계와 안전 경계를 유지하고, host별 현재 값과 일회성 incident 결과는 canonical config/runbook 또는 별도 evidence에 기록한다.

remote-operations

원본: md/system/remote-operations/index.md, md/system/remote-operations/design.md, md/system/remote-operations/operations.md, md/system/remote-operations/config.md

역할: FARM/LAB 서버를 깨우고, 부팅 시 한 번 상태를 확인한 뒤 기존 컨테이너를 시작한다.

이 문서는 remote-operations 문서의 시작점이다. **설계**는 부팅 orchestration의 책임 범위와 각 스크립트의 역할을 설명하고, **운영**은 실제 서버 관리자가 배포·점검·장애 대응에 사용하는 명령을 설명하며, **설정**은 config 파일을 어떻게 준비하는지 설명한다.

문서 구성

문서	핵심 내용
현재 페이지	책임 범위와 문서 안내
설계	디렉터리 지도, 부팅 상태 머신, 핵심 기능, 설계상 지켜야 할 경계
운영	사용법, dry-run과 검증, 실패 처리 원칙, 새 서버 추가
설정	config 파일 준비와 설정 모델

처음 보는 사람은 **설계 문서**에서 전체 구조를 먼저 확인하고, 실제 서버를 켜거나 점검해야 할 때 **운영 문서**로, config 파일을 새로 준비해야 할 때 **설정 문서**로 이동한다.

전체 서버 동시 부팅 전환과 Slack 알림 백엔드 큐 연동 검증은 완료되어 이 문서들에 반영됨. `.sh` → `.py` 전환 작업은 아직 진행 전이며, 반영되면 이 페이지와 설계/운영 문서 내용도 함께 갱신해야 한다.

remote-operations 설계

개요 · 운영 · 설정

역할: FARM/LAB 서버를 깨우고, 부팅 시 한 번 상태를 확인한 뒤 기존 컨테이너를 시작한다.

1. 책임 범위

`remote-operations` 는 management host의 `remote-boot.service` 가 실행하는 부팅 orchestration이다. Wake-on-LAN, 전체 대상 host health gate, 1회 host health check와 container post-check를 소유한다.

주기적인 mount/GPU/container 관측은 `monitoring` 에 있다. 이 경계를 나눈 이유는 부팅 시 복구와 상시 self-healing이 서로 다른 retry 정책과 장애 의미를 가지기 때문이다. 이 디렉터리에서 관리하는 systemd unit은 `remote-boot.service` 하나다.

2. 디렉터리 지도

경로	핵심 기능
<code>script/run_remote_boot.sh</code>	전체 부팅 흐름의 entry point
<code>script/wake_targets.sh</code>	target 선택과 WOL magic packet 전송
<code>script/wait_for_priority_servers.sh</code>	선택된 대상 전체의 host health gate
<code>script/check_server_boot_health.sh</code>	host SSH 도달성, mount, host GPU 점검
<code>script/restart_all_remote_containers.sh</code>	기존 container 시작과 SSH/GPU post-check
<code>script/create_test_container.sh</code>	(부팅 흐름에서 호출되지 않는 독립 도구) 임시 GPU container 생성
<code>script/delete_test_container.sh</code>	(부팅 흐름에서 호출되지 않는 독립 도구) 임시 container 정리
<code>script/common.sh</code>	config, target, Ansible, log, alert 공통 함수
<code>script/install_remote_boot_service.sh</code>	systemd unit 설치·활성화
<code>test/dry_run_remote_boot.sh</code>	WOL/health/container/full flow simulation
<code>test/integration_smoke_test.sh</code>	수동 Ansible/Docker/GPU 통합 확인
<code>test/test_slack_notification.sh</code>	Slack webhook 설정과 알림 발송 테스트
<code>config/remote_boot.example.env</code>	공개 가능한 설정 구조
<code>config/remote_boot.local.env</code>	실제 MAC·webhook 등을 담은 ignored 설정

3. 부팅 상태 머신

```
pre-delay
|
v
wake all selected targets (동시)
|
v
health gate for all selected targets -- failure --> stop + alert
|
v
start stopped target containers
|
v
per-container SSH/GPU post-check
```

과거에는 LAB1 / FARM1 에 있는 사용자 관리 DB(사용자, UID/GID, 할당 port와 사용자 container record를 관리하는 기준 데이터)를 먼저 기동하기 위해 LAB1 / FARM1 을 priority target으로 먼저 깨우고 health gate를 통과시킨 뒤 나머지 서버를 기동하는 2단계 구조였다. 이 priority/remaining 분리 로직은 제거되었고, 지금은 선택된 대상을 항상 동시에 wake + gate한다.

4. 핵심 기능

4.1 target 정규화

설정은 FARM/LAB 전체 목록과 기본 대상을 가진다. 명령행 target이 있으면 기본 대상을 덮어쓴다. all, domain group, 개별 FARM1 / LAB10 을 동일한 parser로 해석하여 중복을 제거한다.

MAC address와 broadcast IP는 WOL에만 쓰고, 상태 확인과 원격 명령은 Ansible inventory를 사용한다. 네트워크 주소와 논리 server ID를 분리하면 SSH port나 주소가 바뀌어도 workflow 이름을 유지할 수 있다.

4.2 1회 host health check

check_server_boot_health.sh 는 target에 대해 다음을 확인한다.

- Ansible/SSH 도달성
- 필수 FARM/LAB share mount
- host nvidia-smi

과거에는 이 뒤에 create_test_container.sh 로 임시 GPU container를 만들어 container 내부의 GPU/SSH까지 확인하고 delete_test_container.sh 로 정리하는 단계가 있었다. 여러 서버가 동시에 부팅되면 이 단계가 서버마다 GPU 전체와 메모리 192g를 잡고 무거운 이미지를 pull하는 부담으로 이어져 제거했다. 이제 host 레벨 확인까지만 하고,

container GPU 연동이 실제로 살아있는지는 `restart_all_remote_containers.sh` 가 기존 사용자 container를 대상으로 재확인한다 (§ 4.3). `create_test_container.sh` / `delete_test_container.sh` 자체는 삭제하지 않았으므로, 필요하면 독립적으로 실행할 수 있다.

4.3 기존 container 재시작과 post-check

선택된 서버의 stopped target container를 시작한 뒤 각 container의 SSH와 GPU를 제한 시간 동안 확인한다. post-check 실패는 container별로 기록하고 전체 stage 실패 정보에 포함한다.

container를 무조건 재생성하지 않는 이유는 사용자 홈과 DB record, 고정 포트, 실행 옵션의 권위가 `user-lifecycle` 에 있기 때문이다. 부팅 모듈은 기존 container를 시작하고 준비 여부만 확인한다.

4.4 alert suppression과 로그

실패는 구성된 경우 내부 notify API를 통해 Slack으로 보내고, 그렇지 않으면 stub log에 남긴다. 동일 실패의 반복 알림을 줄이기 위한 alert state가 별도 디렉터리에 저장된다. `reset_remote_boot_alert_state.sh` 는 이 suppression 상태만 초기화한다.

service log, host health log와 alert state를 나눈 이유는 실행 이력과 알림 deduplication 상태를 독립적으로 보존하기 위해서다.

5. 설계상 지켜야 할 경계

- systemd unit을 추가하기 전에 부팅 1회 책임인지 지속 monitor 책임인지 구분한다.
- 공통 shell helper를 수정하면 모든 stage의 dry-run과 alert redaction을 확인한다.
- 실제 secret을 example 또는 log에 쓰지 않는다.
- retry를 늘리는 것으로 storage/GSS 장애를 감추지 않는다.
- container 생성/삭제 비즈니스 규칙은 `user-lifecycle` CLI를 통해 수행한다.

remote-operations 운영

개요 · 설계 · 설정

1. 사용법

target 확인과 수동 WOL (`admin_infra_server/remote-operations` 에서 실행):

```
./script/wake_targets.sh --list-targets
./script/wake_targets.sh FARM1 LAB1
```

한 서버의 boot health:

```
./script/check_server_boot_health.sh --server-id FARM1
```

한 서버의 기존 container 시작/post-check:

```
./script/restart_all_remote_containers.sh FARM1
```

systemd 설치와 확인:

```
./script/install_remote_boot_service.sh  
sudo systemctl start remote-boot.service  
systemctl status remote-boot.service  
journalctl -u remote-boot.service -b
```

2. dry-run과 검증

```
./test/dry_run_remote_boot.sh wake FARM1 LAB1  
./test/dry_run_remote_boot.sh health FARM1  
./test/dry_run_remote_boot.sh containers FARM1  
./test/dry_run_remote_boot.sh full FARM1 LAB1
```

dry-run은 sleep, WOL, Ansible 변경과 Docker 작업 대신 계획을 기록한다. 설정 파싱, target 분할과 단계 순서를 production 영향 없이 검증하기 위한 공개 계약이다.

실제 연결을 확인할 때:

```
./test/integration_smoke_test.sh
```

Slack 알림 설정을 확인할 때:

```
./test/test_slack_notification.sh --server-id FARM1
```

`REMOTE_BOOT_SLACK_ENABLED` 와 webhook URL이 설정돼 있어야 하며, 테스트 메시지를 실제로 전송해 alert 경로가 살아있는지 확인한다.

운영 전에는 최소한 다음을 확인한다.

- MAC/broadcast IP와 Ansible host가 같은 물리 서버를 가리키는지
- gate timeout이 선택된 서버 전체가 동시에 부팅되는 시간을 감안해도 충분한지
- 실패 알림에 비밀값이 포함되지 않는지

3. 실패 처리 원칙

- 선택된 대상 중 gate를 통과하지 못한 서버가 있으면 container 재시작 단계로 넘어가지 않는 것이 기본이다.
- gate 단계에서 서버별 통과/대기 상태를 구분해 기록한다 (`wait_for_priority_servers.sh`가 통과한 서버와 재시도 중인 서버를 분리해서 로그로 남긴다).
- mount 또는 GPU 문제를 부팅 script에서 무제한 복구하지 않는다. 반복 상태는 `monitoring`으로 넘기고, Kerberos/NFS 고위험 복구는 해당 runbook을 따른다.
- 실제 사용자 container의 DB record나 Docker 옵션을 부팅 script에서 다시 만들지 않는다.

4. 새 서버 추가

1. `user-lifecycle/server_info/servers.jsonl`과 Ansible inventory에 서버를 등록한다.
2. local env의 FARM/LAB target 목록과 MAC을 추가한다.
3. 필요한 경우 필수 mount template를 정한다.
4. WOL dry-run과 개별 WOL을 확인한다.
5. 개별 boot health check를 실행한다.
6. container restart를 한 서버에 제한해 검증한다.
7. `monitoring`의 scrape target/exporter 배포를 별도로 완료한다.

remote-operations 설정

개요 · 설계 · 운영

설정 모델

설정은 `remote_boot.example.env`를 복사하여 local 파일에서 관리한다 (`admin_infra_server/remote-operations`에서 실행).

```
cp config/remote_boot.example.env config/remote_boot.local.env
```

주요 설정 그룹:

그룹	예
target	FARM/LAB 목록, 기본 target
순서/gate	pre-delay, gate timeout/poll
container	restart enable, timeout, post-check poll

그룹	예
network	Ansible inventory, broadcast IP, MAC address
health	필수 mount와 host share template
test container (독립 도구용)	<code>create_test_container.sh</code> / <code>delete_test_container.sh</code> 가 쓰는 image, UID/GID, mount, memory, runtime — 부팅 흐름에서는 더 이상 쓰이지 않음
logging/alert	log path, rotate count, notify API와 webhook

실제 MAC, password와 webhook은 `remote_boot.local.env`에만 둔다. example에는 변수 이름과 안전한 placeholder만 유지한다.

첫 설정 체크리스트

이 저장소를 처음 pull한 뒤 dry-run 검증까지 마치는 순서:

1. 개인 SSH 공개키를 각 FARM/LAB 서버의 `~/.ssh/authorized_keys`에 등록
2. `~/ansible/inventory.ini` 준비 (`ansible_user`를 자신의 계정으로)
3. `remote_boot.example.env` → `remote_boot.local.env` 복사
4. `REMOTE_BOOT_ANSIBLE_INVENTORY`를 자신의 inventory 경로로 설정
5. `REMOTE_BOOT_MAC_*` 값을 실제 MAC으로 채움
6. `REMOTE_BOOT_SLACK_ENABLED=true`와 `REMOTE_BOOT_SLACK_WEBHOOK_URL_FARM` 설정
7. 운영 문서의 dry-run 명령으로 검증

개인 Ansible inventory 준비

관리 데스크탑은 여러 관리자가 공유한다. Ansible inventory는 host 그룹당 `ansible_user`를 하나만 가질 수 있어서, `/etc/ansible/inventory.ini` (특정 관리자 계정으로 고정)를 그대로 쓰면 다른 관리자는 그 계정으로 원격 서버에 접속을 시도하다가 SSH 인증에 실패한다. 그래서 관리자마다 자신의 계정으로 접속하는 개인 inventory가 필요하다. (UID_GID_Management_System 5.2.4 내용을 보고 inventory.ini 파일을 `~/uid_gid/inventory.ini`에 위치시킨 경우 지금 설명하는 단계를 진행해야 한다. inventory.ini의 위치가 변경되기 때문이다.)

- 위치: `~/ansible/inventory.ini`. 특정 프로젝트에 속하지 않는 개인 경로다. remote-operations와 `uid_gid` 등 이 데스크탑의 여러 도구가 각자의 설정에서 이 경로를 가리키기만 하고, 어느 한쪽도 이 파일을 소유하지 않는다.
- 준비: `/etc/ansible/inventory.ini`를 복사해서 `ansible_user`만 자신의 계정으로 바꾼다.
- 원격 서버의 `~/.ssh/authorized_keys`에 자신의 공개키를 등록해야 실제 SSH 인증이 된다. inventory 파일만으로는 접속 권한이 생기지 않는다.
- (선택) `~/ansible/ansible.cfg`를 만들어두면 `-i` 없이 `ansible <host> ...` 명령이 기본으로 이 inventory를 사용한다.

```
[defaults]
inventory = /home/<본인계정>/ansible/inventory.ini
interpreter_python = auto_silent
host_key_checking = False
retry_files_enabled = False
```

Ansible 설정이 완료되면 `remote_boot.local.env` 에서 REMOTE_BOOT_ANSIBLE_INVENTORY를 다음과 같이 수정한다.

```
REMOTE_BOOT_ANSIBLE_INVENTORY="/home/<본인계정>/ansible/inventory.ini"
```

이 개인 설정은 dry-run 검증과 수동 운영에 쓰인다. 실제 자동 부팅을 담당하는 `remote-boot.service` (systemd)는 별도 관리자 계정 소유로 이미 배포되어 있으며 이 개인 설정과 무관하게 동작한다.

MAC 주소

`REMOTE_BOOT_MAC_*` 는 물리 서버 하드웨어의 실제 MAC 주소로 장비통합관리문서에서 값을 가져온다.

Slack 알림 설정

부팅 실패 알림을 실제로 받아보려면 `remote_boot.local.env` 에서 두 값을 채운다.

```
REMOTE_BOOT_SLACK_ENABLED=true
REMOTE_BOOT_SLACK_WEBHOOK_URL_FARM="https://hooks.slack.com/services/..."
```

- `REMOTE_BOOT_SLACK_WEBHOOK_URL_FARM` 도 MAC 주소와 마찬가지로 실제 운영 값이라 임의로 만들 수 없다. 기존 운영 담당자에게 확인하거나, 이미 배포된 운영 설정에서 값을 가져온다.
- LAB/FARM 구분 없이 모든 원격 부팅 알림은 이 FARM webhook 하나로 전송된다.
- `REMOTE_BOOT_SLACK_NOTIFY_API_URL` 은 webhook을 직접 호출하지 않고 백엔드 내부 notify API를 거친다.
- 설정 후 아래 명령으로 실제 Slack 채널에 메시지가 오는지 확인한다.

```
./test/test_slack_notification.sh --server-id FARM1
```

server-state

원본: md/system/server-state/index.md, md/system/server-state/design.md, md/system/server-state/operations.md

역할: FARM/LAB GPU 서버가 모두 같은 공통 설정을 유지하도록 기준을 정의하고, 운영 서버 점검과 신규 서버 구축에 같은 순서를 사용한다.

설계는 이 모듈이 확인·설정하는 범위, profile 구조, 모듈별 소유권과 현재 구현 수준을 설명하고, **운영**은 점검·구축 흐름에 실제로 사용하는 CLI 명령과 공통 설정 추가 절차를 설명한다.

문서 구성

문서	핵심 내용
현재 페이지	책임 범위와 문서 안내
설계	확인/설정 범위, profile 구조, 모듈별 소유권, 상태 표시, 디렉터리 구조, 현재 구현 수준
운영	운영 서버 점검·신규 서버 구축 흐름, 공용 서버 목록, 공통 설정 추가 방법, 테스트

처음 보는 사람은 **설계 문서**에서 이 모듈이 무엇을 소유하고 무엇을 다른 모듈에 위임하는지 먼저 확인하고, 실제로 서버를 점검하거나 새 서버를 구축해야 할 때 **운영 문서**를 확인한다.

server-state 설계

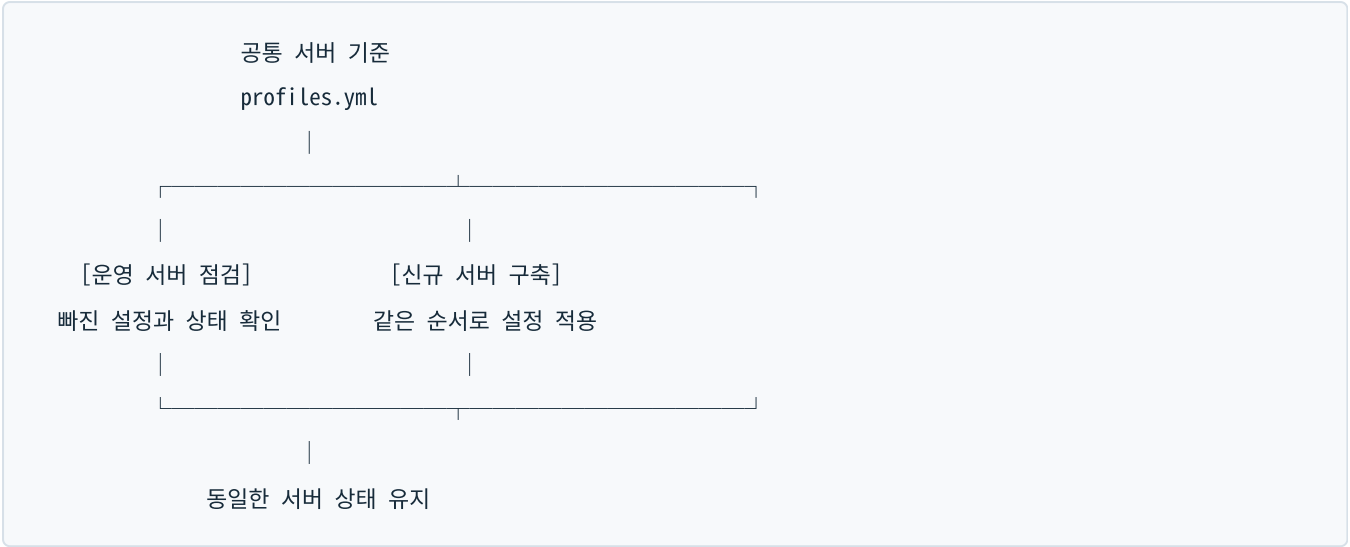
개요 · 운영

1. 이 모듈이 하는 일

server-state 는 FARM/LAB 서버가 모두 같은 공통 설정을 유지하도록 기준을 정의하는 모듈이다. 목적은 크게 두 가지다.

- 운영 서버 점검:** 각 서버에 Docker, NVIDIA driver/toolkit, Kubernetes, network tuning, Kerberos/NFS, monitoring 설정이 빠짐없이 적용되어 있는지 확인하고 차이가 있으면 복구할 방법을 제시한다.
- 신규 서버 구축:** 새 서버에 공통 설정을 같은 순서로 적용하여 운영 서버와 동일한 표준 상태로 만든다.

즉, 서버마다 설치 방법을 다시 기억해서 수동으로 설정하는 대신 하나의 공통 기준을 사용하려고 만든 코드다.



`server-state` 가 모든 기능을 직접 구현하는 것은 아니다. 공통 OS, Docker, NVIDIA와 network 설정은 직접 관리하고, Kerberos/NFS와 monitoring처럼 별도 모듈이 소유한 기능은 해당 모듈의 점검·복구 명령을 한 순서로 연결한다.

2. 디렉터리 지도

경로	핵심 기능
<code>README.md</code>	모듈 개요와 책임 범위 요약
<code>bin/server-state</code>	CLI 실행 파일
<code>script/cli.py</code>	<code>list-hosts</code> , <code>check</code> , <code>plan</code> , <code>apply</code> 출력
<code>script/inventory.py</code>	공용 JSONL inventory 로드와 host 선택
<code>script/profiles.py</code>	profile set 확장과 서버별 변수 치환
<code>script/__main__.py</code>	<code>python -m script</code> 로 CLI를 실행할 수 있게 하는 진입점
<code>config/profiles.yml</code>	공통 상태, 점검과 복구 명령의 기준
<code>ansible_playbook/bootstrap_gpu_server.yml</code>	신규/운영 서버의 공통 설정 task
<code>ansible_playbook/kerberos_nfs_client_recovery.yml</code>	Kerberos NFS 상태 점검과 제한된 복구
<code>docs/standard-gpu-server-pipeline.md</code>	신규/운영 서버 표준 단계의 개발 문서
<code>docs/module-boundaries.md</code>	모듈별 소유권 경계 정의 (§ 6 "모듈별 소유권"의 원본 문서)
<code>docs/repo-inventory-cleanup.md</code>	레포 내 일회성/legacy 파일 정리 감사 기록
<code>docs/reusable-operational-files.md</code>	재사용 가능한 운영 파일 목록
<code>requirements.txt</code>	Python 의존성 (<code>PyYAML</code>)
<code>tests/</code>	inventory와 profile 처리 unit test

3. 무엇을 확인하고 설정하는가

`new-host-bootstrap` 과 `existing-host-drift` 는 다음 항목을 같은 순서로 사용한다. 따라서 새로운 공통 설정을 추가할 때 두 흐름에 함께 넣을 수 있다.

순서	영역	운영 서버에서 확인하는 것	신규 서버에 설정하는 것
1	기본 접속 조건	inventory 등록, Ansible 접속, 비대화형 sudo, hostname	자동 설정 전 SSH, sudo, hostname, IP와 inventory를 확인
2	공통 OS	Ubuntu 계열 여부, 공통 package 설치 여부	apt repository, NFS, Kerberos와 network 도구 설치
3	Docker Engine	service 활성 상태, daemon 응답, systemd cgroup driver	Docker repository/package와 <code>daemon.json</code> 설정
4	NVIDIA driver	<code>nvidia-smi</code> 가 GPU와 driver version을 정상 출력하는지, package hold 여부	설정된 driver package 설치와 apt hold
5	NVIDIA Container Toolkit	<code>nvidia-ctk</code> , Docker NVIDIA runtime, containerd NVIDIA runtime	toolkit 설치 후 Docker/containerd runtime 설정
6	Kubernetes node	kubeadm/kubelet/kubectrl, kubelet, cluster join 파일과 node label	Kubernetes package 설치와 kubelet 활성화
7	network tuning	storage NIC 정보, RX queue 4096 이상, 영속화 service	storage NIC RX queue를 4096으로 유지하는 systemd service
8	Kerberos/NFS	realm 설정, machine keytab, service ticket, <code>rpc-gssd</code> , fstab과 mount 상태	client package/config와 GSS 준비 상태 구성
9	monitoring	두 exporter service와 metrics/health endpoint	monitoring 모듈의 exporter 배포 playbook 사용
10	사용자 container 전제조건	사용자 DB 환경, server inventory, Docker 접근	사용자 생성·삭제는 <code>user-lifecycle</code> 을 통해 처리

3.1 NVIDIA version 확인 범위

현재 점검 명령은 `nvidia-smi` 를 실행하여 GPU 이름과 실제 driver version이 출력되는지 확인한다. 따라서 driver가 GPU를 인식하지 못하거나 명령 자체가 실패하는 상태는 찾을 수 있다.

신규 설치 playbook의 기본 package는 현재 `nvidia-driver-580` 이며 설치 후 의도하지 않은 major version 변경을 막기 위해 apt hold한다. 다만 운영 서버의 driver version을 `580` 과 자동 비교하여 불일치 판정을 내리는 규칙은 아직 없다. 현재는 출력된 version을 관리자가 확인해야 한다.

3.2 network 설정 범위

현재 `network-tuning` 이 관리하는 대상은 스토리지 통신에 사용하는 NIC의 RX queue 크기다. inventory에서 서버별 storage interface를 읽고, RX queue가 4096 이상인지와 `decs-rx-queue.service` 가 활성화되어 있는지 확인한다.

IP 주소, gateway, DNS와 netplan 전체를 자동으로 설정하는 기능은 아직 없다. 신규 서버의 IP, hostname, SSH와 sudo는 bootstrap을 시작하기 전에 준비해야 하는 조건이다. 향후 모든 서버의 netplan까지 통일하려면 별도의 profile과 검증 규칙을 추가해야 한다.

4. 현재 구현 수준

이 모듈은 최종적으로 공통 기준과 실제 서버 상태를 자동 비교하고 필요한 복구까지 안전하게 실행하는 것을 목표로 한다. 현재 구현 수준은 다음과 같다.

기능	현재 상태
전체 서버가 따라야 할 공통 profile 정의	구현됨
신규/운영 서버에 같은 profile 순서 사용	구현됨
서버별 점검 명령 생성	구현됨
신규 서버 설정용 Ansible task	구현됨
<code>check</code> 가 원격 서버를 순회하고 결과 판정	아직 구현되지 않음
<code>apply --execute</code> 로 복구 자동 실행	아직 구현되지 않음. 명시적으로 거부됨
주기적인 전체 서버 drift 검사와 알림	아직 구현되지 않음
IP/DNS/netplan 전체 표준화	아직 구현되지 않음

따라서 “모든 서버가 같은 설정인지 확인하고 새 서버를 똑같이 설정한다”는 이해는 맞다. 다만 현재는 **기준, 점검 명령, 복구 playbook을 한곳에 정리한 단계**이고, 한 명령으로 전체 서버를 자동 검사·복구하는 controller까지 완성된 상태는 아니다.

5. profile 구조

`config/profiles.yml`에는 작은 단위의 profile과 이를 순서대로 묶은 profile set이 있다.

profile set	용도
<code>new-host-bootstrap</code>	신규 SSH-ready 서버의 표준 구축 순서
<code>existing-host-drift</code>	운영 서버의 공통 설정 점검·복구 순서
<code>managed-host</code>	운영 관리 서버용 기본 별칭
<code>monitoring-host</code>	monitoring 항목만 확인할 때 사용

새 공통 설정을 모든 서버에 적용하려면 작은 profile로 추가한 뒤 `new-host-bootstrap`과 `existing-host-drift` 양쪽에 넣는다. 이렇게 해야 새 서버 설치에는 들어갔지만 운영 서버 점검에서는 빠지거나, 그 반대가 되는 문제를 줄일 수 있다.

각 profile은 다음 정보를 가진다.

- 이 설정을 실제로 소유하는 모듈
- 부작용 없이 상태를 읽는 check
- 차이가 있을 때 사용할 remediation 명령
- 자동 실행할 수 없는 작업의 runbook과 safety level

6. 모듈별 소유권

영역	실제 소유자	server-state 의 역할
공통 OS, Docker, NVIDIA, Kubernetes package, network tuning	server-state	공통 기준과 bootstrap task 관리
NAS, AD, Kerberos와 NFS 정책	kerberos-nfs	점검 순서와 승인된 runbook 연결
exporter와 metrics endpoint	monitoring	배포·점검 playbook 연결
사용자와 container 생성·삭제	user-lifecycle	공용 inventory 사용과 전제조건 확인
서버 전원과 boot sequence	remote-operations	이 모듈에서 다루지 않음
사용자 container image	container-images	이 모듈에서 image 내부를 변경하지 않음

소유권을 나눈 이유는 `server-state` 에 모든 운영 코드를 복사하지 않고, 실제 담당 모듈의 rollback과 안전 절차를 그대로 사용하기 위해서다.

7. 상태 표시

상태	의미
OK	inventory나 로컬 파일처럼 즉시 확인 가능한 조건이 충족됨
MISSING	필요한 로컬 파일이나 값이 없음
DRY-RUN	실행할 원격 점검 또는 복구 명령만 표시함
MANUAL	도메인 가입, join, mount처럼 담당자 확인이 필요한 작업
UNKNOWN	아직 지원하지 않는 check 또는 remediation 형식

server-state 운영

개요 · 설계

1. 운영 서버 점검 흐름

1.1 existing-host-drift 란?

`existing-host-drift` 는 `config/profiles.yml` 에 정의된 **점검 항목 묶음의 이름**이다. 그 안에는 접속 확인(baseline-host)부터 `os-common`, `docker-engine`, `nvidia-driver`, `nvidia-container-runtime`, `kubernetes-node`, `network-tuning`, `kerberos-nfs-client`, `monitoring-exporters`, `user-container-host`까지 10개 영역이 순서대로 들어 있다. "운영 중인 FARM/LAB 서버가 이 10개 영역의 공통 기준을 다 지키고 있는지 확인하는 점검표"라고 보면 된다.

실제로 실행하는 스크립트는 `./server-state/bin/server-state` 이고, 내부 동작은 `script/cli.py` 가 맡는다. `cli.py` 가 `config/profiles.yml` 을 읽어서 `existing-host-drift` 에 묶인 10개 영역의 점검·복구 항목을 순서대로 꺼내 쓰는 구조다.

1.2 "명령어를 만들어서 보여준다"는 게 무슨 뜻인가

`profiles.yml` 에는 완성된 명령이 아니라, `{host}` 나 `{repo}` 같은 자리표시자가 들어간 **명령 틀**만 적혀 있다. 예를 들어 `os-common` 영역의 점검 항목 하나는 실제로 다음과 같이 정의되어 있다.

```
# config/profiles.yml (os-common 영역의 점검 항목 중 하나)
command: "ANSIBLE_CONFIG={repo}/monitoring/ansible_playbook/ansible.cfg \
  ansible {host} -b -m shell -a '. /etc/os-release && test \"$ID\" = ubuntu ...'"
```

`check` 명령을 실행하면 `script/cli.py` 가 이 틀의 `{host}` 를 실제 서버 이름(`farm8`)으로, `{repo}` 를 실제 저장소 경로로 바꿔 채운 뒤, 그 결과 문자열을 화면에 출력만 한다. 예시:

```
$ ./server-state/bin/server-state check --hosts farm8 --profile os-common
farm8 [os-common] ubuntu-release: DRY-RUN
  detail: ANSIBLE_CONFIG=/home/jy/server_manage/monitoring/ansible_playbook/ansible.cfg ansible farm8 -b -m
  shell -a '. /etc/os-release && test "$ID" = ubuntu && printf "%s\n" "$VERSION_ID"
```

`detail`: 뒤에 나온 문자열이 "만들어서 보여준 명령"이다. `server-state` 는 자리표시자를 채운 문자열을 Python에서 만들어 출력하기만 할 뿐, 이 명령을 실제로 실행(예: `subprocess` 호출)하지 않는다. 서버 상태를 실제로 확인하려면 관리자가 이 문자열을 복사해서 직접 실행해야 한다.

1.3 전체 점검 흐름

아래 5단계 중 실제로 서버 상태가 바뀌는 건 5번뿐이고, 그마저도 관리자가 직접 실행해야만 일어난다.

1. **점검 대상 서버 선택** — 공용 inventory(3절 "공용 inventory와 서버 목록" 참고)에서 `--hosts` 조건(`all`, `farm8` 등)에 맞는 서버를 로컬에서 골라낸다. 서버에 접속하지 않는다.
2. **점검 항목 생성** — `config/profiles.yml`의 `existing-host-drift` profile을 서버별 변수(hostname, IP 등)로 채워 로컬에서 명령 목록을 만든다. 서버에 접속하지 않는다.
3. **점검 명령을 DRY-RUN으로 출력** — 서버별로 실행하면 될 읽기 전용 Ansible 명령을 화면에 보여줄 뿐, 실행하거나 서버에 접속하지 않는다.
4. **복구 명령을 DRY-RUN으로 출력** — 3번에서 차이가 발견된 항목에 한해 복구용 Ansible 명령(`--check --diff` 포함)을 보여준다. 역시 실행하지 않는다.
5. **관리자가 검토 후 직접 실행** — 3, 4번에서 출력된 명령을 관리자가 그대로 복사해 실행해야 서버 상태가 바뀐다. `server-state` 는 이 단계를 대신 실행하지 않는다.

실제 명령은 다음과 같다. 1~3번 단계가 `check` 한 번으로 함께 실행된다.

```
cd /home/jy/server_manage

# 전체 서버의 점검 항목 확인
./server-state/bin/server-state check \
  --hosts all \
  --profile existing-host-drift

# 특정 서버(farm8)만 확인
./server-state/bin/server-state check \
  --hosts farm8 \
  --profile existing-host-drift
```

두 명령 모두 서버에 접속하거나 서버 상태를 바꾸지 않는다. 화면에 출력된 `DRY-RUN` 명령을 관리자가 직접 실행하거나, 별도 자동화가 그 명령을 실행하도록 연결해야 실제 점검(3번 단계 이후)이 이루어진다.

4번 단계, 즉 차이가 있을 때 사용할 복구 명령만 먼저 볼 수도 있다.

```
./server-state/bin/server-state plan \
  --hosts farm8 \
  --profile existing-host-drift
```

2. 신규 서버 구축 흐름

새 서버를 운영 서버와 같은 상태로 만드는 것이 `new-host-bootstrap`의 목적이다. 다만 아무것도 설치되지 않은 장비의 전원 투입부터 전부 처리하는 형태는 아니다. 다음 조건은 먼저 준비되어 있어야 한다.

- Ubuntu가 설치되어 있다.

- IP, hostname과 SSH 접속이 준비되어 있다.
- 관리 계정이 비대화형 sudo를 사용할 수 있다.
- 서버가 공용 inventory에 등록되어 있다.

이후 표준 설치 순서를 확인한다.

```
./server-state/bin/server-state plan \
  --hosts farm8 \
  --profile new-host-bootstrap
```

`ansible_playbook/bootstrap_gpu_server.yml`에는 package 설치와 설정 파일 변경을 실제로 수행하는 idempotent task가 들어 있다. 각 단계는 tag로 나뉘며, 먼저 생성된 명령의 host, tag와 변수를 확인한 뒤 `--check --diff`로 예상 변경을 검토한다. 실제 적용은 검토가 끝난 Ansible 명령에서 `--check`를 제거하여 관리자가 실행한다.

다음 항목은 공통 설정이라도 자동으로 밀어 넣지 않고 별도 승인을 요구한다.

- NVIDIA driver 변경: reboot와 실행 중 GPU workload 조정이 필요하다.
- Kubernetes join: cluster별 token과 controller 승인이 필요하다.
- Kerberos machine keytab: FARM과 LAB의 도메인 가입 방식이 다르고 secret을 안전하게 전달해야 한다.
- Kerberos NFS mount: 실행 중 container와 사용자 session에 영향을 줄 수 있다.

Kerberos/NFS의 상세 점검과 복구 순서는 [Kerberos/NFS 매뉴얼](#)에서 관리한다.

현재 `apply`도 실제 변경 없이 복구 계획만 출력한다.

```
./server-state/bin/server-state apply \
  --hosts farm8 \
  --profile existing-host-drift
```

`apply --execute`는 아직 구현되지 않았으며 실행하면 오류로 중단된다.

3. 공용 inventory와 서버 목록

기본 inventory는 `user-lifecycle/server_info/servers.jsonl`이다. 서버 한 대당 한 줄(JSON)로 host 이름, `server_id`, domain(FARM/LAB), SSH 접속 정보 (`ansible_host`, `ansible_port`, `ansible_user`), 네트워크 인터페이스, OS 버전 등을 담는다. 이 파일은 사람이 손으로 쓰지 않는다. `user-lifecycle/server_info/generate_servers_jsonl.py`가 Ansible `setup` facts(각 서버에서 실제로 수집한 정보)와 고정 topology 정보를 합쳐서 만든 결과물이다.

이 파일은 원래 `user-lifecycle`(사용자·container 생성)이 관리하던 서버 목록이다. `server-state`는 점검할 서버 목록이 필요할 때 이 파일을 그대로 읽어서 쓰고("공용" inventory), 자신만의 목록을 별도로 만들지 않는다. IP, SSH port와 논리 server ID를 두 군데에서 따로 관리하면 서로 달라지는 문제가 생기기 때문이다. `server-state`의 `script/inventory.py`가 이 파일을 읽어서 `--hosts` selector에 맞는 서버를 골라낸다.

```
./server-state/bin/server-state list-hosts --hosts all
./server-state/bin/server-state list-hosts --hosts farm
./server-state/bin/server-state list-hosts --hosts farm8,lab10
```

selector는 `all`, `farm`, `lab`, 개별 host 이름과 여러 host 조합을 지원한다.

4. 공통 설정 추가 방법

1. 설정을 실제로 관리할 모듈을 정한다.
2. `config/profiles.yml` 에 작은 profile을 추가한다.
3. 부작용 없이 읽을 수 있는 항목만 `checks` 에 둔다.
4. 복구는 가능하면 Ansible `--check --diff` 명령부터 제공한다.
5. driver, cluster join, keytab과 mount처럼 위험한 작업은 `manual` 또는 `gated` 로 둔다.
6. 모든 서버에 필요한 설정이면 `new-host-bootstrap` 과 `existing-host-drift` 에 모두 추가한다.
7. profile 순서와 서버별 변수 치환 test를 추가한다.

5. 테스트

```
cd /home/jy/server_manage/server-state
python3 -m unittest discover -s tests -v

./bin/server-state --format json check \
  --hosts farm8 \
  --profile existing-host-drift

./bin/server-state plan \
  --hosts lab10 \
  --profile new-host-bootstrap
```

테스트와 dry-run 출력에서는 실제 운영 서버를 변경하지 않는다.

user-lifecycle

원본: md/system/user-lifecycle/index.md, md/system/user-lifecycle/design.md, md/system/user-lifecycle/operations.md

역할: 사용자 identity, 그룹, 포트, 컨테이너와 개인 Kerberos/NFS 준비를 하나의 생명주기로 관리한다.

이 문서는 user-lifecycle 문서의 시작점이다. **설계**는 이 모듈이 왜 중심 역할을 하는지, 디렉터리 구조, 코드 구조, 데이터 모델과 container 생성·Kerberos 흐름을 설명하고, **운영**은 실제 CLI 사용법, 설정과 secret 관리, 테스트와 변경 절차를 설명한다.

문서 구성

문서	핵심 내용
현재 페이지	책임 범위와 문서 안내
설계	디렉터리 지도, 코드 구조, 데이터 모델, container 생성·Kerberos 활성화 흐름
운영	CLI 사용법, 설정과 secret, post action과 생성물, 테스트, 변경 가이드, 운영 안전 수칙

처음 보는 사람은 **설계 문서**에서 이 모듈이 어떤 자원을 하나의 생명주기로 묶고 왜 공개 CLI를 통과해야 하는지 먼저 확인하고, 실제로 container를 생성·삭제·연장하거나 설정을 바꿔야 할 때 **운영 문서**로 이동한다.

user-lifecycle 설계

개요 · 운영

역할: 사용자 identity, 그룹, 포트, 컨테이너와 개인 Kerberos/NFS 준비를 하나의 생명주기로 관리한다.

1. 이 모듈이 중심인 이유

DECS 사용자 컨테이너는 Docker object 하나가 아니다. 다음 자원이 서로 같은 identity와 상태를 가져야 하는 분산 작업이다.

- 도메인별 UID MySQL의 사용자, 그룹, 예약 ID, container와 port record
- FARM/LAB target host의 Docker container
- NAS 또는 LAB storage의 사용자 home
- 선택적인 AD RFC2307 user/group과 membership

- host의 root-only user keytab과 ccache refresh timer
- 생성·삭제·연장 안내 메일, DB backup과 Excel export

`user-lifecycle` 는 이 자원의 순서와 실패 처리를 소유한다. 다른 모듈이 DB에 직접 쓰거나 container 생성 규칙을 복제하면 UID, 포트와 실제 Docker 상태가 어긋나므로 공개 `uidctl` CLI를 통과하는 것이 원칙이다.

2. 디렉터리 지도

경로	상태	핵심 기능
<code>script/uid_manager/</code>	primary	Python CLI, model, DB, service 구현
<code>script/uid_manager/services/</code>	primary	create/delete/extend/sync/cleanup/group use case
<code>script/uid_manager/kerberos/</code>	primary	AD, keytab, ccache, NFS 검증 명령과 path 생성
<code>script/ansible_playbook/</code>	primary	storage/NAS home과 host Kerberos identity 준비
<code>script/tests/</code>	primary	fake runner/repository 기반 회귀 테스트
<code>config/*.example.*</code>	template	DB, email, maintenance, GPU profile, Google, admin 설정 구조
<code>config/*.local.*</code>	secret/local	실제 credential과 운영 override; Git 제외
<code>server_info/servers.jsonl</code>	shared authority	FARM/LAB 서버 주소와 SSH/NAT port inventory
<code>nfs_mysql/</code>	data layer	MySQL image, schema와 server config
<code>docker-compose.yml</code>	data layer	local UID DB service/volume
<code>maintenance/</code>	utility	사용자 삭제, email CSV 갱신, schema migration 등 관리 작업
<code>ad_backup/</code>	operations	AD backup 생성·검증과 timer 설치
<code>legacy/</code>	compatibility	이전 shell 구현; 신규 기능의 기준이 아님
<code>excel_exports/</code>	generated	사용자 export 결과, source가 아님

`legacy/` 를 즉시 삭제하지 않은 이유는 기존 운영 절차의 비교·복구 근거가 필요하기 때문이다. 그러나 새 비즈니스 규칙은 Python service에 추가하고 테스트 가능하게 유지한다.

3. 코드 구조

CLI와 service 분리

`uid_manager/cli.py` 는 argument parsing, config/repository 생성과 결과 출력만 담는다. 실제 규칙은 service class가 소유한다.

CLI	Service	기능
<code>create-container</code>	<code>ContainerCreateService</code>	identity/port/GPU profile 결정, storage/Kerberos, Docker, DB transaction
<code>delete-container</code>	<code>ContainerDeleteService</code>	대상 단일화, Docker 삭제, DB 비활성화/port 정리
<code>extend-container</code>	<code>ContainerExtendService</code>	만료일 검색·변경과 알림
<code>expired-cleanup</code>	<code>ExpiredCleanupService</code>	만료 대상 계획 또는 실제 정리
<code>sync-containers</code>	<code>ContainerSyncService</code>	DB record와 local Docker 상태 비교·재생성 계획
<code>manage-group</code>	<code>GroupManagementService</code>	AD/DB group과 membership 관리

이 분리는 CLI 외의 timer/test에서도 동일한 규칙을 호출하고, fake repository와 recording runner로 production 접속 없이 계획을 검증하기 위한 것이다.

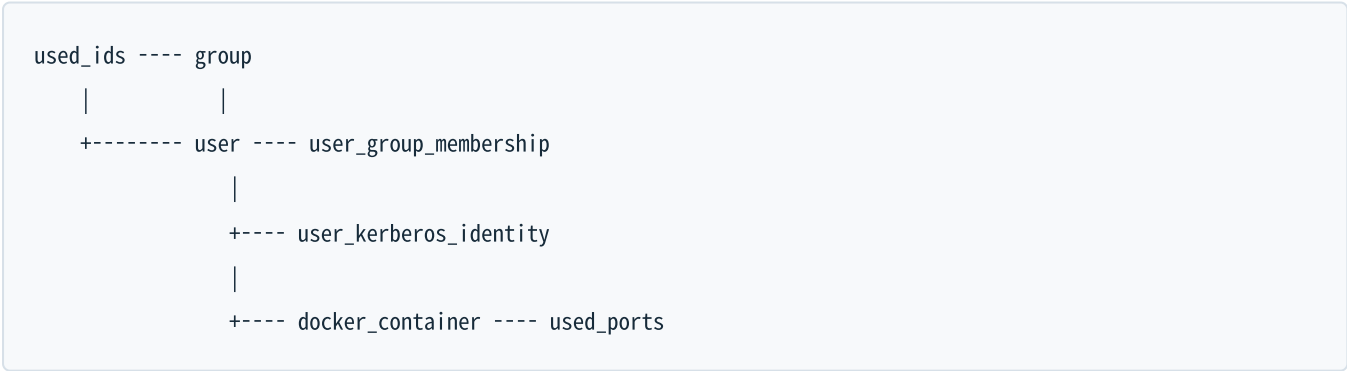
Port와 runner 추상화

`LocalRunner` 와 `AnsibleRunner` 는 subprocess와 원격 shell/playbook 호출을 한 곳에 모은다. `RecordingRunner` 는 실행 대신 command를 기록한다. `PortMapping` 은 host port, container port와 purpose를 함께 보존하여 단순 문자열 port가 SSH/Jupyter/VNC 중 무엇인지 알지 않게 한다.

`OperationPlan` 은 facts, 단계와 명령을 렌더링한다. `--dry-run` 이 실제 실행 경로와 같은 준비 로직을 통과하므로 문서용 예상 명령과 실제 입력 검증이 달라지는 것을 줄인다.

4. 데이터 모델과 권위

MySQL schema의 주요 관계:



데이터	의미
<code>used_ids</code>	UID/GID 숫자의 전역 예약
<code>group</code>	group 이름과 GID
<code>user</code>	실명, username, UID/GID, 연락처
<code>user_group_membership</code>	supplemental/primary group 관계

데이터	의미
<code>user_kerberos_identity</code>	AD realm/SID/RFC2307과 최근 NAS/NFS 관측값
<code>docker_container</code>	image, server, GPU profile, 만료, 생성자와 활성 상태
<code>used_ports</code>	host port, 용도와 container record 연결

foreign key와 unique index를 쓰는 이유는 application 검증만으로 막기 어려운 중복 UID/GID, container와 port 충돌을 DB에서도 거부하기 위해서다. FARM과 LAB은 DB endpoint가 다를 수 있으므로 `AppConfig`가 domain별 host를 선택한다.

5. Container 생성 흐름

5.1 준비와 identity 채택

`ContainerCreateService.prepare()`는 이름, domain/server, 날짜, image와 port를 검증한 뒤 UID/GID source를 결정한다. 신규 사용자의 UID 우선순위는 다음과 같다.

1. DB에 기존 사용자가 있으면 DB UID/GID
2. 운영자가 명시한 `--uid / --gid`
3. Kerberos 모드에서 기존 AD RFC2307 identity
4. 기존 storage home의 숫자 owner
5. 모두 없으면 DB의 다음 사용 가능한 ID

기존 자원을 무조건 새 ID로 덮지 않고 채택하는 이유는 이미 소유권이 있는 NFS home이나 AD object를 고아로 만들지 않기 위해서다. 여러 source가 서로 다른 값을 말하면 자동 보정하지 않고 validation error로 중단한다.

port는 DB 예약과 실제 target host Docker publish port를 함께 확인한다. 기본 SSH/Jupyter, 선택적 VNC와 추가 container port를 배정하거나 `--fixed-port-mappings`로 host:container[:purpose]를 명시할 수 있다.

같은 단계에서 GPU profile도 결정된다. 여러 GPU 모델이 섞여 있는 서버(현재는 `GPU_PROFILE_REQUIRED_SERVERS`에 등록된 LAB5만 해당)는 `--gpu-profile`로 사용할 GPU 묶음을 선택해야 한다. `AppConfig.resolve_gpu_profile()`이 `config/gpu_profiles.local.json` 카탈로그에서 서버별 profile 이름(예: `a6000`, `pro5000`)을 실제 GPU UUID 목록으로 바꾸고, 이 값이 나중에 Docker 실행 시 `--gpus device=<uuid,...>`로 전달된다. 카탈로그에 없는 서버는 GPU 종류가 하나뿐이라고 보고 기본값 `all`(모든 GPU 노출)을 쓴다. 선택된 profile 이름은 `docker_container.gpu_profile`에 저장되어, 이후 `sync-containers`가 DB와 실제 Docker `--gpus` 설정의 drift를 비교하는 기준이 된다.

5.2 실행 순서

입력 검증과 OperationPlan

|

v

target image inspect/pull

|

v

storage home + optional AD/keytab/ccache 준비

|

v

NSS/idmap/실제 NFS write 검증

|

v

docker run + inspect/port 검증

|

v

DB user/group/identity/container/port 단일 transaction

|

v

email + DB backup + export

storage와 Kerberos write 검증을 Docker/DB 확정 전에 두는 이유는 사용자가 로그인했지만 home에 쓸 수 없는 반쪽 container를 만들지 않기 위해서다. Docker 생성 뒤 DB write는 한 transaction으로 묶는다. DB 단계가 실패하면 transaction을 rollback하고 방금 만든 container 정리를 시도하여 실제 상태와 record가 갈라지는 시간을 줄인다.

`--no-db-record`는 특수 검증용으로 Docker를 만들되 DB user/group/container/port write를 생략한다. 일반 운영 생성에는 사용하지 않는다. DB에 없는 container는 GPU user attribution, 만료 정리와 sync에서 정상 관리 대상이 아니기 때문이다.

5.3 domain별 storage

일반 LAB root_squash 경로는 storage server에서 실제 export home을 만들고 선택한 UID/GID로 owner를 설정한 뒤, target host에 mount된 `/home/tako<N>/share/user-share`를 container `/home`에 bind한다.

FARM은 NAS의 user-share 구조를 사용한다. Kerberos가 아닌 모드도 target host root가 NFS root_squash 때문에 직접 owner를 바꾸려 하지 않고 storage/NAS 소유 playbook을 통해 준비한다.

6. Kerberos 활성화 흐름

6.1 공통 보안 모델

```

AD user/private group + RFC2307
  |
  v
root-only keytab on target host
  |
  v
systemd refresh -> /run/user/<uid>/krb5cc
  |
  v
container receives ccache only

```

사용자 password를 DB나 container에 저장하지 않는다. AD에서 keytab을 export한 뒤 target host의 `/etc/decs-krb/keytabs/`에 mode `0400`으로 설치한다. refresh service는 ticket을 renew하거나 만료 여유가 부족하면 keytab으로 재발급하고, container에는 ccache만 전달한다.

keytab rotation은 `--rotate-kerberos-keytab`처럼 명시적으로 요청한다. 정상 container 재생성이나 만료 연장이 credential rotation을 뜻하지 않게 하기 위해서다.

6.2 FARM

FARM 흐름은 AD user/group과 RFC2307을 보장하고 NAS mapping과 home을 준비한 뒤 target host에서 해당 UID와 ccache로 실제 NFS write를 수행한다.

공유 NAS GSS/idmap service restart는 기본적으로 꺼져 있다. 새 identity의 cache 문제가 의심되더라도 NAS service 재시작은 모든 client의 GSS context를 뚫을 수 있으므로 명시적 유지보수 옵션으로만 허용한다.

6.3 LAB

LAB Kerberos PoC/지원 흐름은 LAB Samba AD, storage의 전용 Kerberos export, target host의 NSS/idmap과 NFSv4.1을 함께 확인한다. storage와 client의 NFSv4 idmap domain이 다르면 owner가 `nobody`로 보이거나 `chgrp`가 실패할 수 있다. 따라서 ticket 성공만으로 완료하지 않고 숫자 owner와 실제 write를 검증한다.

6.4 DB에 남기는 Kerberos 정보

DB에는 password/keytab이 아니라 다음 비밀이 아닌 identity metadata만 둔다.

- AD username, realm, NetBIOS domain
- domain/object SID
- AD `uidNumber`, `gidNumber`
- 최근 NAS internal UID/GID
- 최근 NFS에서 본 UID/GID와 검증 시각

SID와 최근 관측값을 보존하면 같은 username이 재생성되었거나 mapping이 달라진 상황을 단순 문자열 일치보다 안전하게 발견할 수 있다.

user-lifecycle 운영

개요 · 설계

1. 주요 CLI 사용법

설치/실행 위치:

```
cd /home/jy/server_manage/user-lifecycle/script
python3 -m pip install -e .
uidctl --help
```

생성 dry-run

```
uidctl create-container \
  --name '홍길동' \
  --username hong \
  --server-id FARM8 \
  --expiration-date 2026-12-31 \
  --image decs \
  --version cuda12.5-tf2.20-ubuntu22.04-260706 \
  --created-by admin \
  --email hong@example.org \
  --phone 000-0000-0000 \
  --dry-run
```

Kerberos/VNC는 필요할 때 명시한다.

```
uidctl create-container ... --enable-kerberos --enable-vnc --dry-run
```

GPU 모델이 여러 종류인 서버(현재는 LAB5)는 `--gpu-profile` 로 사용할 GPU 묶음을 선택한다. 생략하면 서버의 기본 profile이 쓰이고, 그 서버가 `GPU_PROFILE_REQUIRED_SERVERS` 에 등록되어 있는데 카탈로그에 없는 profile을 요청하면 오류로 중단된다.

```
uidctl create-container ... \  
  --server-id LAB5 \  
  --gpu-profile a6000 \  
  --dry-run
```

삭제

DB record 검색 조건으로 대상이 하나인지 먼저 확인하고 dry-run한다.

```
uidctl delete-container \  
  --server-id FARM8 \  
  --container-name hong \  
  --dry-run
```

만료 연장과 정리

연장은 기본적으로 계획만 만들고 실제 변경에 `--apply` 가 필요하다.

```
uidctl extend-container \  
  --username hong \  
  --expiration-date 2027-06-30  
  
uidctl extend-container \  
  --username hong \  
  --expiration-date 2027-06-30 \  
  --apply
```

만료 정리도 대상 날짜와 domain을 제한해 먼저 dry-run한다.

```
uidctl expired-cleanup \  
  --today 2026-07-17 \  
  --domains FARM,LAB \  
  --dry-run
```

sync와 group

```
uidctl sync-containers --domain FARM --dry-run
uidctl manage-group show --domain FARM --group research
uidctl manage-group add-user \
  --domain FARM \
  --group research \
  --user hong \
  --dry-run
```

`--force`, `--auto-delete`, 실제 group delete는 영향 범위를 확인한 뒤 사용한다. `sync-containers` 는 DB의 `gpu_profile` 과 실제 Docker `--gpus` 설정이 어긋나는 경우도 함께 찾아낸다.

2. 설정과 secret

공개 template:

- `config/db_config.example.env`
- `config/email_config.example.env`
- `config/daily_maintenance.example.env`
- `config/google-client.example.json`
- `config/reminder_admins.example.txt`
- `config/gpu_profiles.example.json`
- `ad_backup/config.example.env`

실제 값은 대응하는 `.local.*` 또는 ignored 운영 파일에 둔다. 문서, log, OperationPlan과 exception에 DB password, SMTP credential, Google secret, AD password, keytab 내용이 나타나지 않도록 command redaction을 유지한다.

`config/gpu_profiles.local.json` 은 서버별 GPU profile 이름과 실제 GPU UUID 목록의 카탈로그다.

`config/db_config.local.env` 의 `GPU_PROFILES_FILE` 이 이 파일의 경로를, `GPU_PROFILE_REQUIRED_SERVERS` 가 profile 지정을 강제할 서버 목록을 정한다. GPU UUID 자체는 credential이 아니지만, 실제 서버의 GPU 구성과 어긋나면 엉뚱한 GPU가 배정되므로 하드웨어를 바꿀 때마다 함께 갱신해야 한다.

`config/network_topology.json` 과 `server_info/servers.jsonl` 은 secret 저장소가 아니다. 접근 경로에 필요한 주소와 port만 넣고 password/private key는 외부 SSH/Ansible 설정이 관리한다.

3. Post action과 생성물

성공한 생명주기 작업 뒤 `PostActions` 가 이메일, DB backup과 export 도구를 호출한다. 핵심 transaction과 알림을 분리한 이유는 메일 실패가 identity/DB 일관성을 되돌리는 근거가 되지 않기 때문이다. 반대로 DB transaction이 실패했는데 성공 메일을 보내지 않도록 실행 순서는 transaction 뒤에 둔다.

`excel_exports/`, `AD backups/`, `server_info`의 생성된 raw 정보는 운영 artifact다. source code처럼 수정하거나 매 뉴얼의 권위 있는 입력으로 삼지 않는다. 보존·암호화·삭제 주기는 별도 운영 정책에 따라야 한다.

4. 테스트

```
cd /home/jy/server_manage/user-lifecycle/script
python3 -m unittest discover -s tests -v
```

또는 프로젝트 test runner:

```
python3 tests/run_tests.py
```

중요 회귀 범위:

- 자동/고정 port 할당과 중복 거부
- DB/AD/storage 기존 identity 채택 우선순위
- GPU profile 카탈로그 해석과 필수 서버의 fail-closed 동작
- create 실패 시 DB rollback과 container cleanup
- dry-run에서 원격/DB 변경이 없는지
- FARM/LAB Kerberos command와 path 차이
- group 삭제/primary 변경의 안전 gate
- sync가 DB에 없는/멈춘 container와 GPU profile drift를 구분하는지
- command/log에서 password redaction

shell legacy test는 호환성 확인용이며 Python service test를 대체하지 않는다.

5. 변경 가이드

새 lifecycle command

1. request/result 또는 record를 `models.py`에 정의한다.
2. 비즈니스 순서를 `services/`의 class로 구현한다.
3. DB query는 `Repository` protocol과 `MySQLRepository`에 추가한다.
4. 외부 명령은 runner를 통하고 secret redaction을 적용한다.
5. CLI에는 parsing과 dependency 조립만 추가한다.
6. fake repository/runner를 사용한 dry-run, success와 partial failure test를 작성한다.

schema 변경

- 기존 운영 DB에 idempotent하게 적용되는 migration/ensure path를 준비한다. (예: `maintenance/migrate_add_gpu_profile.sql` 처럼 컬럼 존재 여부를 먼저 확인한다.)
- unique/foreign key와 기존 데이터 backfill 순서를 검토한다.
- FARM/LAB DB를 독립적으로 적용·검증한다.
- exporter가 읽는 column과 query 호환성을 `monitoring` 에서 함께 확인한다.

Kerberos 변경

- password/keytab이 DB, plan과 log에 들어가지 않는지 확인한다.
- DB, AD, NAS/NFS와 container identity 불변 조건을 test한다.
- 공유 NAS service restart를 정상 경로에 추가하지 않는다.
- 운영 policy 변경이면 `kerberos-nfs/docs/farm.md` 또는 `docs/lab.md` 를 갱신한다.

6. 운영 안전 수칙

- 모든 destructive 명령은 domain, server와 대상 container가 하나인지 확인한다.
- 실제 생성 전 `--dry-run` 의 identity source, mount, image, port와 GPU profile을 검토한다.
- 기존 AD 또는 storage identity와 요청 UID/GID 충돌을 강제로 덮지 않는다.
- DB와 실제 Docker 상태가 어긋나면 원인을 확인하고 `sync --auto-delete` 를 바로 사용하지 않는다.
- keytab과 사용자 password를 container나 backup export에 포함하지 않는다.
- GPU profile 카탈로그가 실제 서버의 GPU 구성과 다르면 먼저 카탈로그를 맞추고, DB의 `gpu_profile` 값을 임의로 고쳐 맞추지 않는다.
- `legacy/` 를 수정해 신규 규칙을 우회하지 않는다.
- DB 없는 test container는 이름, 수명과 정리 책임을 명확히 한다.